

现代 C++ 数据结构教程

基于张铭、王腾蛟、赵海燕《数据结构与算法》重编
高等教育出版社 2008 年版教学内容的现代化讲义

目录

写给学生	i
如何使用本教程	i
各章一览	i
阅读约定	ii
验证与边界	ii
 第 1 章 概论	 1
1.1 问题求解	1
1.2 数据结构	9
1.3 算法	10
1.4 算法分析	10
 第 2 章 线性表	 13
先跑一遍	13
2.1 线性表的概念	14
2.2 顺序表	15
与原书的对照	20
2.3 链表	20
2.4 线性表实现方法的比较	24
 第 3 章 栈与队列	 25
先跑一遍	25
3.1 栈	26
与原书的对照	45
3.2 队列	46
3.3 栈与队列的比较	48
 第 4 章 字符串	 51
先跑一遍	51
4.1 字符串的基本概念	52
4.2 字符串的存储结构和实现	53
4.3 字符串的模式匹配	57
与原书的对照	62
 第 5 章 二叉树	 63
5.1 二叉树的概念	63

5.2 二叉树的周游	65
5.3 二叉树的存储结构	66
5.4 二叉搜索树	71
5.5 堆与优先队列	73
5.6 Huffman 树及其应用	76
第 6 章 树	79
6.1 树的定义和基本术语	79
6.2 树的链式存储结构	80
6.3 树的顺序存储结构	87
6.4 K 叉树	87
第 7 章 图	89
7.1 图的定义和基本术语	89
7.2 图的抽象数据类型	89
7.3 图的存储结构	89
7.4 图的周游	90
7.5 最短路径	92
7.6 最小生成树	94
第 8 章 内部排序	97
8.1 排序问题的基本概念	97
8.2 插入排序	97
8.3 选择排序	99
8.4 交换排序	100
8.5 归并排序	100
8.6 分配排序和索引排序	101
8.7 排序算法的时间代价	101
第 9 章 文件管理与外部排序	103
9.1 主存储器和外存储器	103
9.2 文件的组织和管理	103
9.3 外排序	103
第 10 章 检索	111
10.1 基于线性表的检索	111
10.2 集合的检索	112
10.3 散列方法	113
第 11 章 索引技术	119
11.1 线性索引	119

11.2 静态多分树	119
11.3 倒排索引	119
11.4 B 树与 B+ 树	120
11.5 位图、签名和红黑树	120
11.6 练习路径	120
第 12 章 高级数据结构	121
12.1 多维数组	121
12.2 广义表和存储管理	121
12.3 Trie 结构和 Patricia 树	124
12.4 改进的二叉搜索树	124
原书勘误	127
怎么读	127
编译不过	127
能编译、跑起来是错的	128
书内自相矛盾	129
反复出现、但单独一条通常还能「跑两下」	129
曾经误判、已撤回	130
还没收进本表的	130

写给学生

基于《数据结构与算法》（张铭、王腾蛟、赵海燕编著，高等教育出版社，2008）重编。

保留原书的章节脉络、数据结构与算法思想；全部示例统一为可编译、可测试的 C++17。

如何使用本教程

每一段印在书上的 C++ 都与配套源码逐字一致。建议按第 1 至第 12 章顺序阅读：先理解问题，再运行该章的示例程序，最后对照实现。

建议的学习顺序是第 1 至第 10 章；第 11 章给出索引技术导读；第 12 章收束内存复用与动态规划。每章先理解抽象模型，再阅读实现，最后运行对应单元的测试。所有提取操作（例如 pop、dequeue）都用 `std::optional` 表达正常的空状态；参数、容量或权重不合法则抛标准异常。

```
# 编译并运行全书的现代实现与测试
python3 tools/check_code.py --allow-degraded

# 检查书稿中的代码与源码是否仍逐字一致
python3 tools/check_doc.py
```

单章可以先跑那个单元的 `demo.cpp`。第 1 章的编译命令在 `ch01-adt.md` 里逐项解释过；后面各章的命令形状相同，只改 `-I` 路径和源文件。

各章一览

1. 概论：1.1 问题求解，1.2 数据结构，1.3 算法，1.4 算法分析。
2. 线性表：2.1 概念，2.2 顺序表，2.3 链表，2.4 比较。
3. 栈与队列：3.1 栈，3.2 队列，3.3 比较。
4. 字符串：4.1 概念，4.2 存储与实现，4.3 模式匹配。
5. 二叉树：5.1 概念，5.2 周游，5.3 存储，5.4 BST，5.5 堆，5.6 Huffman。
6. 树：6.1 定义，6.2 链式存储，6.3 顺序存储，6.4 K 叉树。
7. 图：7.1–7.3 概念与存储，7.4 周游，7.5 最短路，7.6 最小生成树。
8. 内部排序：8.1 概念，8.2–8.6 各类排序，8.7 时间代价。
9. 文件管理与外部排序：9.1–9.2 外存与文件，9.3 置换选择与选择树。
10. 检索：10.1 线性表检索，10.2 集合，10.3 散列。
11. 索引技术：11.1–11.6 线性/倒排/B 树/位图/红黑树。

12. 高级数据结构：12.1 多维数组，12.2 广义表与存储，12.3 Trie，12.4 改进 BST。

对照 2008 年纸书时，编译级硬伤与算法错见书末「原书勘误」。

阅读约定

- 算法思想优先。没有用标准库容器或排序算法替代本章要教的核心结构或算法。
- 资源所有权显式。手写数组和结点链使用 RAII、析构、深复制和移动语义管理寿命。
- 测试是正文的一部分。每个实现目录的 `test.cpp` 都覆盖其声明的原书清单；运行测试比 阅读一段未经执行的伪代码更可靠。
- 风险如实保留。递归树周游与析构仍适合教学，但极深退化结构存在栈溢出风险；详见 `collab/UNVERIFIED-RISKS.md`。

验证与边界

本教程覆盖原书 105 条算法/代码清单。`code/` 下的每个单元都有对应的 `unit.json`、现代实现、原书问题证据和可执行测试；书稿的 C++ 代码由工具逐字同步。完整的边界条件、未覆盖故障路径与 递归深度风险见 `collab/UNVERIFIED-RISKS.md`。

本教程采用 C++17，侧重数据结构的实现与算法语义。它不提供文件系统、数据库页缓存、并发控制 或跨平台性能承诺；这些属于使用数据结构的系统层设计。

第 1 章 概论

数据结构刻画信息及其关系，算法描述求解过程。二者互相依赖：结构选错了，算法再巧也补不回来。本章先用股市传言把问题说到能写程序，再补回逻辑结构、存储映射、抽象数据类型和渐进分析——这些原书没错，是后面十一章共用的词汇。

1.1 问题求解

程序是指令的组合，算法是求解过程的逻辑抽象，数据结构是数据及其关系在计算机里的反映。Wirth 的「程序 = 数据结构 + 算法」说的就是这件事。求解路径通常是：把现实问题抽象成模型，选定逻辑结构与存储方式，再设计算法并写成可运行的程序。

1.1.1 问题描述：股市的传言

本章把原书 1.1 的“股市传言”问题写成一个可直接调用的程序。它不是在找“边最多”的经纪人，而是在找一个起点：从这个人出发能把消息传给所有人，并且最慢的那条最短传播路径尽可能短。

1.1.2 问题分析和抽象

把 B1 到 B5 看成五个顶点。一条 $B_i \rightarrow B_j$ 边表示消息能从 B_i 直接传给 B_j ；边权是传播时间。没有路径就记为 ∞ ，表示消息永远传不到那里。B3 是唯一可以到达全部经纪人的起点。

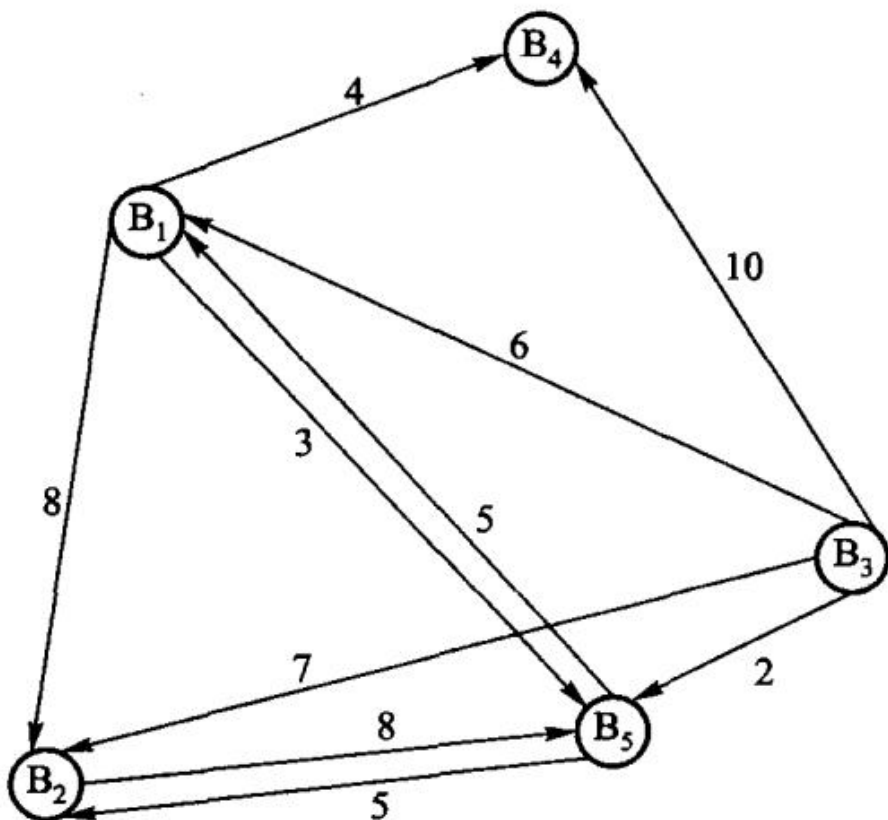


图 1.1 经纪人间的消息传递图

原书算法 1.1 的工作可以拆成三步：

1. Floyd 算法算出任意两人间的最短传播时间。
2. 对每一个候选起点，找出“从它出发到每个人的最短时间”中最慢的一项。
3. 从这些最大值中选最小的一个；如果某行仍有不可达的 ∞ ，该起点不能胜任。

原书中的 D 对应现代代码里的 `shortest` 矩阵，`max[i]` 对应每一行的 `largest_distance`，`pos` 对应 `best_source()` 的返回值。下标从 0 开始，因此 B_3 的返回值是 2。

为什么要取“最慢的一项”

假设从 B_3 开始，Floyd 算出的最短传播时间为：到 B_1 是 6，到 B_2 是 7，到自身是 0，到 B_4 是 10，到 B_5 是 2。消息要“传遍所有人”，必须等最慢的 B_4 收到消息，因此 B_3 的完成时间是这五个数里的最大值 10，而不是它们的和，也不是最小值。

把每个人都当一次起点，结果如下。 ∞ 表示至少有一人永远收不到消息，所以这一行没有可用的完成时间。

起点	到 B1	到 B2	到 B3	到 B4	到 B5	最慢的最短时间	是否可选
B1	0	∞	∞	4	3	∞	否
B2	13	0	∞	17	8	∞	否
B3	6	7	0	10	2	10	是
B4	∞	∞	∞	0	∞	∞	否
B5	5	5	∞	9	0	∞	否

因此答案是 B3。这个目标常称为“最小化最大距离”：不是选离某一个人最近的起点，而是选让 最后一个收到消息的人也尽可能早收到消息的起点。

1.1.3 数据结构和算法设计

下面是完整可运行的程序。

下面是完整可运行的程序。RumorNetwork(5) 创建 B1 至 B5；add_route(from, to, cost) 添加一条 有向传播路径，三个参数均从 0 开始编号。best_source() 返回 std::optional<std::size_t>：有解时保存起点下标，无解时是 std::nullopt。

源码文件在这里：可运行示例、数据结构实现、测试用例。demo.cpp 负责输入/输出；modern.hpp 只负责数据和计算，这就是把“怎么展示”与“怎么算”分开。

```
#include "modern.hpp"

#include <iostream>

int main() {
    // B1 ... B5 对应下标 0 ... 4。
    dsa::adt::RumorNetwork network(5);
    network.add_route(0, 3, 4);    // B1 -> B4, 耗时 4
    network.add_route(0, 4, 3);    // B1 -> B5, 耗时 3
    network.add_route(1, 4, 8);    // B2 -> B5, 耗时 8
    network.add_route(2, 0, 6);    // B3 -> B1, 耗时 6
    network.add_route(2, 1, 7);    // B3 -> B2, 耗时 7
    network.add_route(2, 3, 10);   // B3 -> B4, 耗时 10
    network.add_route(2, 4, 2);    // B3 -> B5, 耗时 2
    network.add_route(4, 0, 5);    // B5 -> B1, 耗时 5
    network.add_route(4, 1, 5);    // B5 -> B2, 耗时 5

    const auto source = network.best_source();
    if (source) {
        std::cout << " 最佳传播起点是 B" << *source + 1 << '\n';
    } else {
```

```
std::cout << " 不存在能到达全部经纪人的起点\n";
}
}
```

在仓库根目录运行：

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch01/adt code/ch01/adt/demo.cpp -o /
↳ tmp/rumor-demo
/tmp/rumor-demo
```

第一行是“编译”，第二行是“运行刚刚编译出的程序”。逐项解释如下：

片段	含义
c++	C++ 编译器命令。在 macOS 上通常指 Clang；Linux 上也可能指 GCC。
-std=c++17	按 C++17 语言规则编译；本教程的 <code>std::optional</code> 需要 C++17。
-Wall -Wextra	打开常见和额外的编译器警告，例如未使用变量或可疑转换。
-Werror	把警告当作错误；用于学习时可及早发现问题。初学调试时可暂时去掉它。
-Icode/ch01/adt	加一个头文件搜索目录，因此 <code>#include "modern.hpp"</code> 能找到 <code>code/ch01/adt/modern.hpp</code> 。
code/ch01/adt/demo.cpp	要编译的源文件。它包含 <code>main()</code> ，所以可以成为可执行程序。
-o /tmp/rumor-demo	指定输出文件名为 <code>/tmp/rumor-demo</code> 。/ <code>tmp</code> 是临时目录，重启后文件可能消失。
/tmp/rumor-demo	运行该可执行文件，打印程序结果。

如果系统提示 `c++: command not found`，需要先安装 C++ 编译器；如果想把程序留在当前目录，可以把 `-o /tmp/rumor-demo` 改成 `-o rumor-demo`，再运行 `./rumor-demo`。

输出是：

```
最佳传播起点是 B3
```

如果删除 B3 的某条出边，例如 `network.add_route(2, 1, 7)`，B3 将无法到达 B2；当不存在任何 能覆盖全部顶点的起点时，`best_source()` 返回空值，程序会输出“不存在能到达全部经纪人的起点”。

可以先只改边和权值，再运行同一条命令观察结果。例如把 B3 -> B4 的时间从 10 改为 1，答案 仍是 B3，只是它的最慢传播时间从 10 缩短为 7。

实现导读

先只关注三个公开操作：构造函数建立距离矩阵，add_route 填入直接边，best_source 计算答案。best_source 内部复制一份矩阵，因而多次调用不会改变原始网络。下面按代码出现顺序解释。

预处理、头文件与命名空间

#pragma once 告诉编译器：同一个头文件即使被间接包含多次，也只处理一次，避免类定义重复。

头文件	本程序用它做什么
<cstdint>	提供 std::size_t，表示非负的大小或下标。
<limits>	提供 std::numeric_limits<int>::max()，取得 int 可表示的最大值。
<optional>	提供 std::optional，表达“可能没有答案”。
<stdexcept>	提供 std::invalid_argument，用于报告非法参数。
<vector>	提供动态数组 std::vector，这里用它保存二维距离矩阵。

namespace dsa::adt { ... } 把名字放入 dsa::adt 命名空间。完整类名因此是 dsa::adt::RumorNetwork，可避免项目中另一个人也定义 RumorNetwork 时发生重名。代码块中的 // >>> adt 与 // <<< adt 只是本书稿的同步标记，不参与 C++ 逻辑。

类、常量和构造函数

class RumorNetwork 定义一种新类型。public：以下是调用者可以使用的接口；private：以下是 实现细节，调用者不能直接改写。

```
static constexpr int infinity = std::numeric_limits<int>::max() / 4;
```

static 表示 infinity 属于类本身，而不是每个对象各存一份；constexpr 表示编译期常量。取最大值的四分之一而非最大值，是为了计算 a + b 时仍留有余量，避

免整数溢出。

```
explicit RumorNetwork(std::size_t people)
    : distance_(people, std::vector<int>(people, infinity)) { ... }
```

这是构造函数：RumorNetwork network(5) 会调用它。冒号后的部分叫成员初始化列表，先创建 distance_，再进入花括号。它构造一个 5 行、每行 5 列的整数矩阵，初始全为 infinity。随后循环把对角线改为 0，因为一个人到自己不需要传播时间。矩阵的第 from 行、第 to 列总是表示从 from 到 to 的当前已知最短时间。

添加一条边

```
void add_route(std::size_t from, std::size_t to, int cost)
```

这三个参数分别是起点编号、终点编号和直接传播时间。第一段 if 检查编号是否越界、时间是否为负；不满足题目定义时抛出 std::invalid_argument，调用者可用 try/catch 处理。第二段 if 只在新边更短时更新矩阵，所以即使重复添加同一方向的边，也保留较小的时间。

Floyd 三重循环

auto shortest = distance_; 复制输入矩阵。auto 让编译器自动推断类型，这里实际是 std::vector<std::vector<int>>。

三层循环的含义不是“随便循环三次”，而是按中转站逐步扩大可用路径：

```
for each via:      允许路径经过 via
  for each from:   固定起点 from
    for each to:   固定终点 to
      比较 原来的 from->to 与 from->via->to
```

条件 shortest[from][via] != infinity && shortest[via][to] != infinity 先确认两段都可达。若 from -> via -> to 的总时间更小，就更新 shortest[from][to]。例如 B2 到 B4 没有直接边，但 B2→B5 是 8、B5→B1 是 5、B1→B4 是 4，所以 Floyd 最终得到 B2→B4 是 8 + 5 + 4 = 17。

从最短路矩阵选答案

```
std::optional<std::size_t> result;
int smallest_eccentricity = infinity;
```

默认构造的 result 是空值，表示“还没有找到合格起点”。smallest_eccentricity

保存目前见过的 最小完成时间。这里的 eccentricity（离心率）就是某起点到全部顶点的最短距离中的最大值。

外层循环每次处理一行，即一个候选起点；内层循环扫描该行。三目表达式 `distance > largest_distance ? distance : largest_distance` 等价于“若新值更大就采用新值，否则保留 旧值”，最终得到这行的最大值。若它比当前最佳值小，就同时更新最佳时间与 `result`。

任何一行含 `infinity` 时，该行最大值也是 `infinity`，不会优于有限答案；若所有行都是 `infinity`，`result` 保持为空，函数返回 `std::nullopt`。[[nodiscard]] 提醒编译器：调用者不应 无意丢弃这个可能为空的重要返回值。

私有数据成员

```
std::vector<std::vector<int>> distance_;
```

外层 `vector` 是行，内层 `vector<int>` 是一行中的列，合起来是二维矩阵。末尾下划线是本教程的 约定，表示私有成员；它与函数参数 `distance` 或局部变量 `shortest` 不会混淆。

时间复杂度为 $O(V^3)$ ，空间复杂度为 $O(V^2)$ 。这适合顶点数较少、需要比较所有起点的示例；大图 通常应根据图的稀疏度和查询需求选择其他最短路径算法。

现代实现

```
#pragma once

#include <cstddef>
#include <limits>
#include <optional>
#include <stdexcept>
#include <vector>

namespace dsa::adt {

// >>> adt
class RumorNetwork {
public:
    static constexpr int infinity = std::numeric_limits<int>::max() / 4;

    explicit RumorNetwork(std::size_t people)
        : distance_(people, std::vector<int>(people, infinity)) {
        for (std::size_t person = 0; person < people; ++person) {
            distance_[person][person] = 0;
        }
    }
};
```

```

    }
}

void add_route(std::size_t from, std::size_t to, int cost) {
    if (from >= distance_.size() || to >= distance_.size() || cost < 0) {
        throw std::invalid_argument("route");
    }
    if (cost < distance_[from][to]) {
        distance_[from][to] = cost;
    }
}

[[nodiscard]] std::optional<std::size_t> best_source() const {
    auto shortest = distance_;
    for (std::size_t via = 0; via < shortest.size(); ++via) {
        for (std::size_t from = 0; from < shortest.size(); ++from) {
            for (std::size_t to = 0; to < shortest.size(); ++to) {
                if (shortest[from][via] != infinity &&
                    shortest[via][to] != infinity &&
                    shortest[from][to] > shortest[from][via] + shortest[via][to])
↪ {
                    shortest[from][to] = shortest[from][via] + shortest[via][to];
                }
            }
        }
    }

    std::optional<std::size_t> result;
    int smallest_eccentricity = infinity;
    for (std::size_t from = 0; from < shortest.size(); ++from) {
        int largest_distance = 0;
        for (int distance : shortest[from]) {
            largest_distance = distance > largest_distance ? distance :
↪ largest_distance;
        }
        if (largest_distance < smallest_eccentricity) {
            smallest_eccentricity = largest_distance;
            result = from;
        }
    }
    return result;
}

private:
    std::vector<std::vector<int>>> distance_;
};
// <<< adt

```

```
} // namespace dsa::adt
```

1.2 数据结构

数据的逻辑结构描述对象之间的关系，不依赖计算机；存储结构是同一套关系在内存或外存上的映射。同一种逻辑结构可以有多种存储方式，后面各章都在做这种选择。

1.2.1 数据的逻辑结构

常见的逻辑结构有四类：集合（元素之间只有「同属」）、线性结构（除两端外每个元素一个前驱一个后继）、树形结构（一个前驱、多个后继）、图形结构（前驱后继都可以多个）。传言问题的经纪人关系是有向带权图。

1.2.2 数据的存储结构

四种基本映射：

1. 顺序：逻辑相邻用地址相邻表示，随机访问 $O(1)$ ，插入删除要搬迁。
2. 链接：结点自带后继指针，插入删除改常数条指针，按位置访问仍须循链。
3. 索引：另造一张「整数 $\rightarrow$ 地址」的表，用空间换定位时间。
4. 散列：由关键码直接算出地址；冲突与装载因子是后话，见第 10 章。

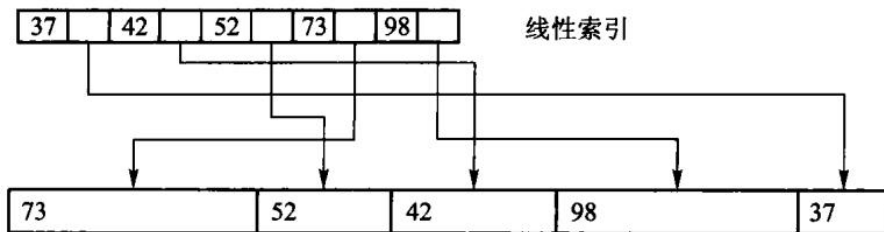


图 1.4 索引示例

1.2.3 抽象数据类型

ADT 是三元组 (D, R, P) ：数据对象、对象上的关系、一组运算。对外只看见运算，看不见存储。原书【代码 1.2】用一个空模板类描述这件事；本书各章把运算直接写在实现类上，对元素类型的要求用 `static_assert` 写在类头——空基类给不了多态，还会带来非虚析构。

1.3 算法

算法是有穷、确定、有效的指令序列，对符合类型的任意输入都能停机并给出正确结果。程序是算法在某种语言里的实现。

1.3.1 算法的概念

四条性质：通用（同类输入都能算）、有效（每一步都能执行）、确定（下一步不模糊）、有穷（不会死循环）。「有穷条指令」并不自动保证有穷步执行。

1.3.2 算法设计

常用路数：穷举、回溯、分治与递归、贪心与动态规划。第 3 章背包是回溯，第 8 章快排和归并是分治，第 7 章 Prim / Kruskal / Dijkstra 是贪心，Floyd 和最优 BST 是动态规划。传言问题选 Floyd，是因为它要任意两点最短路，而且顶点数很小。

1.4 算法分析

比较算法不看墙上的秒数，而看基本操作次数随输入规模 n 怎么长。同一段代码在巨型机和 PC 上差几个数量级，但增长趋势不变。

1.4.1 渐进分析方法

只保留主导项，丢掉常数和低阶项。 $f(n) = n^2 + 100n + \log n + 1000$ 在 n 很大时就是 $\Theta(n^2)$ 。

- $\mathcal{O}$ ：存在 c, N 使 $n \geq N$ 时 $f(n) \leq c g(n)$ ，增长率的上限。
- Ω ：不等式反过来，下限。
- Θ ：上下限同阶。

常见阶： $O(1)$ 、 $O(\log n)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 、 $O(a^n)$ 。指数增长比任何多项式都快： $n = 1000$ 时 2^n 的微秒数已经远超地球年龄。

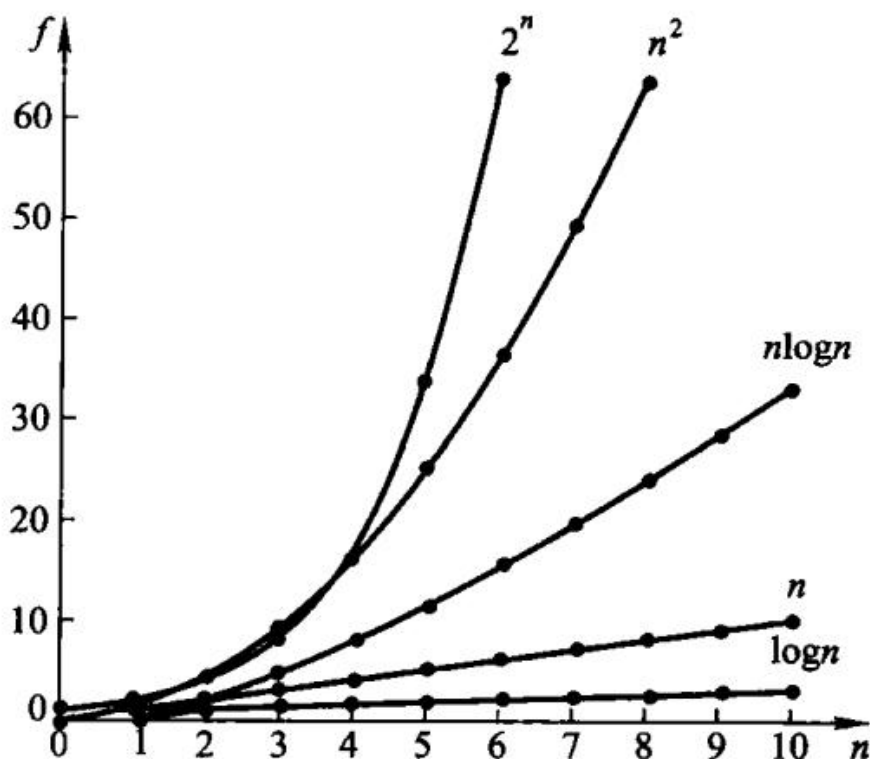


图 1.5 常用函数的增长趋势

对数组求和是 $2 + 2n$ 次赋值，即 $O(n)$ ；两层循环扫子数组就是 $O(n^2)$ 。后面各章的「时间代价」都按这个口径写。

1.4.2 最佳、最差和平均情况

同一算法对不同输入的操作次数可以差很多。快排平均 $O(n \log n)$ 、最坏 $O(n^2)$ ；顺序检索最好 1 次、最坏 n 次。分析时必须说清是哪一种，不能只报一个数字。

1.4.3 时间和空间的折衷

预排序、建索引、开散列表都是用预处理或额外空间换检索时间。没有免费的加速。

1.4.4 求解问题时数据结构的选择和评价

先看运算集合（要不要随机访问、会不会频繁插入），再看规模和存储层次（内存还是外存）。传言问题要任意两点最短路径且 n 很小，所以选邻接矩阵加 Floyd，而不是邻接表加反复 Dijkstra。

第 2 章 线性表

线性表把一串同类型元素排成唯一的先后次序。顺序表把元素连续放在数组里，能 $O(1)$ 按下标访问；链表用链接保存相邻关系，能在已知结点位置 $O(1)$ 插入或删除。关键是区分「按位置找元素」和「已知结点后改链接」这两类操作。

源码：顺序表、顺序表示例、链表、链表示例。

先跑一遍

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::ArrayList<int> values;
    values.append(10);
    values.append(30);
    values.insert(1, 20);
    std::cout << " 顺序表:";
    for (std::size_t index = 0; index < values.size(); ++index) {
        std::cout << ' ' << values.at(index);
    }
    std::cout << "\n查找 20 的下标: " << *values.find(20) << '\n';
    std::cout << " 删除位置 1 得到 " << values.remove(1) << ", 剩余:";
    for (std::size_t index = 0; index < values.size(); ++index) {
        std::cout << ' ' << values.at(index);
    }
    std::cout << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch02/array_list \
    code/ch02/array_list/demo.cpp -o /tmp/list-demo
/tmp/list-demo
```

```
顺序表: 10 20 30
查找 20 的下标: 1
删除位置 1 得到 20, 剩余: 10 30
```

`insert(1, 20)` 要把后面的元素右移一位，这是顺序表的固有代价。链表同一组操作只改两条链接，但按位置找前驱仍是 $O(n)$ ：

```

#include "modern.hpp"

#include <iostream>

int main() {
    dsa::LinkedList<int> values;
    values.append(10);
    values.append(30);
    values.insert(1, 20);
    std::cout << " 链表:";
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << "\n删除位置 0 得到 " << values.remove(0) << ", 剩余:";
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << "\nappend 之后尾元素是 " << values.at(values.size() - 1) << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch02/linked_list \
    code/ch02/linked_list/demo.cpp -o /tmp/link-demo
/tmp/link-demo

```

```

链表: 10 20 30
删除位置 0 得到 10, 剩余: 20 30
append 之后尾元素是 30

```

append 经尾指针 $O(1)$ 接链，不必再从头走到尾。

2.1 线性表的概念

线性表 (linear list) 是由 $n(n \geq 0)$ 个类型相同的数据元素组成的有限序列。除第一个元素外，每个元素有且仅有一个直接前驱；除最后一个元素外，每个元素有且仅有一个直接后继。

线性表的抽象数据类型给出的是一组运算：置空、判空、在表尾追加、在指定位置插入、删除指定位置的元素、按位置取值与改值、按值查找位置。

原书【代码 2.1】用一个类模板 `List<T>` 来写这组运算。那份清单有两处今天必须改：

1. 它声明了 `bool delete(const int p);`——**delete** 是 C++ 关键字，不能作函数名。整章的删除操作都建立在这个编译不过的名字上。

2. 它写的是 `class List { void clear(); ... };`，通篇没有 **public:**。class 的默认访问权限是 `private`，于是这个抽象数据类型的每一个运算都调不到。（同一本书第 3 章的代码 3.1 是写了 `public:` 的。）

第 2 点尤其值得停一下：抽象数据类型的意义就是对外给出一组运算。一个所有运算都私有的 ADT，在语法上成立、在语义上是空的。

本书把这组运算直接定义在 `ArrayList<T>` 上，不再另设一个基类——理由与第 3 章相同：那样的空基类给不了多态，却会带来非虚析构的未定义行为。对元素类型的要求用 `static_assert` 写在类头：

```
/// 顺序表（按顺序方式存储的线性表，又称向量）。
///
/// 与原书 arrList 的差别：容量不足时自动翻倍，而不是打印 "The list is overflow"
/// 然后返回 false；位置非法抛 std::out_of_range，而不是打印一行再返回 false；
/// 查找返回 std::optional<size_type>，而不是「出参 + bool」双通道。
template <typename T>
class ArrayList {
    static_assert(std::is_default_constructible<T>::value,
                  "ArrayList<T>: T 必须可默认构造（底层 new T[n] 会构造整块槽位）");
    static_assert(std::is_move_assignable<T>::value,
                  "ArrayList<T>: T 必须可移动赋值（插入/删除要搬动元素）");
    static_assert(std::is_copy_assignable<T>::value ||
                  ↪ std::is_nothrow_move_assignable<T>::value,
                  "ArrayList<T>: 不可复制的 T 必须可无异常移动赋值（扩容保持强异常保证）");
    static_assert(!std::is_reference<T>::value, "ArrayList<T>: T 不能是引用类型");

public:
    using value_type = T;
    using size_type = std::size_t;
```

2.2 顺序表

按顺序方式存储的线性表称为顺序表 (array-based list)，又称向量 (vector)，通过数组建立。

假设每个元素占用 L 个存储单元，顺序表的开始结点 k_0 的存储位置记为 $b = \text{loc}(k_0)$ ，称为首地址；则下标为 i 的元素 k_i 的存储位置为

$$\text{loc}(k_i) = b + i \times L$$

每个元素的存储位置都与起始位置相差一个与位序成正比的常数。只要确定了基地址，表中任一元素的地址都能直接算出——顺序表因此是一种随机存取的存储结构，按下标取值的时间代价为 $O(1)$ 。物理相邻表示了逻辑相邻。

(a) 线性表的逻辑结构

数据元素	k0	k1	...	k _i	...	k _{n-1}	...
逻辑地址	0	1	...	i	...	n-1	... maxSize-1

(b) 线性表的顺序存储结构

数据元素	k0	k1	...	k _i	...	k _{n-1}	...
存储地址	b	b+L	...	b+i · L	...	b+(n-1) · L	...

图 2.1 顺序表的示意图

2.2.1 存储与所有权

原书【代码 2.2】用四个成员表示顺序表：`T* aList`、`int maxSize`、`int curLen`、`int position`。前三个对应缓冲区、容量、当前长度，现代实现一一对应（改用 `std::size_t`，因为「负下标」不该在类型层面存在）。

第四个 `position` 是个当前处理位置游标，配合正文提到的 `setPos / setStart / next / prev` 用来「依次处理元素」。这个设计今天不能要：遍历状态一旦住进容器，`const` 对象就没法遍历（游标要改），两处代码不能同时遍历，嵌套遍历直接互相踩。本书删掉这个成员，改为提供 `begin()/end()`，把遍历状态交回调调用方——`range-for` 因此可以直接用在顺序表上。（顺带一提：原书那个 `position`，在书中展示的所有算法里一次都没被用到。）

与第 3 章一样，缓冲区是裸指针加显式五法则：顺序表的存储管理正是本节的教学内容，换成 `std::unique_ptr<T[]>` 会让五法则退化成走过场。原书 `arrList` 有析构函数却没有拷贝构造与拷贝赋值，一次 `arrList<int> b = a;` 就二次释放——与第 3 章 `arrStack` 是同一个错误，同一份 ASan 报告可以复现。

另外原书的 `clear()` 是这样写的：

```
void clear() { delete [] aList; curLen = position = 0; aList = new T[maxSize]; }
```

释放整块再重新分配。既没必要（把长度归零即可，容量留着复用），也不是异常安全的：若 `new` 抛异常，对象就停在「指针已释放、长度已归零」的破碎状态，之后析构还会再 `delete[]` 一次。本书的 `clear()` 是 `noexcept` 的，只把长度归零。

2.2.2 顺序表的检索

顺序表上的检索分按位置和按内容两类。按位置的检索直接由地址公式算出， $O(1)$ ：

```

/// 按下标读取, 0(1)。越界抛 std::out_of_range。
/// 原书 getValue 用「出参 + bool」, 越界时打印一行再返回 false。
[[nodiscard]] const T& at(size_type index) const {
    check_index(index, "ArrayList::at");
    return data_[index];
}

[[nodiscard]] T& at(size_type index) {
    check_index(index, "ArrayList::at");
    return data_[index];
}

/// 修改指定位置的值。越界抛 std::out_of_range。
void set(size_type index, const T& value) {
    check_index(index, "ArrayList::set");
    data_[index] = value;
}

```

按内容的检索是把待查值与表中元素依次比较, $O(n)$ 。原书【算法 2.3】写作 `bool getPos(int& p, const T value)`——位置由出参带出、成败由返回值带出。这个形状的问题是: 调用方忘了检查返回值, 读到的就是没被写过的 `p`。

(顺带一提, 算法 2.3 那段按印刷原样是编译不过的: 循环写的是 `for (i = 0; i < n; i++)`, 而 `n` 从未声明, 按上下文应为 `curLen`。)

```

/// 按内容查找, 返回第一次出现的下标; 没有则返回 std::nullopt。0(n)。
///
/// 原书【算法 2.3】是 'bool getPos(int& p, const T value)': 出参带位置、
/// 返回值带成败。调用方忘了检查返回值, 读到的就是没被写过的 p。
/// 这里让「找没找到」进入类型系统, 忽略返回值还会被 -Wunused-result 拦下。
[[nodiscard]] std::optional<size_type> find(const T& value) const {
    for (size_type i = 0; i < size_; ++i) {
        if (data_[i] == value) {
            return i;
        }
    }
    return std::nullopt;
}

```

检索的时间代价体现在比较次数上。最好情况是第 1 个元素即为所求, 比较 1 次; 最差情况是表中没有该元素, 比较 n 次。等概率假设下平均比较次数为

$$\sum_{i=1}^n p \times i = \frac{1}{n}(1 + 2 + \cdots + n) = \frac{n+1}{2}$$

即平均需要检查表中一半的元素，时间开销为 $O(n)$ 。

2.2.3 顺序表的插入

插入要在指定位置腾出一个空位：从表尾起，把 pos 之后的元素逐个右移一位。

```
/// 在位置 pos 插入元素，pos 可以等于 size()（即追加到表尾）。
/// 位置非法抛 std::out_of_range；容量不足自动翻倍。
///
/// 时间代价仍是  $O(n)$ ——pos 之后的元素都要右移一位。这是顺序表的固有代价，
/// 也是第 2.3 节要拿它和链表对比的地方，没有被"优化"掉。
void insert(size_type pos, const T& value) {
    make_gap(pos);
    data_[pos] = value;
    ++size_;
}

void insert(size_type pos, T&& value) {
    make_gap(pos);
    data_[pos] = std::move(value);
    ++size_;
}

void append(const T& value) { insert(size_, value); }
void append(T&& value) { insert(size_, std::move(value)); }
```

两处与原书不同：

- 容量不足时自动翻倍，而不是打印 "The list is overflow" 然后返回 false。扩容策略与第 3 章算法 3.3 相同，搬迁判据见 3.1.3 节末（移动赋值是否 noexcept）。
- 位置非法抛 `std::out_of_range`，而不是打印一行再返回 false。按公约，可预期的空状态用 optional，真正的错误抛异常——插到表外是错误。

插入位置 p 处腾空位的过程如下（图 2.2）：p 及其之后的元素整体右移一位，空出的 p 位置写入新元素。

aList	k0	k1	...	value	kp	...	kn-1	...
下标	0	1	...	p	p+1	...	n	... maxSize-1

图 2.2 顺序表元素插入示意图

时间代价仍然是 $O(n)$ 。最好情况是插在表尾，移动 0 次；最差情况是插在表头，n 个元素全要移动；等概率假设下平均移动次数为

$$\sum_{i=0}^n p \times (n-i) = \frac{1}{n+1} \sum_{i=0}^n (n-i) = \frac{n}{2}$$

扩容没有改变这个结论：翻倍带来的搬迁摊还到每次插入是 $O(1)$ ，而元素右移本身仍是 $O(n)$ 。这一点是 2.3 节拿顺序表与链表对比的依据，不能被优化掉。

2.2.4 顺序表的删除

删除是插入的镜像：把 pos 之后的元素逐个左移一位。

```

/// 删除位置 pos 上的元素并返回它。位置非法抛 std::out_of_range。
///
/// 空表上删除必然越界，所以不需要原书那句单独的空表检查——
/// 「表空」在这里不是一种可预期状态，而就是下标非法的一个特例。
T remove(size_type pos) {
    check_index(pos, "ArrayList::remove");
    T removed = std::move(data_[pos]);
    for (size_type i = pos; i + 1 < size_; ++i) {
        data_[i] = std::move(data_[i + 1]); // 左移一位，O(n)
    }
    --size_;
    return removed;
}

```

删除位置 p 上的元素后，其后的元素整体左移一位（图 2.3）：
删除前：

aList	k0	k1	k2	...	kp	...	kn-1	...
下标	0	1	2	...	p	...	n-1	...

删除后：

aList	k0	k1	k2	...	kp+1	...	kn-1	...
下标	0	1	2	...	p	...	n-2	...

图 2.3 顺序表元素删除示意图

原书【算法 2.5】的函数名是 delete——C++ 关键字，编译不过；本书叫 remove。另外它返回 bool，被删掉的值就此丢失；这里把它返回给调用方，「删除并取用」不必先 getValue 再 delete 两步走。

删除的时间代价分析与插入相同，平均为 $O(n)$ 。

与原书的对照

原书	现在	为什么
<code>bool delete(const int p)</code>	<code>T remove(size_type pos)</code>	<code>delete</code> 是关键字，原样编译不过；顺带把被删元素还给调用方
<code>class List { ... } 无 public:</code>	不设基类，要求写成 <code>static_assert</code>	所有运算私有的 ADT 在语义上是空的；空基类给不了多态
<code>for (i = 0; i < n; i++)</code>	<code>i < size_</code>	原书 <code>n</code> 从未声明，编译不过
<code>bool getPos(int& p, T v)</code>	<code>std::optional<size_type> find(const T&)</code>	「找没找到」交给类型系统，忽略返回值会告警
<code>bool getValue(int p, T& v)</code>	<code>const T& at(size_type),</code> 越界抛异常	同上；越界是错误，不是可预期状态
<code>int maxSize / curLen,</code> 满了拒绝	<code>size_t capacity_ /</code> <code>size_</code> ，自动翻倍	<code>int</code> 溢出是未定义行为；「负下标」不该存在；容量不该是使用者的负担
<code>int position</code> 游标 + <code>next/prev</code>	<code>begin()/end()</code>	遍历状态住在容器里， <code>const</code> 不能遍历、不能嵌套遍历
<code>cout << "The list is overflow"</code>	不做任何 I/O	数据结构不该和标准输出耦合
<code>clear()</code> 释放后重新分配	<code>clear() noexcept</code> 只归零长度	原写法没必要，且 <code>new</code> 抛异常会留下破碎对象

刻意没改的：连续存储、按下标 $O(1)$ 随机存取、插入与删除搬动 $O(n)$ 个元素。这三条正是 2.3 节拿顺序表和链表做对比的全部依据，一条都不能优化掉。

完整实现见 `code/ch02/array_list/modern.hpp`，测试见同目录 `test.cpp` (47 项断言，覆盖上表每一行；用 `python3 tools/check_code.py` 在 `-Werror + ASan/UBSan` 与 `-O2` 两种构建下各跑一遍)。

2.3 链表

顺序表用物理相邻表示逻辑相邻，所以按下标读取是 $O(1)$ ，但中间插入和删除需要搬动后续元素。链表把逻辑相邻写进结点的链接域：结点可以散落在内存中；给定一个前驱结点后，插入或删除只改常数条指针。代价也必须如实保留：要按位

置找到那个前驱，仍须从表头循链，所以按位置访问、查找、插入和删除的总时间仍是 $O(n)$ 。

2.3.1 单链结点与头结点

【代码 2.6】单链表的结点定义。

```
/// 原书【代码 2.6】的单链结点：数据域与指向后继的链接域。
///
/// 实际容器把它藏在私有实现里，避免调用方任意改坏链；这里保留独立类型，
/// 因为后续栈、队列可复用同一种结点形状。
template <typename T>
struct SinglyLink {
    T data;
    SinglyLink* next{nullptr};

    template <typename U>
    explicit SinglyLink(U&& value, SinglyLink* successor = nullptr)
        : data(std::forward<U>(value)), next(successor) {}
};

/// 原书【代码 2.12】的双链结点：额外保存前驱链接。
template <typename T>
struct DoublyLink {
    T data;
    DoublyLink* next{nullptr};
    DoublyLink* prev{nullptr};

    template <typename U>
    explicit DoublyLink(U&& value, DoublyLink* predecessor = nullptr, DoublyLink*
        ↪ successor = nullptr)
        : data(std::forward<U>(value)), next(successor), prev(predecessor) {}
};
```

【代码 2.6 结束】

`LinkedList<T>` 在对象内嵌一个不承载 `T` 的头结点。它等价于原书的“第 -1 个结点”，从而表头插入和删除都变成“修改某个前驱的 `next`”；空表不必另写一套分支。尾指针指向最后一个实际结点，空表时回指头结点，故 `append` 不需要从头寻找末尾。

【代码 2.7】单链表的类型定义。

```
/// 带头结点、尾指针的单链表。
///
/// 原书的 `setPos(-1)` 返回头结点；本实现把这个实现细节留在 predecessor_at，
/// 对外位置统一为 [0, size())。按值查找返回 optional，位置错误抛 out_of_range。
```

```

template <typename T>
class LinkedList {
    struct NodeBase {
        NodeBase* next{nullptr};
    };
    struct Node final : NodeBase {
        T value;

        template <typename U>
        explicit Node(U&& item, NodeBase* successor = nullptr)
            : NodeBase{successor}, value{std::forward<U>(item)} {}
    };

public:
    using value_type = T;
    using size_type = std::size_t;

```

【代码 2.7 结束】

2.3.2 构析与循链定位

【算法 2.8】带有头结点的单链表构造函数与析构函数。

本实现的头结点嵌入对象本身，不需要动态分配；clear() 沿 next 逐结点释放实际结点，析构函数调用它。复制构造采用“先逐个接入新链，失败即清理”的规则，避免半成品对象在元素复制抛异常时泄漏。

【算法 2.8 结束】

【算法 2.9】寻找链表的第 i 个结点。

定位从头结点开始逐步走 next，不新建任何结点。原书的 setPos(-1) 是处理头结点的技巧；现代接口不暴露 -1 这个特殊位置，内部 predecessor_at(0) 直接返回头结点。

【算法 2.9 结束】

2.3.3 插入与删除

【算法 2.10】插入单链表的第 i 个结点。

```

/// 尾插直接经 tail_ 接链, 0(1)。不能转调 insert(size_), 后者必须循链定位前驱。
void append(const T& value) { append_impl(value); }
void append(T&& value) { append_impl(std::move(value)); }

void insert(size_type pos, const T& value) { insert_impl(pos, value); }
void insert(size_type pos, T&& value) { insert_impl(pos, std::move(value)); }

```

【算法 2.10 结束】

插入先定位前驱、再构造新结点、最后接入链接；结点构造或分配失败时，链接和长度尚未变化。若前驱恰是尾结点，同步让 `tail_` 指向新结点。给定前驱后的接链是 $O(1)$ ，定位仍是 $O(n)$ 。

【算法 2.11】单链表的删除算法。

```
T remove(size_type pos) {
    NodeBase* predecessor = predecessor_at(pos);
    NodeBase* removed = predecessor->next;
    if (removed == nullptr) {
        throw std::out_of_range("LinkedList::remove: 下标越界");
    }
    // 先移动出值；若 T 的移动构造抛，链接尚未改变，容器仍完整。
    T value = std::move(static_cast<Node*>(removed)->value);
    predecessor->next = removed->next;
    if (removed == tail_) {
        tail_ = predecessor;
    }
    delete static_cast<Node*>(removed);
    --size_;
    return value;
}
```

【算法 2.11 结束】

删除不能只摘除链接：被删结点必须释放，且删除尾结点后 `tail_` 必须回退到前驱。否则下一次 `append` 会写入已释放内存。位置不合法统一抛 `std::out_of_range`，容器不打印提示。

2.3.4 双链结点

【代码 2.12】双链表的结点定义和实现。

上面的 `DoublyLink<T>` 已保留 `prev` 与 `next` 两个链接域。双链表删除某一结点时，需要同时维护前驱的 `next` 和后继的 `prev`；这能高效向前走，但每个结点多一个指针且不变量更多。原书在此只给出结点定义，没有给出完整双链表算法，因此本轮不假装已经现代化完整双链表。

【代码 2.12 结束】

完整可运行实现见 `code/ch02/linked_list/modern.hpp`，测试覆盖头/中/尾插入、删尾后的尾指针修复、深拷贝、移动、元素构造异常和 `move-only` 元素；变异自检还确认“删尾不回退 `tail`”会在后续尾插崩溃，“复制构造失败不清理”会留下元素对象。原书逐条证据见 `code/ch02/linked_list/legacy.md`。

2.4 线性表实现方法的比较

顺序表按下标随机访问是 $O(1)$ ，插入删除要搬动后续元素，平均 $O(n)$ ；链表在已知前驱时插入删除是 $O(1)$ ，但按位置找前驱仍是 $O(n)$ 。顺序表局部性好、无指针开销；链表按需分配、没有预留空洞。选择取决于「按位置读」多，还是「在已知结点旁改链接」多。

第 3 章 栈与队列

栈是「最后放入、最先取出」，适合递归调用、括号匹配和表达式计算；队列是「先放入、先取出」，适合排队服务和广度优先搜索。空栈、空队列是正常状态，现代接口以 `std::optional` 返回。

源码：顺序栈、栈示例、链式栈、表达式求值、背包、队列、队列示例。

先跑一遍

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::ArrayStack<int> stack;
    stack.push(1);
    stack.push(2);
    stack.push(3);
    std::cout << " 栈顶是 " << *stack.top() << '\n';
    std::cout << " 依次弹出:";
    while (auto value = stack.pop()) {
        std::cout << ' ' << *value;
    }
    std::cout << "\n空栈再弹? " << (stack.pop() ? " 有值" : " 空") << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch03/array_stack \
    code/ch03/array_stack/demo.cpp -o /tmp/stack-demo
/tmp/stack-demo
```

```
栈顶是 3
依次弹出: 3 2 1
空栈再弹? 空
```

后进先出：最后压入的 3 最先出来。空栈上 `pop()` 返回空 `optional`，不打印、不崩溃。

循环队列牺牲一个槽位区分空与满，所以逻辑容量 3 实际申请 4 个槽：

```
#include "modern.hpp"
```

```
#include <iostream>

int main() {
    dsa::ArrayQueue<int> queue(3);
    if (!queue.enqueue(1) || !queue.enqueue(2) || !queue.enqueue(3)) {
        std::cout << " 入队失败\n";
        return 1;
    }
    std::cout << " 逻辑容量 3 时再入队? " << (queue.enqueue(4) ? " 成功" : " 已满") <<
        '\n';
    std::cout << " 依次出队:";
    while (auto value = queue.dequeue()) {
        std::cout << ' ' << *value;
    }
    std::cout << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch03/queue \
    code/ch03/queue/demo.cpp -o /tmp/queue-demo
/tmp/queue-demo
```

```
逻辑容量 3 时再入队? 已满
依次出队: 1 2 3
```

3.1 栈

3.1.1 栈的抽象数据类型

栈(stack)是限定仅在一端进行插入和删除运算的线性表,该端称为栈顶(top),另一端称为栈底(bottom)。栈的元素按后进先出(LIFO, last in first out)的次序访问:最后压入的元素最先弹出。

基于栈的特性,栈的抽象数据类型包含进栈push、出栈pop、读栈顶top等常用操作,以及判断栈是否为空的边界操作。根据抽象和封装的原则,只可通过抽象数据类型定义的运算来操作栈。

原书【代码 3.1】用一个成员函数既非纯虚、析构函数也非 virtual 的空基类 Stack<T> 来表达这层抽象。那样的基类给不了任何多态能力:成员函数既不是纯虚的、也没有定义,派生类“实现”它们并不构成覆盖;反而埋下了「通过 Stack<T>* 删除派生对象」这一未定义行为。

抽象数据类型描述的是「一组运算」,不是「一个基类」。在 C++ 里,模板本身已经承担了这层抽

象——`ArrayStack<T>` 提供哪些运算，由它的接口决定，不需要继承任何东西。真正需要写下来的是对元素类型 `T` 的要求，C++17 用 `static_assert` 配合 `<type_traits>` 表达：编译期检查、不付虚函数表的运行期代价，而且错误信息就停在实例化处，不会炸出一屏模板内部细节。

```
/// 基于数组的栈（后进先出）。
///
/// 与原书 arrStack 的差别：容量耗尽时自动翻倍（原书要么溢出报错，要么靠算法 3.3
/// 手工换成扩容版本）；不打印任何东西；空栈上的 pop/top 返回空 optional，
/// 而不是靠「出参 + bool」双通道返回。
template <typename T>
class ArrayStack {
    // 原书用一个成员函数既非纯虚、析构又非 virtual 的空基类 Stack<T> 来表达「抽象」。
    // 那样的基类给不了多态，还带来「通过基类指针删除派生对象」的未定义行为。
    // C++17 里表达「T 要满足什么」的直接工具是 static_assert + 类型特征：
    // 编译期检查、不付虚表代价，而且错误信息就停在实例化处。
    static_assert(std::is_default_constructible<T>::value,
                  "ArrayStack<T>: T 必须可默认构造（底层 new T[n] 会构造整块槽位）");
    static_assert(std::is_move_assignable<T>::value,
                  "ArrayStack<T>: T 必须可移动赋值（push/pop 需要移动元素）");
    static_assert(std::is_copy_assignable<T>::value ||
                  std::is_nothrow_move_assignable<T>::value,
                  "ArrayStack<T>: 不可复制的 T 必须可无异常移动赋值（扩容保持强异常保证）");
    static_assert(!std::is_reference<T>::value, "ArrayStack<T>: T 不能是引用类型");

public:
    using value_type = T;
    using size_type = std::size_t;
```

四条 `static_assert` 把类对 `T` 的要求摆在了明面上。

第一条尤其诚实：底层的 `new T[n]` 会把整块槽位都默认构造出来，所以 `T` 必须可默认构造——这是原书 `new T[mSize]` 就有的限制，我们没有恶化它，也还没有解决它（真正的容器做法是申请未初始化存储再逐个 `placement new`）。

第三条（不可拷贝的 `T` 必须能无异常移动赋值）是为了守住扩容的强异常保证，理由见 3.1.3 节末——那是本节最容易踩空的一处。

把这些限制写成 `static_assert`，好过让它们以一条晦涩的模板报错、或者某次扩容失败后的谜案出现。

3.1.2 顺序栈

采用顺序存储结构的栈称为顺序栈 (array-based stack)，需要一块连续的区域存储栈中元素。

对元素数目为 `n` 的栈，首先要确定数组的哪一端表示栈顶。如果把数组的第 0

个位置作为栈顶，所有的插入和删除都在第 0 个位置进行，每次 push 或 pop 都要把栈中所有元素后移或前移一个位置，时间代价为 $O(n)$ 。反之，把最后一个元素的位置作为栈顶，新元素添加在表尾、出栈也只删除表尾元素，每次操作的时间代价仅为 $O(1)$ 。图 3.2 所示为按后一种方案实现的栈。

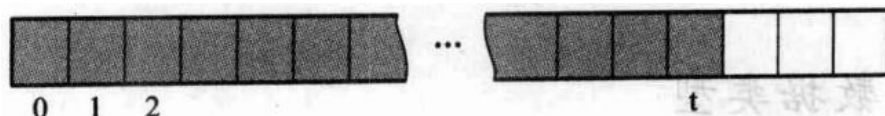


图 3.2 栈的顺序存储结构示意图

存储与所有权

原书【代码 3.2】用三个成员表示一个顺序栈：int mSize（容量）、int top（栈顶位置）、T * st（裸指针数组）。这套写法有三处在今天必须改：

1. int top 与成员函数 bool top(T&) 重名，导致这个类按印刷原样根本编译不过；
2. 无参构造只设了 top = -1，mSize 与 st 都没初始化，析构时 delete[] 一个不确定指针；
3. 有析构函数却没有拷贝构造与拷贝赋值，一次 arrStack<int> b = a；就会二次释放。

第 2、3 条是未定义行为，编译器开到 -Wall -Wextra -Wpedantic 也一句警告都不给。

现代实现保留裸指针 T* data_——顺序栈的存储管理正是本节要教的内容，把它换成 std::unique_ptr<T[]> 固然更省事，但五法则也就随之退化成走过场（编译器生成的版本就够用了）。留着裸指针，这五个函数才是承重的：少写一个，AddressSanitizer 立刻能复现出崩溃。

```
// 三/五法则：原书只写了析构函数，没有拷贝构造与拷贝赋值，
// 于是 `arrStack<int> b = a;` 之后两个对象持有同一根指针，各析构一次 → 二次释放。
//
// 这里用的是 ** 裸指针 **，所以这五个函数是承重的，不是仪式——
// 少写一个，ASan 立刻能复现出崩溃（见 legacy.md 缺陷 4 的实测输出）。
ArrayStack(const ArrayStack& other)
    : capacity_(other.capacity_),
      top_index_(other.top_index_),
      data_(other.capacity_ ? new T[other.capacity_] : nullptr) {
    for (size_type i = 0; i < top_index_; ++i) {
        data_[i] = other.data_[i];
    }
}

/// 拷贝并交换：先构造完整副本再与自身交换，天然自赋值安全，
/// 且拷贝失败时原对象不受影响。
```

```

ArrayStack& operator=(const ArrayStack& other) {
    if (this != &other) {
        ArrayStack copy(other);
        swap(copy);
    }
    return *this;
}

ArrayStack(ArrayStack&& other) noexcept { swap(other); }

ArrayStack& operator=(ArrayStack&& other) noexcept {
    if (this != &other) {
        ArrayStack moved(std::move(other)); // other 交出所有权
        swap(moved);                       // 自己原来的缓冲区随 moved 析构释放
    }
    return *this;
}

~ArrayStack() { delete[] data_; }

```

拷贝赋值用的是拷贝并交换 (copy-and-swap): 先构造一个完整副本, 再与自身交换。它自然地做到了自赋值安全, 也在拷贝失败时保证原对象不受影响。

出栈: 用 **optional** 取代双通道返回

原书的 `bool pop(T & item)` 用出参带值、用返回值带成败。调用方一旦忘了检查返回值, 读到的就是没被修改过的 `item`。现代写法让「有没有值」这件事进入类型系统: 空栈不是错误, 而是一种可预期的状态, 用 `std::optional` 表达它; 真正的错误 (下标越界、容量溢出) 才抛异常。

```

/// 出栈。空栈返回 std::nullopt——原书是「返回 false + 往 cout 打一行中文」,
/// 调用方既没法在库里复用, 也容易忽略返回值。
[[nodiscard]] std::optional<T> pop() {
    if (empty()) {
        return std::nullopt;
    }
    --top_index_;
    return std::optional<T>(std::move(data_[top_index_]));
}

/// 读栈顶但不弹出, 返回 ** 副本 **。空栈返回 std::nullopt, 不是未定义行为。
/// 要求 T 可拷贝; move-only 元素请用 peek()。
[[nodiscard]] std::optional<T> top() const {
    if (empty()) {
        return std::nullopt;
    }
}

```

```

    }
    return std::optional<T>(data_[top_index_ - 1]);
}

/// 只读观望栈顶：不拷贝、不弹出。空栈返回 nullptr。
///
/// 与 top() 的分工——top() 给你一份可以带走的副本（安全，但要求 T 可拷贝，
/// 而且确实拷贝了一次）；peek() 零拷贝，move-only 元素也能用，代价是
/// ** 返回的指针在下次 push / pop / clear 之后即失效 **（扩容会换掉整块缓冲区）。
/// 生命周期由调用方负责，这一点必须写在文档里，不能靠使用者猜。
[[nodiscard]] const T* peek() const noexcept {
    return empty() ? nullptr : &data_[top_index_ - 1];
}

```

[[nodiscard]] 使「调用了 pop 却扔掉返回值」成为一条编译警告（本书的构建开 -Werror，即编译失败）。同样重要的是这里没有 `std::cout`：原书在空栈出栈时直接打印中文提示，把一个数据结构与标准输出焊死——它没法在库里复用，提示无法本地化，失败路径也无从测试。

「读栈顶」有两种正当需求，因此这里给了两个接口，不要把它们合并成一个：

- `top()` 返回 `std::optional<T>`，是一份可以带走的副本。安全，但要求 `T` 可拷贝，而且确实拷贝了一次。
- `peek()` 返回 `const T*`，空栈为 `nullptr`，零拷贝。`std::unique_ptr` 这类只能移动的元素只能用它来观望。

代价也是一对：`optional` 的安全靠拷贝换来，`peek()` 的零拷贝则把生命周期交给了调用方——它返回的指针在下次 `push / pop / clear` 之后即失效，因为扩容会换掉整块缓冲区。这条失效契约必须写在文档里，不能靠使用者猜。

两种代价都真实存在。把任何一种藏起来，都是替读者做了本该由读者做的选择。

顺序栈的扩容

栈中元素动态变化，当栈满时继续进栈会产生上溢出 (overflow)。原书【算法 3.3】给出的改进办法是：申请一个扩大一倍的新数组，把原有内容顺序移动过去，再执行进栈。这个策略本身是对的——每个元素在均摊意义下只被搬运常数次，`push` 的摊还时间代价仍是 $O(1)$ ——但那段代码有两个问题：循环变量 `i` 从未声明（编译不过），以及先 `delete[] st` 再赋新指针，搬运中途若抛出异常，栈就停在半新半旧的状态。

```

static constexpr size_type kInitialCapacity = 4;

void ensure_capacity() {
    if (top_index_ < capacity_) {
        return;
    }
}

```

```

    }
    constexpr size_type kMax = std::numeric_limits<size_type>::max();
    if (capacity_ > kMax / 2) {
        throw std::overflow_error("ArrayStack: 容量翻倍会溢出");
    }
    const size_type next = capacity_ == 0 ? kInitialCapacity : capacity_ * 2;
    T* fresh = new T[next];
    try {
        for (size_type i = 0; i < top_index_; ++i) {
            // 这里是 ** 赋值 ** 而不是构造，所以不能用 std::move_if_noexcept:
            // 它检查的是移动 ** 构造 ** 是否 noexcept，而可抛的移动赋值会在搬迁
            // 中途把原栈的元素掏空——红队 T-002 实测复现过（见 legacy.md 缺陷 11）。
            //
            // 判据必须落在「移动赋值抛不抛」这一个维度上：
            // 移动赋值 noexcept → 移动。不可能抛，强异常保证不受影响，也不白白深拷贝。
            // 否则 → 复制。拷贝赋值取 const&，抛了也动不了原栈；
            // 上面的 static_assert 保证走到这里的 T 一定可复制赋值。
            if constexpr (std::is_nothrow_move_assignable<T>::value) {
                fresh[i] = std::move(data_[i]);
            } else {
                fresh[i] = data_[i];
            }
        }
    } catch (...) {
        // 裸指针的代价：RAII 版本不用写这一段。搬迁失败要自己收拾新缓冲区，
        // 且此时还没动 data_/capacity_，所以原栈完好——这就是强异常保证。
        delete[] fresh;
        throw;
    }
    delete[] data_;
    data_ = fresh;
    capacity_ = next;
}

```

四处值得注意：

- 先建后换。新缓冲区搬完才动 data_，中途抛异常则原栈原封不动，这就是强异常保证。原书先 delete[] st 再赋新指针，一旦搬运途中抛出，栈就停在一个既不是旧状态也不是新状态的地方。
- try/catch 是裸指针的代价。搬迁失败要自己把新缓冲区收掉再重新抛出；如果底层换成 std::unique_ptr<T[]>，这一段可以整个删掉。取舍在这里是显式的：本节要教存储管理，就得连这份代价一起看见。
- 搬迁用移动还是拷贝，判据必须选对。这一条值得单独讲，见下一小节。
- 容量用 std::size_t。原书用 int，mSize * 2 溢出是未定义行为；这里在翻倍前先判界并抛 std::overflow_error。

这不是纸面推理。测试里有一个「第 3 次拷贝赋值必定抛异常」的元素类型，用它撑满栈再触发扩容，然后逐项断言：长度不变、容量不变、原有元素逐个完好、栈之后仍能继续使用；另一个类型让 `new T[]` 本身抛 `std::bad_alloc`，同样逐项断言。把「先建后换」改回原书的「先释放旧的」，Debug 构建下 UBSan 当场报空指针引用，Release 构建直接段错误。

一个容易踩空的地方：`move_if_noexcept` 在这里是错的

搬迁元素时，一个几乎条件反射的写法是 `std::move_if_noexcept(data_[i])`——「移动可能抛就退回拷贝」，标准库容器扩容正是这么做的。但在这段代码里它是错的。

`std::move_if_noexcept` 的判据是 `T` 的移动构造是否 `noexcept`。而这里 `fresh` 已经由 `new T[next]` 默认构造好了，搬迁执行的是移动赋值。两者是不同的函数，可以有不同的异常规格：一个类完全可以移动构造 `noexcept`、移动赋值却会抛。这时 `move_if_noexcept` 会放行移动，而抛出发生在搬到一半时——原栈里已经被移动走的那些元素，已经被掏空了，强异常保证就此破裂。

这个洞不是推演出来的：本书的测试里有一个正是这种形状的类型（移动构造 `noexcept`，第 3 次移动赋值抛异常），它让原来的实现真的失败了。

判据要落在实际执行的那个操作上：

- 移动赋值 `noexcept` → 移动。不可能抛，强异常保证不受影响，也不必白白深拷贝。
- 否则 → 拷贝。拷贝赋值取 `const&`，抛了也动不了原栈。

第二条要求 `T` 可拷贝赋值，所以类头处那条 `static_assert` 写明：不可拷贝的 `T` 必须满足 `is_nothrow_move_assignable`。这是本容器对元素类型的一条真实约束，写成编译期断言，好过让它变成运行期某次扩容失败后的谜案。

反过来也要小心：判据若写成「可拷贝就拷贝」，`std::string`（移动赋值本就是 `noexcept`）每次扩容都会退化成深拷贝，算法 3.3 摊还 $O(1)$ 的分析就打了折扣。测试里因此有一条用例专门数拷贝次数——这两种写法都能通过其他所有断言，只有这条能把它们分开。

进栈本身随之简化为「保证容量、写入、长度加一」：

```
/// 入栈。容量不足时按算法 3.3 的策略翻倍。
/// 强异常保证：搬迁在新缓冲区上完成，中途抛异常则原栈原封不动。
void push(const T& item) {
    ensure_capacity();
    data_[top_index_] = item;
    ++top_index_;
}

void push(T&& item) {
```

```

ensure_capacity();
data_[top_index_] = std::move(item);
++top_index_;
}

```

两个重载分别处理左值和右值。原书的 `bool push(const T item)` 按值传参，顶层 `const` 对调用方没有意义，却强制了一次拷贝，而且 `std::unique_ptr` 这类只能移动的类型根本传不进去。

3.1.3 链式栈

采用链式存储结构的栈称为链式栈。结点分散在堆上，压栈只是接一个新结点，不需要连续空间，也没有“栈满”这回事——这正是它与顺序栈的核心差别。

```

/// 链式栈：结点分散在堆上，压栈只是接一个新结点，不需要连续空间、也不需要扩容。
///
/// 与原书 lnkStack 的差别：`top` 这个名字只留给成员函数（原书 `Link<T>* top`
/// 与 `bool top(T&)` 重名，导致整个类编译不过）；补齐五法则；不做任何 I/O；
/// 出栈返回 `std::optional<T>`。
template <typename T>
class LinkedStack {
    struct Node {
        T value;
        Node* next;

        template <typename U>
        Node(U&& item, Node* successor) : value(std::forward<U>(item)),
        ↪ next(successor) {}
    };

public:
    using value_type = T;
    using size_type = std::size_t;

```

原书【代码 3.4】的 `lnkStack` 有一处与顺序栈完全相同的错误：成员 `Link<T>*` `top` 与成员函数 `bool top(T&)` 重名，整个类编译不过。同一本书在两个存储结构上犯了同一个命名错误，两处都没有被编译器验证过。

另有两处值得指出：

- 构造函数是 `lnkStack(int defSize)`，而那个参数从未被使用。链式栈不需要预设容量，这个参数是从 `arrStack(int size)` 照抄来的；更麻烦的是它没有默认构造函数，使用者被迫为一个无意义的参数编个数出来。
- 有 `~lnkStack(){ clear(); }` 却没有拷贝构造与拷贝赋值。这是本书第五次遇到同一个错误（顺序栈、顺序表、链表、字符串、链式栈）。

```

// 原书有 `~lnkStack(){ clear(); }` 却没有拷贝构造与拷贝赋值:
// 一次 `lnkStack<int> b = a;` 之后两个栈共享同一串结点, 各自析构一次 → 二次释放。
// 与顺序栈、顺序表、链表、字符串是同一个错误, 本书第五次遇到它。
LinkedStack(const LinkedStack& other) {
    // 先按原序收集, 再逆序压回, 避免递归拷贝 (深链会爆栈, 见 UNVERIFIED-RISKS.md)
    Node* source = other.top_;
    Node** tail = &top_;
    try {
        while (source != nullptr) {
            *tail = new Node(source->value, nullptr);
            tail = &(*tail)->next;
            ++size_;
            source = source->next;
        }
    } catch (...) {
        clear(); // 半截链必须自己收拾: 构造函数抛出时析构函数不会运行
        throw;
    }
}

LinkedStack& operator=(const LinkedStack& other) {
    if (this != &other) {
        LinkedStack copy(other);
        swap(copy);
    }
    return *this;
}

LinkedStack(LinkedStack&& other) noexcept
: top_(std::exchange(other.top_, nullptr)), size_(std::exchange(other.size_,
↪ 0)) {}

LinkedStack& operator=(LinkedStack&& other) noexcept {
    if (this != &other) {
        clear();
        top_ = std::exchange(other.top_, nullptr);
        size_ = std::exchange(other.size_, 0);
    }
    return *this;
}

~LinkedStack() { clear(); }

```

压栈与出栈:

```

/// 入栈：接一个新结点。** 没有" 栈满" 这回事 **——这正是链式栈相对顺序栈的差别，
/// 原书顺序栈那边要判 `top == mSize - 1` 并打印" 栈满溢出"。
void push(const T& item) { top_ = new Node(item, top_); ++size_; }
void push(T&& item) { top_ = new Node(std::move(item), top_); ++size_; }

/// 出栈。空栈返回 std::nullopt (D-001 §3c: 空是可预期状态，不抛异常)。
[[nodiscard]] std::optional<T> pop() {
    if (top_ == nullptr) {
        return std::nullopt;
    }
    Node* dying = top_;
    std::optional<T> result(std::move(dying->value));
    top_ = dying->next;
    delete dying;
    --size_;
    return result;
}

/// 取栈顶副本；零拷贝的观望用 peek() (D-001 §3b)。
[[nodiscard]] std::optional<T> top() const {
    return top_ == nullptr ? std::nullopt : std::optional<T>(top_->value);
}

[[nodiscard]] const T* peek() const noexcept {
    return top_ == nullptr ? nullptr : &top_->value;
}

```

一处实现上的取舍：clear() 与析构都写成迭代而非递归。链式结构最自然的写法是递归释放，但深链会耗尽调用栈——第 5 章有实测数字。本书的测试用 20 万个结点压栈再全部弹出，正是为这条兜底。

3.1.4 表达式求值

后缀（逆波兰）表达式不需要括号，也不需要优先级规则：求值时遇操作数压栈，遇操作符弹出两个算完再压回，读到末尾时栈里剩下的唯一元素就是结果。这条主线本书一字未改，用的还是本章自己的 ArrayStack<double>。

```

/// 对后缀（逆波兰）表达式求值。记号之间用空白分隔，例如 "3 4 + 2 *" → 14。
///
/// 与原书 `class Calculator` 的差别，逐条对应它的三个问题：
///
/// 1. ** 原书 `Run()` 直接从 `cin` 读、往 `cout` 写 **，算法与终端焊死：没法写测试、
///    没法在库里复用、没法处理来自别处的表达式。这里接受一个 `string_view`、返回结果。
/// 2. ** 原书 `GetTwoOperands` 在第二个操作数缺失时已经把第一个弹掉了 **，
///    返回 `false` 时栈已被破坏 (legacy.md 缺陷 1)。

```

```

/// 但要说准确：真正让这个 bug 无害的，** 不是 ** 下面那句“先查够不够”，
/// 而是 ** 出错即抛出、整次求值随即作废 **——栈根本没有机会被下一步用到。
/// 预检查只是让错误信息更早、更准，属于锦上添花。
/// 原书的 bug 之所以要命，是因为它 `cerr` 一行之后 ** 继续跑 **。
/// 3. ** 原书出错时 `cerr` 打一行然后 `s.clear()` 继续跑 **，调用方拿不到任何信号。
/// 这里抛 `std::invalid_argument`（表达式不合法）或 `std::domain_error`（除零）。
[[nodiscard]] inline double evaluate_postfix(std::string_view expression) {
    ArrayStack<double> operands;
    std::size_t i = 0;

    const auto pop_two = [&operands] {
        // 先确认有两个再弹。注意这不是“修复”原书 bug 的关键（见函数注释第 2 条），
        // 而是让报错更早、更准。
        if (operands.size() < 2) {
            throw std::invalid_argument(" 后缀表达式：操作数不足");
        }
        right = *operands.pop(); // 先弹出的是右操作数
        left = *operands.pop();
    };

    while (i < expression.size()) {
        const char c = expression[i];
        if (c == ' ' || c == '\t' || c == '\n') {
            ++i;
            continue;
        }

        // 操作符：+ - * / 各弹两个。注意 '-' 也可能是负号，靠后面是不是数字来区分。
        const bool is_sign = (c == '-' || c == '+') && i + 1 < expression.size()
            && (std::isdigit(static_cast<unsigned char>(expression[i
                ↪ + 1]))
                || expression[i + 1] == '.');
        if (!is_sign && (c == '+' || c == '-' || c == '*' || c == '/')) {
            double left = 0.0;
            double right = 0.0;
            pop_two(left, right);
            switch (c) {
                case '+': operands.push(left + right); break;
                case '-': operands.push(left - right); break;
                case '*': operands.push(left * right); break;
                default:
                    // 原书写 `if (operand1 == 0.0)`，正文又说这样比较浮点数不对、
                    // 该用阈值。两者都值得推敲：除以 1e-300 是 ** 合法 ** 的，
                    // 结果是个很大的数或 inf，用阈值把它当成错误是另一个决定。
                    // 这里只拦精确的 0.0（含 -0.0），并把这个取舍写在明面上。
                    if (right == 0.0) {
                        throw std::domain_error(" 后缀表达式：除数为零");
                    }
            }
        }
    }
}

```

```

        }
        operands.push(left / right);
        break;
    }
    ++i;
    continue;
}

// 操作数：交给标准库解析，顺便拿到它吃掉了多少字符
std::size_t consumed = 0;
double value = 0.0;
try {
    value = std::stod(std::string(expression.substr(i)), &consumed);
} catch (const std::exception&) {
    throw std::invalid_argument(std::string(" 后缀表达式：无法识别的记号 '" +
        ↪ c + "'"));
}
operands.push(value);
i += consumed;
}

if (operands.size() != 1) {
    // 空表达式、操作数多余、操作符不足，都落在这里
    throw std::invalid_argument(" 后缀表达式：不是一个完整的表达式");
}
return *operands.pop();
}

```

原书【算法 3.5】把它写成一个 class Calculator，有三处今天必须改。

一、算法与终端焊死。Run() 直接 cin >> c 读、cout << res 写，出错时 cerr 打一行。后果是这个算法没法写测试、没法在库里复用、也没法处理来自别处的表达式。本书改为接受一个字符串、返回结果、出错抛异常。

二、GetTwoOperands 在第二个操作数缺失时，第一个已经被弹掉了。栈就此少一个元素。但要说准确——真正让这件事变成 **bug** 的是调用方继续跑：Compute 拿到 false 之后清空栈然后接着读下一个记号，Run() 的循环照转不误。

这一点改变了” 怎样才算修好”：本书的实现出错即抛出、整次求值作废，栈根本没有机会被下一步用到。（写这一节时验证过：把实现改回” 先弹一个再查”，全部断言依然通过——因为在抛异常即作废的设计里，栈被破坏与否观测不到。于是补了一条测试，钉住” 上一次失败不给下一次留残留” 这条真正可测的性质。）

三、除零判断。原书正文自己写道：

在实际编写程序时，浮点数是否为 0 不能直接与 0 进行相等比较，而是要通过某个很小的阈值来判断，例如采用 if(abs(operand1) < 1E-7)

但印出来的代码仍然是 `if (operand1 == 0.0)`。书知道更好的写法，却没有印它。

不过原书这条建议本身也值得推敲：除以 `1e-300` 是合法的，结果是一个极大的有限值或无穷，把它当作错误是另一个决定，不是“更正确”。本书只拦精确的零，并把这个取舍写在代码注释里；测试中有一条断言专门钉住“除以 `1e-300` 得到极大值而非报错”。

3.1.5 栈与递归

本节开篇先摆一组实测数字。原书正文说递归「需要在内存中开辟一个称为 运行栈 (runtime stack) 的足够大的动态区」。多大算足够大？这是可以量的，而量出来的结果里有一条相当反直觉。

运行栈有多大：三个档位，三种结果

同一份递归源码（每层做一次加法后返回），在一台 Linux 机器上（gcc 13.3, `ulimit -s` 为默认的 8 MB）逐档加深度：

构建档	20 万层	50 万层	100 万层
-O0（不优化）	通过	段错误	段错误
-O1 + ASan/UBSan	通过	stack-overflow	stack-overflow
-O2	通过	通过	通过

把同样的计算改成显式栈（数据压进本章的 `ArrayStack`，也就是放到堆上）：三个档位在 **1000** 万层都通过。

反直觉的那一条：**-O2** 为什么不崩

不是因为它栈更大，而是因为编译器把递归消掉了。查汇编可以确认：

```
-O0: recursive_sum 函数体内调用自己的次数 = 2
-O2: recursive_sum 函数体内调用自己的次数 = 0
```

-O2 下这个函数根本没有自调用——它被转成了循环。所以：

「这段递归会不会爆栈」不是源码单独决定的，是源码 × 编译器 × 优化档共同决定的。

这件事对学习者的实际含义是：在 -O2 下跑通的递归程序，换成 -O0 调试、或者换个编译器、或者递归形状稍微复杂到无法被转换，就可能在同样的输入上崩掉。而崩掉时你看到什么，也取决于构建方式——开了 sanitizer 的构建会明确告诉你 `stack-overflow` 并给出递归回溯，-O0 的构建只有一个段错误，一行解释都没有。

递归吃运行栈，显式栈吃堆

这正是本节的主题。原书用阶乘作例子，给了三个版本：

【算法 3.6】 递归——每一层的返回地址与局部变量都压在运行栈上，深度由进程栈上限决定：

```
/// 【算法 3.6】 递归实现。保留原书的递归形状——那正是本节要教的东西。
///
/// 加了原书没有的两道检查：负数是定义域错误，溢出是真错误 (D-001 §3)。
/// 原书 `if (n <= 0) return 1;` 把负数静默当成 0 处理，返回 1。
[[nodiscard]] inline factorial_type factorial_recursive(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围 (20! 是上限)");
    }
    if (n <= 1) {
        return 1; // 递归出口
    }
    return static_cast<factorial_type>(n) * factorial_recursive(n - 1);
}
```

【算法 3.8】 迭代——不用栈，也不占深度：

```
/// 【算法 3.8】 迭代实现。不用栈，也不占运行栈深度。
[[nodiscard]] inline factorial_type factorial_iterative(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围 (20! 是上限)");
    }
    factorial_type m = 1;
    for (long long i = 2; i <= n; ++i) {
        m *= static_cast<factorial_type>(i);
    }
    return m;
}
```

【算法 3.9】 显式栈——把待处理的数据压进一个自己管理的栈，用原书的话说是「模拟编译系统处理递归的机制，使用栈等数据结构保存回溯点」：

```
/// 【算法 3.9】 用显式栈模拟递归。
///
/// 这一版存在的意义不是"更快"——它比迭代版慢——而是 ** 演示编译系统处理递归的机制 **:
```

```

/// 遇到递归规则就压栈，遇到递归出口就出栈返回。原书的话是
/// 「模拟编译系统处理递归的机制，使用栈等数据结构保存回溯点」。
///
/// 关键差别在 ** 数据放在哪 **：递归版把每层的返回地址与局部变量放在 ** 运行栈 ** 上，
/// 大小由进程栈上限决定；这一版把待处理的数据压进 ArrayStack，** 在堆上 **，
/// 只受内存限制。实测数字见书稿 3.1.5 节。
///
/// 原书写的是 'while (s.pop(&tmp))'——传的是 ** 指针 **，而同书代码 3.1 的栈 ADT
/// 声明的是 'bool pop(T& item)'（引用）。两处对不上（legacy.md 缺陷 3）。
[[nodiscard]] inline factorial_type factorial_with_explicit_stack(long long n) {
    if (n < 0) {
        throw std::invalid_argument("factorial: 负数没有阶乘");
    }
    if (static_cast<factorial_type>(n) > kMaxFactorialInput) {
        throw std::overflow_error("factorial: 结果超出 64 位无符号范围 (20! 是上限)");
    }
    ArrayStack<factorial_type> pending;
    for (long long i = n; i > 1; --i) { // 按递归规则压栈
        pending.push(static_cast<factorial_type>(i));
    }
    factorial_type m = 1; // 递归出口的返回值
    while (auto top = pending.pop()) { // 出栈即"递归返回"
        m *= *top;
    }
    return m;
}

```

三者的差别不在快慢（显式栈版最慢），而在数据放在哪：前者在运行栈上，受进程栈上限约束；后者在堆上，只受内存约束。上面那张表量的就是这个差别。

原书这三个版本共同的问题：不查溢出

三个版本都是 `long factorial(long n)`，既不检查负数也不检查溢出。64 位 `long` 装得下的最大阶乘是 20!，从 21 开始：

```

factorial(20) = 2432902008176640000
factorial(21) = -4249290049419214848    ← 负数
factorial(66) = 0                        ← 零
factorial(-5) = 1                        ← 负数静默当成 0 处理

```

而且这不只是“答案错”——有符号整数溢出在 C++ 里是未定义行为：

```

runtime error: signed integer overflow: 21 * 2432902008176640000
cannot be represented in type 'long int'

```

本书改用 `std::uint64_t` 并在入口显式判界：超过 20 抛 `std::overflow_error`，负数抛 `std::invalid_argument`。三个版本在边界上的行为完全一致——测试要求它们对同一输入给出相同答案，包括同样地抛出异常。

（另有一处书内不一致：算法 3.9 写 `s.pop(&tmp)` 传指针，而同章代码 3.1 的栈 ADT 声明的是 `bool pop(T& item)` 传引用，两处配不上。详见 `code/ch03/recursion_and_stack/legacy.md`。）

一个更复杂的例子：背包问题

阶乘只有一条递归规则，改写成循环几乎是显然的。原书接着给了一个有两条递归规则的例子——背包问题（更准确地说是子集和判定：能否从若干物品中选出一部分，使重量之和恰好等于背包承重）。

```
/// 【算法 3.10】递归解法。两条递归规则、两个递归出口，原书的结构一字未改：
/// 出口 1：承重恰为 0 → 有解（什么都不再选）
/// 出口 2：承重为负，或承重为正但已无物品可选 → 无解
/// 规则 1：选第  $n-1$  件 → 求解  $\text{knap}(s - w[n-1], n-1)$ 
/// 规则 2：不选第  $n-1$  件 → 求解  $\text{knap}(s, n-1)$ 
///
/// 与原书的差别只有两处：
/// 1. ** 原书直接 `cout << w[n-1]` 把选中的物品打印出来——算法与终端焊死，
///    调用方拿不到结果，也没法测试。这里把下标收集进 `chosen` 返回。
/// 2. ** 原书的 `w[]` 是一个从未声明的全局数组 **（legacy.md 缺陷 3）。这里作参数传入。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_recursive(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);
    knapsack_solution chosen;

    // 返回 true 表示  $\text{weights}[0..n)$  中存在一个子集，其和恰为  $s$ 
    const auto solve = [&weights, &chosen -> bool {
        if (s == 0) {
            return true; // 递归出口 1
        }
        if (s < 0 || n == 0) {
            return false; // 递归出口 2
        }
        if (self(self, s - weights[n - 1], n - 1)) { // 规则 1：选它
            chosen.push_back(n - 1);
            return true;
        }
        return self(self, s, n - 1); // 规则 2：不选它
    };

    return solve(solve, capacity, weights.size())
        ? std::optional<knapsack_solution>(std::move(chosen))
```

```

        : std::nullopt;
    }

```

两个递归出口、两条递归规则，本书一字未改。改的是两处：原书 `w[]` 是一个从未声明的全局数组，这里作参数传入；原书在选中物品时 `cout << w[n-1]` 直接打印，调用方拿不到解、正确性也无从检验，这里把下标收集起来返回——测试因此可以独立验算：把返回的下标对应的重量加起来，必须恰好等于承重。

把它机械地改写成循环

原书的做法是引入一个显式栈，每帧保存四个域：参数 `s` 与 `n`、返回地址 `rd`、结果单元 `k`。返回地址是关键——它记的是“这一层算完之后该回到哪一步继续”，正是编译器为你做的那件事。原书用 `goto label0/1/2/3` 表达它。

本书保留这套机制，但把“返回地址”写成栈帧里的一个 `stage` 字段：

```

/// 【算法 3.11】把上面的递归机械地改写成显式栈驱动。
///
/// 原书用 `goto label0/1/2/3` 表示“执行到哪一步”，本书把同一件事写成栈帧里的
/// 一个 `stage` 字段——** 语义完全对应 **，只是不用 goto: goto 跳进跳出会让编译器
/// 无法保证局部对象的构造与析构配对，在有 RAII 的 C++ 里不能这么写。
///
/// `Enter`      ↔ label0，递归调用入口：判出口，否则按规则 1 展开
/// `AfterRule1` ↔ label1，规则 1（选第 n-1 件）返回后的处理
/// `AfterRule2` ↔ label2，规则 2（不选第 n-1 件）返回后的处理
///
/// 每一帧存原书说的四个域：参数 s 与 n、返回地址（这里是 stage）、结果单元 k。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_with_explicit_stack(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);

    enum class Stage { Enter, AfterRule1, AfterRule2 };
    struct Frame {
        int s = 0;
        std::size_t n = 0;
        Stage stage = Stage::Enter;
    };

    ArrayStack<Frame> stack;
    stack.push(Frame{capacity, weights.size(), Stage::Enter});
    knapsack_solution chosen;
    bool child_result = false;    // 下层刚刚返回的结果单元 k

    while (!stack.empty()) {
        Frame frame = *stack.pop();

```

```

    if (frame.stage == Stage::Enter) {
        if (frame.s == 0) {           // 递归出口 1
            child_result = true;
            continue;                // 相当于 goto label3: 直接向上返回
        }
        if (frame.s < 0 || frame.n == 0) { // 递归出口 2
            child_result = false;
            continue;
        }
        frame.stage = Stage::AfterRule1;    // 记下" 回来时该走哪一步"
        stack.push(frame);
        stack.push(Frame{frame.s - weights[frame.n - 1], frame.n - 1,
↪ Stage::Enter});
        continue;
    }

    if (frame.stage == Stage::AfterRule1) {
        if (child_result) {           // 规则 1 成功: 第 n-1 件被选中
            chosen.push_back(frame.n - 1);
            continue;                // k 已是 true, 继续上传
        }
        frame.stage = Stage::AfterRule2;    // 回溯, 改用规则 2
        stack.push(frame);
        stack.push(Frame{frame.s, frame.n - 1, Stage::Enter});
        continue;
    }

    // Stage::AfterRule2: 规则 2 的结果就是本层的结果, 原样上传
}

return child_result ? std::optional<knapsack_solution>(std::move(chosen)) :
↪ std::nullopt;
}

```

Enter / AfterRule1 / AfterRule2 与原书的 label0 / label1 / label2 一一对应。不用 **goto** 的理由是硬的: goto 跳进跳出会让编译器无法保证局部对象的构造与析构配对, 在有 RAII 的 C++ 里不能这么写。

优化: 让每层要记的东西更少

原书接着指出两点可以省:

1. 结果单元 **k** 可以提到栈外——一旦某层为真就逐层上传且不再变化, 一个函数级变量就够了;
2. 参数 **n** 可以由栈深推出——每递归一层 **n** 减 1、栈深加 1, 所以 $n = n_0 - \text{栈深}$ 。

于是栈帧从四个域缩到两个:

```

/// 【算法 3.12】原书" 优化版" 的两点观察，本书照单实现：
///
/// 1. ** 结果单元  $k$  可以提到栈外 **——一旦某层为 true 就逐层上传且不再变化，
///    因此一个函数级变量即可，栈帧里的 ` $k$ ` 域连同它的反复赋值、进出栈都省掉。
///    （上面那版其实已经这么做了：`child_result` 就在栈外。）
/// 2. ** 参数  $n$  可以由栈深推出 **——每递归一层  $n$  减 1、栈深加 1，
///    所以 ` $n = n_0 - \text{栈深}$ `，栈帧里的 ` $n$ ` 域也能省掉。
///
/// 于是栈帧从四个域缩到两个 ( $s$  与  $stage$ )。这是本节真正的" 优化"：
/// 不是让它更快，而是 ** 让每层要记的东西更少 **——这正是手工模拟递归的意义。
///
/// 原书这一版另有一处致命问题：它同时把 `stack.top` 当 ** 数据成员 ** 用
/// (t = stack.top;) 又当 ** 成员函数 ** 用 (stack.top(&tmp);)。
/// 这在任何一种解释下都编译不过，而且它恰恰依赖代码 3.2/3.4 那个 `top` 重名缺陷。
[[nodiscard]] inline std::optional<knapsack_solution> knapsack_optimized(
    int capacity, const std::vector<int>& weights) {
    detail::validate(capacity, weights);

    enum class Stage { Enter, AfterRule1, AfterRule2 };
    struct Frame {
        int s = 0; // 只剩两个域
        Stage stage = Stage::Enter;
    };

    const std::size_t n0 = weights.size();
    ArrayStack<Frame> stack;
    knapsack_solution chosen;
    bool child_result = false;

    stack.push(Frame{capacity, Stage::Enter});
    std::size_t depth = 1; // 栈中帧数；当前帧的  $n = n_0 - (\text{depth} - 1)$ 

    while (!stack.empty()) {
        Frame frame = *stack.pop();
        --depth;
        const std::size_t n = n0 - depth; // 观察 2:  $n$  由栈深推出，不再入栈

        if (frame.stage == Stage::Enter) {
            if (frame.s == 0) { child_result = true; continue; }
            if (frame.s < 0 || n == 0) { child_result = false; continue; }
            frame.stage = Stage::AfterRule1;
            stack.push(frame);
            stack.push(Frame{frame.s - weights[n - 1], Stage::Enter});
            depth += 2;
            continue;
        }
    }
}

```

```

    if (frame.stage == Stage::AfterRule1) {
        if (child_result) { chosen.push_back(n - 1); continue; }
        frame.stage = Stage::AfterRule2;
        stack.push(frame);
        stack.push(Frame{frame.s, Stage::Enter});
        depth += 2;
        continue;
    }
    // AfterRule2: 结果原样上传
}

return child_result ? std::optional<knapsack_solution>(std::move(chosen)) :
    ↪ std::nullopt;
}

```

这才是本节真正的”优化”：不是让它跑得更快，而是让每层要记的东西更少。手工模拟递归的全部意义就在这里——你必须想清楚”每一层到底需要记住什么”，而这正是编译器替你想过的那个问题。

一句实话：写这个单元时，第一版显式栈实现我把”返回地址”塞进了位标志里，结果死循环、撞上构建超时；改写成与原书 label 一一对应的状态机之后才一次通过。手工模拟递归确实容易写错——而原书那份用 goto 的版本从未被编译器验证过。算法 3.12 里 stack.top 同时被当成数据成员（t = stack.top;）和成员函数（stack.top(&tmp);），任何一种解释下都编译不过。

与原书的对照

原书	现在	为什么
class Stack<T> 空基类	三条 static_assert + <type_traits>	空基类给不了多态，还带来非虚析构的未定义行为；对 T 的要求应当是编译期约束
int mSize / int top / T* st	size_t capacity_ / size_t top_index_ / T* data_	int 溢出是未定义行为；top_index_ 避开与成员函数 top() 重名
只有 ~arrStack() bool pop(T& item)	五个特殊成员函数补齐 [[nodiscard]] std::optional<T> pop()	否则一次拷贝就二次释放「有没有值」交给类型系统，忽略返回值会告警

原书	现在	为什么
<code>bool top(T& item)</code>	<code>optional<T> top()</code> 取副本; <code>const T* peek()</code> 零拷贝	两种正当需求, 两种代价, 都摆在明面上
<code>cout << " 栈满溢出"</code>	不做任何 I/O; 越界抛 <code>std::out_of_range</code>	数据结构不该和标准输出耦合; 可预期的空状态用 <code>optional</code> , 真错误才抛异常
<code>delete[] st; st = newSt;</code>	先建后换 + <code>try/catch</code> , 按移动赋值是否 <code>noexcept</code> 决定移动还是拷贝	搬迁中途抛异常不破坏原栈 (强异常保证); 判据落在实际执行的操作上, 不是 <code>move_if_noexcept</code> 看的移动构造

刻意没改的: 它仍然是一个手写的、自己管缓冲区的数组栈, 缓冲区仍是裸的 `T* data_`, 扩容仍是算法 3.3 的翻倍策略, `clear()` 仍然只把长度归零、保留已分配容量。把它换成 `std::stack` 的薄封装固然更短, 但这一节要教的正是这些实现细节本身。

完整实现见 `code/ch03/array_stack/modern.hpp`, 测试见同目录 `test.cpp` (58 项断言, 覆盖上表每一行; 用 `python3 tools/check_code.py` 在 `-Werror + ASan/UBSan` 与 `-O2` 两种构建下各跑一遍)。

3.2 队列

队列只允许在一端插入、在另一端删除, 元素按先进先出 (FIFO) 访问。插入端是队尾, 删除端是队头。空队列上的提取是可预期状态, 返回 `std::nullopt`。

3.2.1 队列的抽象数据类型

常用运算是入队 `enqueue`、出队 `dequeue`、读队头, 以及判空。顺序实现还需要判满。原书【代码 3.13】同样把这些运算写成一个假抽象基类; 本书直接定义在 `ArrayQueue` 与 `LinkedQueue` 上。

3.2.2 顺序队列

循环队列牺牲一个槽位区分空与满: 逻辑容量为 `n` 时实际申请 `n+1` 个槽。`front_ == rear_` 为空, `(rear_ + 1) % slots_ == front_` 为满。这正是章首 demo 里「容量 3 时第 4 个入不进去」的原因。

```

template <typename T>
class ArrayQueue {
public:
    explicit ArrayQueue(std::size_t capacity) : slots_(capacity + 1), data_(slots_ ?
        ↪ new T[slots_] : nullptr) {}
    ArrayQueue(const ArrayQueue& other) : slots_(other.slots_),
    ↪ front_(other.front_), rear_(other.rear_), data_(slots_ ? new T[slots_] :
    ↪ nullptr) { for (std::size_t i = front_; i != rear_; i = (i + 1) % slots_)
    ↪ data_[i] = other.data_[i]; }
    ArrayQueue& operator=(const ArrayQueue& other) { if (this != &other) {
    ↪ ArrayQueue copy(other); swap(copy); } return *this; }
    ArrayQueue(ArrayQueue&& other) noexcept { swap(other); }
    ArrayQueue& operator=(ArrayQueue&& other) noexcept { if (this != &other) {
    ↪ ArrayQueue moved(std::move(other)); swap(moved); } return *this; }
    ~ArrayQueue() { delete[] data_; }
    void swap(ArrayQueue& other) noexcept { using std::swap; swap(slots_,
    ↪ other.slots_); swap(front_, other.front_); swap(rear_, other.rear_);
    ↪ swap(data_, other.data_); }
    [[nodiscard]] bool empty() const noexcept { return front_ == rear_; }
    [[nodiscard]] bool full() const noexcept { return slots_ != 0 && (rear_ + 1) %
    ↪ slots_ == front_; }
    [[nodiscard]] std::size_t size() const noexcept { return rear_ >= front_ ? rear_
    ↪ - front_ : slots_ - front_ + rear_; }
    [[nodiscard]] bool enqueue(const T& value) { if (full()) return false;
    ↪ data_[rear_] = value; rear_ = (rear_ + 1) % slots_; return true; }
    [[nodiscard]] bool enqueue(T&& value) { if (full()) return false; data_[rear_] =
    ↪ std::move(value); rear_ = (rear_ + 1) % slots_; return true; }
    [[nodiscard]] std::optional<T> dequeue() { if (empty()) return std::nullopt; T
    ↪ value = std::move(data_[front_]); front_ = (front_ + 1) % slots_; return
    ↪ value; }
    [[nodiscard]] const T* front() const noexcept { return empty() ? nullptr :
    ↪ &data_[front_]; }
    void clear() noexcept { front_ = rear_ = 0; }
private: std::size_t slots_{0}, front_{0}, rear_{0}; T* data_{nullptr};
};

```

3.2.3 链式队列

链式队列用首尾指针维持 FIFO，入队接在尾、出队摘下头，并具备独立复制所有权。

```

template <typename T>
class LinkedQueue {
    struct Node { T value; Node* next{nullptr}; template <typename U> explicit
    ↪ Node(U&& value) : value(std::forward<U>(value)) {} };

```

```

public:
    LinkedQueue() = default;
    LinkedQueue(const LinkedQueue& other) { for (Node* n = other.front_; n !=
    ↪ nullptr; n = n->next) enqueue(n->value); }
    LinkedQueue& operator=(const LinkedQueue& other) { if (this != &other) {
    ↪ LinkedQueue copy(other); swap(copy); } return *this; }
    LinkedQueue(LinkedQueue&& other) noexcept { swap(other); }
    LinkedQueue& operator=(LinkedQueue&& other) noexcept { if (this != &other) {
    ↪ LinkedQueue moved(std::move(other)); swap(moved); } return *this; }
    ~LinkedQueue() { clear(); }
    void swap(LinkedQueue& other) noexcept { using std::swap; swap(front_,
    ↪ other.front_); swap(rear_, other.rear_); swap(size_, other.size_); }
    [[nodiscard]] bool empty() const noexcept { return front_ == nullptr; }
    [[nodiscard]] std::size_t size() const noexcept { return size_; }
    void enqueue(const T& value) { append(new Node(value)); }
    void enqueue(T&& value) { append(new Node(std::move(value))); }
    [[nodiscard]] std::optional<T> dequeue() { if (front_ == nullptr) return
    ↪ std::nullopt; Node* old = front_; front_ = old->next; if (front_ == nullptr)
    ↪ rear_ = nullptr; --size_; T value = std::move(old->value); delete old;
    ↪ return value; }
    [[nodiscard]] const T* front() const noexcept { return front_ == nullptr ?
    ↪ nullptr : &front_->value; }
    void clear() noexcept { while (front_ != nullptr) { Node* old = front_; front_ =
    ↪ old->next; delete old; } rear_ = nullptr; size_ = 0; }
private: void append(Node* node) noexcept { if (rear_ == nullptr) front_ = rear_ =
    ↪ node; else { rear_->next = node; rear_ = node; } ++size_; } Node*
    ↪ front_{nullptr}; Node* rear_{nullptr}; std::size_t size_{0};
};

```

3.3 栈与队列的比较

3.3.1 顺序栈与链式栈

顺序栈预分配连续缓冲区，扩容要搬迁；链式栈按需分配，没有「栈满」，但每个结点多一个指针。两者的 push/pop 都是 $O(1)$ 。

3.3.2 顺序队列与链式队列

两种队列的端点操作都是常数时间。循环数组适合容量上界已知、希望局部性好的场合；链表适合长度变化大、不愿预留空洞槽位的场合。

3.3.3 限制存取点的表

栈和队列都是限制存取点的线性表：栈只开一端，队列开两端但方向相反。双端队列同时开放两端，不在本章实现范围内。

第 4 章 字符串

字符串是字符的有限序列。先区分「保存字符串」的类与「在文本中找模式」的算法：前者处理容量、复制和下标边界，后者处理比较顺序。朴素匹配在失配后移动模式；KMP 预先计算 next 信息，避免重复比较已经知道相等的前缀。

源码：字符串类、字符串示例、模式匹配、匹配示例。

先跑一遍

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::String text = "Hello";
    text.append(' ').append('C').append('+').append('+');
    std::cout << " 拼接后: " << text.c_str() << '\n';
    const auto slice = text.substr(6, 3);
    std::cout << " 子串: " << slice.c_str() << '\n';
    const auto found = text.find('C');
    std::cout << " 首次出现 C 的下标: ";
    if (found) {
        std::cout << *found << '\n';
    } else {
        std::cout << " 无\n";
    }
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch04/string_class \
    code/ch04/string_class/demo.cpp -o /tmp/str-demo
/tmp/str-demo
```

```
拼接后: Hello C++
子串: C++
首次出现 C 的下标: 6
```

缓冲区仍是手写的 `char*`，换成 `std::string` 这一节就没了。`find` 找不到时返回空 optional，不用 `-1` 和位置 `0` 抢同一个数字。

图 4.12 自己的那对串，原书两个匹配算法都返回 11，正确答案是 10：

本书的 String 把这三处分别改为:类名大写、bool empty()、std::optional<size_type> find(...)、修改器返回 String&。

```
/// 变长字符串。内部保存一块以 '\0' 结尾的字符数组和当前长度。
///
/// 与原书 String 的差别: 构造函数取 const char* (原书取 char*,
/// 使书中自己的例子 `String s1 = "Hello";` 在 C++11 起就是非法转换);
/// 越界抛 std::out_of_range, 而不是 `return NULL` 让调用方拿到一个必然崩溃的对象;
/// 补齐五法则; 不做任何 I/O。
class String {
public:
    using size_type = std::size_t;

    /// 空串。注意它仍然持有一块 1 字节的缓冲区, 于是 c_str() 永远可用、永不为空指针。
    String() : data_(new char[1]{'\0'}), size_(0) {}

    /// 从 C 字符串构造。故意 ** 不加 explicit**: 原书 `String s1 = "Hello";` 这种写法
    /// 是本节的教学用例, 保留它; 代价是隐式转换, 值得知道但这里可以接受。
    String(const char* s) { // NOLINT(google-explicit-constructor)
        if (s == nullptr) {
            // 原书的 Substr 在越界时 `return NULL`, 随后 strlen(nullptr) 当场 SEGV。
            // 这里把它挡在门口, 并且说清楚是什么问题。
            throw std::invalid_argument("String: 不能用空指针构造字符串");
        }
        size_ = std::strlen(s);
        data_ = new char[size_ + 1];
        std::memcpy(data_, s, size_ + 1);
    }
}
```

4.2 字符串的存储结构和实现

String 采用动态变长的存储结构: 内部持有一块以 '\0' 结尾的字符数组和当前长度, 构造时按初值长度分配, 赋值时按新长度重新分配。这正是本节要教的内容, 所以缓冲区是裸 char*——换成 std::string 这一节就没了。

4.2.1 构造与所有权

原书【算法 4.4】的构造函数有两处问题。

第一处, assert(str != '\0') 本身就编译不过: '\0' 是 char 而不是空指针常量, 拿指针与它比较是 ill-formed (error: ISO C++ forbids comparison between pointer and integer)。而且即便改写成 assert(str != nullptr) 也是无效断言——new 失败时抛 std::bad_alloc, 从不返回空指针; assert 更会在 NDEBUG 构建里整个消失。

第二处, 参数类型是 char*。原书 4.2.2 节自己写下:

String s1 = "Hello"; 隐含地调用构造函数 String::String(char* s)

而字符串字面量的类型是 const char[6]，转成 char* 在 C++11 起已被移除。GCC 默认降级为警告，在本书的 -Werror 构建下即是错误。

还有一处原书没有做的检查：构造函数对 s == nullptr 毫无防备——而 4.2.3 节的 Substr 恰好会喂给它一个空指针（见 4.2.3 节）。

原书的类里有 ~string()，却从未把拷贝构造与拷贝赋值作为清单给出（正文描述过赋值时“必须释放 s1 的原有空间”，但没有代码）。只要有析构而没有这两个，一次 String b = a; 就是二次释放——与第 2、3 章是同一个错误。

```
// 原书只在正文里描述了赋值时“必须释放 s1 的原有空间 (delete [] s1.str)",
// 却没有把拷贝构造和拷贝赋值作为清单给出。只要有析构函数而没有这两个,
// 一次 `String b = a;` 就是二次释放——与第 2、3 章是同一个错误。
String(const String& other) : data_(new char[other.size_ + 1]), size_(other.size_) {
    // 注意读的是 other.raw() 不是 other.data_: 源可能是被移动过的对象 (data_ 为空),
    // 从空指针 memcpy 即便长度为 0 也是未定义行为。
    std::memcpy(data_, other.raw(), size_ + 1);
}

String& operator=(const String& other) {
    if (this != &other) {
        String copy(other); // 拷贝并交换: 自赋值安全, 且拷贝失败时原对象不受影响
        swap(copy);
    }
    return *this;
}

/// 移动 ** 不分配 **: 直接接管缓冲区, 把对方置为 nullptr。
/// 被移动方仍是一个可用的空串——读取路径统一走 raw(), 它在 data_ 为空时返回 ""。
/// (若在这里 new 一块空缓冲区来"修复"对方, 移动就不再是 noexcept 的了。)
String(String&& other) noexcept : data_(other.data_), size_(other.size_) {
    other.data_ = nullptr;
    other.size_ = 0;
}

String& operator=(String&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}
```

```
~String() { delete[] data_; }
```

移动操作声明为 `noexcept`，因此不能在里面分配。被移动方的指针置空，读取路径统一走一个私有的 `raw()`：指针为空时返回静态空串。于是“被移动之后仍是可用的空串”这个保证不花任何分配就成立，`c_str()` 也永远不会是空指针。

4.2.2 追加与拼接

```
/// 在串尾添加一个字符，返回自身引用。
///
/// 原书【代码 4.1】声明的是 `string append(const char c);`——** 按值返回 **。
/// 代码 4.1 只有声明没有函数体，所以「它到底改不改本串」在书里无从查证；
/// 而这正是问题所在：一个修改器按值返回，调用方无法从签名判断
/// `s.append('x');` 是改了 s 还是返回了一个新串而 s 原封不动。
/// 返回自身引用把这件事说死，同时支持链式调用。
String& append(char c) {
    char* fresh = new char[size_ + 2];
    std::memcpy(fresh, raw(), size_);
    fresh[size_] = c;
    fresh[size_ + 1] = '\0';
    delete[] data_;
    data_ = fresh;
    ++size_;
    return *this;
}

/// 把 s 连接在本串后面。s 为空指针时抛 std::invalid_argument。
String& concatenate(const char* s) {
    if (s == nullptr) {
        throw std::invalid_argument("String::concatenate: 空指针");
    }
    const size_type extra = std::strlen(s);
    char* fresh = new char[size_ + extra + 1];
    std::memcpy(fresh, raw(), size_);
    std::memcpy(fresh + size_, s, extra + 1);
    delete[] data_;
    data_ = fresh;
    size_ += extra;
    return *this;
}

String& operator+=(char c) { return append(c); }
String& operator+=(const String& other) { return concatenate(other.c_str()); }
```

每次追加都重新分配一块、拷贝、释放旧块——这就是变长串管理的代价，也

是后面章节讨论“预留容量”的动机。

4.2.3 抽取子串

原书【算法 4.5】在起始位置越界时写 `return NULL;`。这里的返回类型是 `String`，所以 `NULL` 不是“空串”：它先转成 `char*`，再走 `String(char*)` 构造函数，于是 `strlen(nullptr)`。这段能编译，崩在运行期：

```
$ ./s7
准备调用 s.Substr(99, 1)——原书会走到 return NULL 那一支
runtime error: null pointer passed as argument 1, which is declared to never be null
AddressSanitizer:DEADLYSIGNAL
ERROR: AddressSanitizer: SEGV on unknown address 0x000000000000
```

本书越界就抛 `std::out_of_range`，让错误停在发生的地方，而不是变成调用方某处的段错误。

```
/// 从 pos 开始抽取长度至多为 len 的子串。
///
/// 原书【算法 4.5】在 `pos >= size` 时 `return NULL;`——那不是“返回空串”，
/// 而是拿 NULL 走 String(char*) 构造函数，接着 strlen(nullptr) 当场崩溃
/// （证据见 legacy.md 缺陷 3）。这里越界就抛 std::out_of_range，
/// 让错误停在发生的地方，而不是变成调用方某处的段错误。
///
/// pos == size() 是合法的，得到空串——与“从末尾取 0 个字符”的直觉一致。
[[nodiscard]] String substr(size_type pos, size_type len) const {
    if (pos > size_) {
        throw std::out_of_range("String::substr: 起始位置越界");
    }
    const size_type available = size_ - pos;
    const size_type take = len < available ? len : available; // 原书的 if (n >
    ↪ left) n = left
    String result;
    char* fresh = new char[take + 1];
    std::memcpy(fresh, raw() + pos, take);
    fresh[take] = '\0';
    delete[] result.data_;
    result.data_ = fresh;
    result.size_ = take;
    return result;
}
```

`len` 超出剩余长度时截断而不报错，与原书的 `if (n > left) n = left;` 语义一致。

4.2.4 查找与比较

```
/// 从 start 开始查找字符 c，返回下标；没有则 std::nullopt。
/// 原书 `int find(const char c, const int start)` 用 -1 表示没找到，
/// 与" 位置 0" 只差一个符号。
[[nodiscard]] std::optional<size_type> find(char c, size_type start = 0) const {
    for (size_type i = start; i < size_; ++i) {
        if (raw()[i] == c) {
            return i;
        }
    }
    return std::nullopt;
}

/// 三路比较，负/零/正 表示 小于/等于/大于。
///
/// 原书【算法 4.3】自己实现了一个 strcmp，返回值固定为 -1/0/1，
/// 并在正文里指出" 这与 C/C++ 语言中通常的大小比较习惯（0 和非 0）不一致"——
/// 其实不一致的是原书自己：标准 strcmp 返回的就是差值的符号，
/// 调用方只该看符号，不该看具体数值。这里保持标准语义。
[[nodiscard]] int compare(const String& other) const noexcept {
    return std::strcmp(raw(), other.raw());
}
```

关于【算法 4.3】：原书自己实现了一个 strcmp，固定返回 -1/0/1，并在正文里说“这与 C/C++ 语言中通常的大小比较习惯（0 和非 0）不一致”。其实标准 strcmp 返回的就是差值的符号，调用方本来就只该看符号——不一致的是原书固定返回 ± 1 的写法。本书保持标准语义，并据此提供关系运算符。

（另外补一句实测结论：原书那个与标准库同名同签名的 strcmp 定义，既能编译也能链接，不构成冲突——这是我们查过之后否掉的一个猜测，记在 legacy.md 第五节。）

4.3 字符串的模式匹配

给定目标串 T 和模式串 P，模式匹配要回答：P 是否作为 T 的连续子串出现，若出现，首次出现在哪个位置。

本节的教学内容是算法本身，因此下面的实现用 std::string_view 接收输入：不拷贝、不拥有，把注意力留在匹配过程上。字符串容器的实现是 4.2 节的事。

4.3.1 朴素的模式匹配算法

朴素算法的思路直白：把模式的首字符对齐目标的每一个位置，逐字符比较；一旦失配，就把模式整体右移一位，从头再来。

```
/// 朴素模式匹配：返回 pattern 在 text 中首次出现的 ** 起始下标 **；没有则 std::nullopt。
///
/// 与原书【算法 4.6】的关键差别是返回值：原书写的是 `return (j - pLen + 1);`，
/// 而在 0 起始的下标体系里正确的是 `j - pLen`——** 原书这里差了 1**。
```

/// 用书中自己的例子可以当场看出来：T="abcddabcab..."、P="abcdaabcab" 匹配始于下标 10，
/// 原书返回 11（证据见 *legacy.md* 缺陷 1）。
///
/// 空模式约定返回 0（与 `std::string::find("")` 一致）；原书用 `assert(m>0)` 把它挡在门外，
/// 而 `assert` 在 *NDEBUG* 下会被整个编译掉。

```
[[nodiscard]] inline std::optional<std::size_t> naive_search(std::string_view text,
                                                             std::string_view pattern) {

    const std::size_t n = text.size();
    const std::size_t m = pattern.size();
    if (m == 0) {
        return std::size_t{0};
    }
    if (n < m) {
        return std::nullopt;
    }
    std::size_t i = 0; // 模式下标
    std::size_t j = 0; // 目标下标
    while (i < m && j < n) {
        if (text[j] == pattern[i]) {
            ++i;
            ++j;
        } else {
            j = j - i + 1; // 回退到本趟起点的下一个位置
            i = 0;
        }
    }
    return i >= m ? std::optional<std::size_t>(j - m) : std::nullopt;
}
```

失配时 `j = j - i + 1` 把目标下标退回本趟起点的下一个位置——这一步的“+1”是对的，它表示“换一个起点重来”。最坏情况下，每个起点都要比较到模式末尾才失配（例如 T = “aaa...a”、P = “aa...ab”），总比较次数为 $O(|T| \cdot |P|)$ 。

一处必须指出的错误：原书的返回值差 1

原书【算法 4.6】与【算法 4.8】在匹配成功时都写：

```
if (i >= pLen) return (j - pLen + 1);
```

匹配成功时 j 已经走到匹配段的末尾之后，所以起始位置是 j - pLen。那个 +1 是多余的。把原书两段代码照抄进程序，拿标准库的 find 做参照：

T=abc	P=abc	原书朴素	= 1	正确答案	= 0
T=xabc	P=abc	原书朴素	= 2	正确答案	= 1
T=aaab	P=ab	原书朴素	= 3	正确答案	= 2
T=abcdcdabcababcdaabcbabcbdaabcbabaa	P=abcdaaabcbab	原书朴素	= 11	正确答案	= 10

每一组都恰好多 1，最后一组正是书中图 4.12 自己用来演示 KMP 的那对串。原书用它逐趟画了匹配过程，却没有给出返回值，这个错误因此在书里没有暴露。

这不是排版或 OCR 造成的，有三重佐证： $j - pLen + 1$ 在两个算法里独立印出、写法一致；同一段代码里的 $j = j - i + 1$ 说明作者用的就是 0 起始下标；而 4.3 节开头的约定更是把这一点写死了——

P 和 T 的第一个字符都从位置 0 开始。

本书的实现返回 j - m，并且每一条匹配用例都拿标准库的 **find** 逐个对拍，再加 3000 组随机对拍。只断言”找到了”的测试，在原书那份实现下同样全绿——那样的测试等于没写。

4.3.2 字符串的特征向量

KMP 的想法是：失配时不必把模式退回从头开始，因为已经匹配上的那一段本身携带了信息——它的最长相同前后缀长度决定了模式可以直接右移多少。把这个信息对模式的每个位置预先算出来，就是特征向量（next 数组）。

```

4.7.1 计算模式的特征向量 (next 数组)，原书【算法 4.7】的"优化版"。
//
// 与原书的差别只有所有权：原书 `int* findNext(String P)` 用 `new int[m]` 返回裸数组，
// 而书中 ** 从未展示过与之配对的 `delete[]`——每调用一次泄漏一个数组。
// 计算过程一字未改，包括 `next[i] = next[k]` 这一步优化。
//
// 空模式返回空向量；原书是 `assert(m > 0)`，而 `assert` 在 `NDEBUG` 下会被编译掉，
// 于是 `release` 构建里 `new int[0]` 加 `next[0] = -1` 就是一次越界写。
[[nodiscard]] inline std::vector<next_type> build_next(std::string_view pattern) {
    const std::size_t m = pattern.size();
    std::vector<next_type> next(m);
    if (m == 0) {
        return next;
    }
    next_type i = 0;
    next_type k = -1;

```

```

next[0] = -1;
while (i < static_cast<next_type>(m)) {
    while (k >= 0 && pattern[static_cast<std::size_t>(i)] !=
        ↪ pattern[static_cast<std::size_t>(k)]) {
        k = next[static_cast<std::size_t>(k)]; // 沿已算好的特征值回退
    }
    ++i;
    ++k;
    if (i == static_cast<next_type>(m)) {
        break;
    }
    const auto ui = static_cast<std::size_t>(i);
    const auto uk = static_cast<std::size_t>(k);
    // P[i] 与 P[k] 相等时可以直接借用 next[k], 省掉一次注定失败的比较——这就是“优化
    ↪ 版”。
    next[ui] = (pattern[ui] == pattern[uk]) ? next[uk] : k;
}
return next;
}

```

next[i] = next[k] 那一步是原书所说的”优化”：如果 P[i] 与 P[k] 相同，那么用 k 作为回退目标必然会再失配一次，不如直接借用 next[k] 一步到位。

对模式 "abcdaabcb", 算法产生：

i	0	1	2	3	4	5	6	7	8	9
P[i]	a	b	c	d	a	a	b	c	a	b
next[i]	-1	0	0	0	-1	1	0	0	3	0

这与原书图 4.11 最后一行完全一致。

注意原书正文与图 4.11 不一致。正文写的是 next = {-1,0,0,0,0,-1,1,0,0,3,0}，数一下是 11 个值，而模式只有 10 个字符。图 4.11 给的是 10 个值，与算法实算的结果相符。正文里多出来的那个 0 是错的。本书的测试逐个比对图 4.11 的十个值，并单独断言”模式只有 10 个字符”，把这处矛盾钉住。

关于所有权：原书 findNext 用 new int[m] 返回裸数组，而书中调用它的地方——算法 4.8 的 N 参数、正文的示例——一次都没有出现配对的 delete[]。每匹配一个模式就漏一个数组。本书返回一个拥有所有权的容器，计算过程一字未改。

4.3.3 KMP 模式匹配算法

有了特征向量，匹配过程与朴素算法几乎一样，只有失配那一步不同：不再把模式退回开头，而是退到 next[i]。

```

/// KMP 模式匹配。失配时不再把模式右移一位，而是按特征值决定右移多少。
///
/// 返回值与 naive_search 一致，也修正了原书【算法 4.8】同样的差一错误。
/// next 由调用方传入：同一个模式可以只算一次、多次匹配复用——
/// 这正是原书强调的性质，接口把它显式表达出来。
[[nodiscard]] inline std::optional<std::size_t> kmp_search(std::string_view text,
                                                           std::string_view pattern,
                                                           const std::vector<next_type>&
                                                           ↪ next) {

    const std::size_t n = text.size();
    const std::size_t m = pattern.size();
    if (m == 0) {
        return std::size_t{0};
    }
    if (next.size() != m) {
        throw std::invalid_argument("kmp_search: next 数组长度与模式不符");
    }
    if (n < m) {
        return std::nullopt;
    }
    next_type i = 0;    // 模式下标，可以退到 -1
    std::size_t j = 0;  // 目标下标，只增不减
    while (i < static_cast<next_type>(m) && j < n) {
        if (i == -1 || text[j] == pattern[static_cast<std::size_t>(i)]) {
            ++i;
            ++j;
        } else {
            i = next[static_cast<std::size_t>(i)];
        }
    }
    return i >= static_cast<next_type>(m) ? std::optional<std::size_t>(j - m) :
    ↪ std::nullopt;
}

/// 便利重载：模式只用一次时，自己把 next 算掉。
[[nodiscard]] inline std::optional<std::size_t> kmp_search(std::string_view text,
                                                           std::string_view pattern) {
    return kmp_search(text, pattern, build_next(pattern));
}

```

时间代价的关键在于目标下标 *j* 只增不减。循环体里 *++j* 至多执行 $|T|$ 次，与它同处一条语句的 *++i* 也不超过 $|T|$ 次；能让 *i* 减少的只有 $i = \text{next}[i]$ ，而 $\text{next}[i] < i$ ，所以每执行一次 *i* 至少减 1；一旦减到 -1，下一步必然进入 *++i*, *++j* 分支。因此 $i = \text{next}[i]$ 的执行次数不超过 *++i*, *++j* 的次数加 1，整个循环体至多执行 $2|T| + 1$ 次——与目标串长度成线性关系。

加上计算 next 的 $O(P)$ ，KMP 整体为 $O(|P| + |T|)$ 。而同一个模式的特征向量 只需算一次即可跨多个目标复用，这一点接口里用「next 由调用方传入」显式表达。

本书的测试用朴素算法的最坏情况 ($T = 20$ 万个 ‘a’， $P = 1999$ 个 ‘a’ 加一个 ‘b’) 验证这条线性性质：KMP 在其上瞬间完成，而失配后不按特征值回退的实现会退化成 $O(|T| \cdot |P|)$ ，直接撞上构建闸门的超时。

与原书的对照

原书	现在	为什么
<code>return (j - pLen + 1)</code>	<code>return j - m</code>	原书差 1 ，四组数据逐个对拍证实
<code>int</code> 返回值，-1 表示没找到	<code>std::optional<std::size_t></code> 与”位置 0” 只差一个符号，漏判就把未匹配当成匹配在开头	
<code>int* findNext()</code> 返回 <code>new int[]</code>	返回拥有所有权的容器	书中从未展示配对的 <code>delete[]</code> ，每次调用漏一个数组
<code>assert(m > 0)</code>	空模式返回 0	<code>assert</code> 在 NDEBUG 下整个消失， <code>release</code> 构建里是越界写
<code>assert(next != 0)</code>	删除	<code>new</code> 失败抛 <code>bad_alloc</code> ，从不返回空指针，该断言永远为真

刻意没改的：朴素匹配仍是回溯式的；next 仍是原书的优化版（图 4.11 对的是这一版）；KMP 的 next 仍由调用方传入并可跨目标复用。这三条是本节全部的教学内容。

完整实现见 `code/ch04/pattern_matching/modern.hpp`，测试见同目录 `test.cpp` (56 项断言，含 3000 组随机对拍；用 `python3 tools/check_code.py` 在 `-Werror + ASan/UBSan` 与 `-O2` 两种构建下各跑一遍)。

第 5 章 二叉树

二叉树的每个结点最多有左、右两个孩子。周游回答「以什么顺序访问全部结点」；二叉搜索树加上左小右大；堆把最小（或最大）值放在根；Huffman 树不断合并两个最小权值。

源码：二叉树与二叉搜索树、最小堆与 Huffman 树、树的示例、堆与 Huffman 的示例。

5.1 二叉树的概念

二叉树由结点的有限集合构成：或者为空，或者由一个根和两棵互不相交的左、右子树组成。左右次序不能颠倒。根没有父结点；其余每个结点恰有一个父结点，至多两个孩子。没有孩子的是叶，其余是内部结点。从根到某结点的边数是该结点的层数，根在第 0 层。

5.1.1 定义和基本术语

下面这棵树：



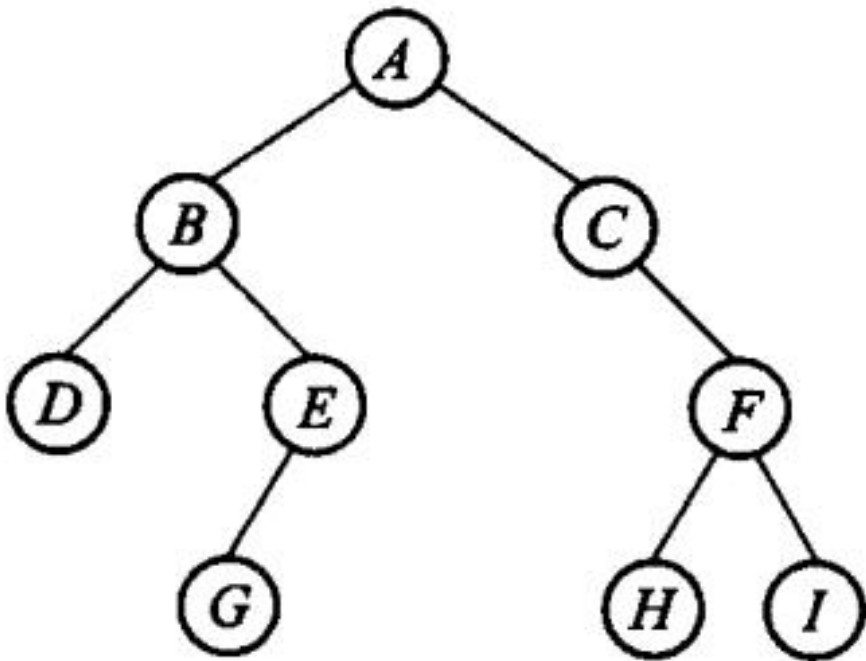


图 5.5 二叉树示例

四种周游访问的是同一组结点，次序不同：

周游	顺序	本例
先序	根，左，右	A B D E C
中序	左，根，右	D B E A C
后序	左，右，根	D E B C A
层次	按离根的距离	A B C D E

递归版最贴合定义，也是 5.2 节要教的东西。极深的退化树会耗尽调用栈：Release 档大约在百万层段错误且没有诊断，ASan 档会打印 `stack-overflow` 并指到具体行。析构和拷贝走的也是递归，调用方看不见。数字和复现程序见仓库中的未验证风险说明。

5.1.2 满二叉树、完全二叉树、扩充二叉树

任何结点或者是叶，或者左右子树都非空，叫做满二叉树。叶只出现在最下两层、且最下层靠左对齐，叫做完全二叉树。在空子树位置补上空树叶，得到扩充二叉树；外部路径长度 E 与内部路径长度 I 满足 $E = I + 2n$ 。

5.1.3 主要性质

第 i 层至多 2^i 个结点；深度为 k 的二叉树至多 $2^{k+1} - 1$ 个结点；叶结点数 $n_0 = n_2 + 1$ 。 n 个结点的完全二叉树高度为 $\lceil \log_2(n+1) \rceil$ 。按层从 0 编号时，结点 i 的父是 $\lfloor (i-1)/2 \rfloor$ ，左右孩子是 $2i+1$ 与 $2i+2$ 。这些性质没错，原样保留。

5.2 二叉树的周游

5.2.1 先跑一遍

先建树并打印四种周游，再插一棵 BST：

```
#include "modern.hpp"

#include <iostream>
#include <utility>

int main() {
    dsa::BinaryTree<char> left_leaf;
    dsa::BinaryTree<char> right_leaf;
    dsa::BinaryTree<char> left;
    dsa::BinaryTree<char> right;
    dsa::BinaryTree<char> root;
    left_leaf.create_tree('D');
    right_leaf.create_tree('E');
    left.create_tree('B', std::move(left_leaf), std::move(right_leaf));
    right.create_tree('C');
    root.create_tree('A', std::move(left), std::move(right));

    std::cout << " 先序: ";
    root.preorder([](char value) { std::cout << value; });
    std::cout << "\n中序: ";
    root.inorder([](char value) { std::cout << value; });
    std::cout << "\n后序: ";
    root.postorder([](char value) { std::cout << value; });
    std::cout << "\n层次: ";
    root.level_order([](char value) { std::cout << value; });
    std::cout << '\n';

    dsa::BinarySearchTree<int> tree;
    for (int key : {8, 3, 10, 1, 6, 14, 4, 7}) {
        (void)tree.insert(key);
    }
    std::cout << "BST 中序:";
    tree.inorder([](int key) { std::cout << ' ' << key; });
    std::cout << "\n含 6? " << (tree.contains(6) ? " 是" : " 否")
```

```

        << " 删 3 后含 3? ";
    (void)tree.remove(3);
    std::cout << (tree.contains(3) ? " 是" : " 否") << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch05/binary_tree \
    code/ch05/binary_tree/demo.cpp -o /tmp/tree-demo
/tmp/tree-demo

```

```

先序: ABDEC
中序: DBEAC
后序: DEBCA
层次: ABCDE
BST 中序: 1 3 4 6 7 8 10 14
含 6? 是 删 3 后含 3? 否

```

`create_tree(value, left, right)` 接管两棵子树。参数是右值，表示所有权被移走；`left_leaf` 在 `std::move` 之后变空，不会和 `left` 抢着析构同一结点。

5.2.2 深度优先周游

先序 / 中序 / 后序的递归版就是三行：访问自己与走进左右孩子的次序不同。迭代版用一棵手写链式栈模拟调用栈，按 D-001 §3d 只作补充，不替换递归主实现。

5.2.3 广度优先周游

层次周游用手写链式 FIFO，不用 `std::queue` 替代本节要教的队列用法。

5.3 二叉树的存储结构

`create_tree` 先 `new` 出新根，再把两棵子树的根指针挪过来，最后才清空自己原来的树。这样即使 `new` 抛异常，调用方的两棵子树也不动。链式存储是本节主实现；完全二叉树还可以按 5.1.3 的编号放进数组，堆就是这种用法。

```

template <typename T>
class BinaryTree {
private:
    template <typename U>
    class LinkedStack {
        struct Entry {
            U value;
            Entry* next;

```

```

};

public:
    LinkedStack() = default;
    LinkedStack(const LinkedStack&) = delete;
    LinkedStack& operator=(const LinkedStack&) = delete;
    ~LinkedStack() { clear(); }

    [[nodiscard]] bool empty() const noexcept { return top_ == nullptr; }
    void push(const U& value) { top_ = new Entry{value, top_}; }
    void push(U&& value) { top_ = new Entry{std::move(value), top_}; }
    [[nodiscard]] std::optional<U> pop() {
        if (top_ == nullptr) return std::nullopt;
        Entry* entry = top_;
        top_ = entry->next;
        U value = std::move(entry->value);
        delete entry;
        return value;
    }

private:
    void clear() noexcept {
        while (top_ != nullptr) {
            Entry* entry = top_;
            top_ = entry->next;
            delete entry;
        }
    }
    Entry* top_{nullptr};
};

public:
    struct Node {
        T value;
        Node* left{nullptr};
        Node* right{nullptr};

        template <typename U>
        explicit Node(U&& item) : value(std::forward<U>(item)) {}
    };

    BinaryTree() noexcept = default;
    BinaryTree(const BinaryTree& other) : root_(clone(other.root_)) {}
    BinaryTree& operator=(const BinaryTree& other) {
        if (this != &other) {
            BinaryTree copy(other);
            swap(copy);
        }
    }

```

```

    }
    return *this;
}
BinaryTree(BinaryTree&& other) noexcept : root_(other.release_root()) {}
BinaryTree& operator=(BinaryTree&& other) noexcept {
    if (this != &other) {
        make_empty();
        root_ = other.release_root();
    }
    return *this;
}
~BinaryTree() { make_empty(); }

void swap(BinaryTree& other) noexcept {
    using std::swap;
    swap(root_, other.root_);
}

[[nodiscard]] bool empty() const noexcept { return root_ == nullptr; }
[[nodiscard]] const Node* root() const noexcept { return root_; }
[[nodiscard]] Node* root() noexcept { return root_; }

/// 构造一棵新树并接管两个子树，强异常保证：根结点创建失败时两棵子树不动。
template <typename U>
void create_tree(U&& value, BinaryTree&& left = {}, BinaryTree&& right = {}) {
    Node* fresh = new Node(std::forward<U>(value));
    fresh->left = left.release_root();
    fresh->right = right.release_root();
    make_empty();
    root_ = fresh;
}

/// 后序释放整个树。递归删除与递归周游同样受树高限制。
void make_empty() noexcept {
    destroy(root_);
    root_ = nullptr;
}

// Stack Overflow Risk: 以下递归周游忠实保留原书算法 5.3；病态深树可能耗尽调用栈。
template <typename Visitor>
void preorder(Visitor&& visit) const { preorder_impl(root_, visit); }
template <typename Visitor>
void inorder(Visitor&& visit) const { inorder_impl(root_, visit); }
template <typename Visitor>
void postorder(Visitor&& visit) const { postorder_impl(root_, visit); }

// 算法 5.4 至 5.6：作为补充保留原书的手写栈式非递归周游。

```

```

template <typename Visitor>
void preorder_iterative(Visitor&& visit) const {
    LinkedStack<const Node*> pending;
    if (root_ != nullptr) pending.push(root_);
    while (auto node = pending.pop()) {
        visit((*node)->value);
        if ((*node)->right != nullptr) pending.push((*node)->right);
        if ((*node)->left != nullptr) pending.push((*node)->left);
    }
}

template <typename Visitor>
void inorder_iterative(Visitor&& visit) const {
    LinkedStack<const Node*> pending;
    const Node* current = root_;
    while (current != nullptr || !pending.empty()) {
        while (current != nullptr) {
            pending.push(current);
            current = current->left;
        }
        current = *pending.pop();
        visit(current->value);
        current = current->right;
    }
}

template <typename Visitor>
void postorder_iterative(Visitor&& visit) const {
    struct Frame { const Node* node; bool expanded; };
    LinkedStack<Frame> pending;
    if (root_ != nullptr) pending.push(Frame{root_, false});
    while (auto frame = pending.pop()) {
        if (frame->expanded) {
            visit(frame->node->value);
        } else {
            pending.push(Frame{frame->node, true});
            if (frame->node->right != nullptr)
                ↪ pending.push(Frame{frame->node->right, false});
            if (frame->node->left != nullptr)
                ↪ pending.push(Frame{frame->node->left, false});
        }
    }
}

```

/// 算法 5.7 的层次周游。内部手写链式 *FIFO*，不以 *STL queue* 替代本章内容。

```

template <typename Visitor>
void level_order(Visitor&& visit) const {
    struct Pending { const Node* node; Pending* next; };
    Pending* front = nullptr;

```

```

Pending* back = nullptr;
auto enqueue = & {
    Pending* item = new Pending{node, nullptr};
    if (back == nullptr) front = item; else back->next = item;
    back = item;
};
try {
    if (root_ != nullptr) enqueue(root_);
    while (front != nullptr) {
        Pending* item = front;
        front = front->next;
        if (front == nullptr) back = nullptr;
        const Node* node = item->node;
        delete item;
        visit(node->value);
        if (node->left != nullptr) enqueue(node->left);
        if (node->right != nullptr) enqueue(node->right);
    }
} catch (...) {
    while (front != nullptr) { Pending* item = front; front = front->next;
        ↪ delete item; }
    throw;
}

}

[[nodiscard]] const Node* parent_of(const Node* wanted) const noexcept {
    return parent_of_impl(root_, wanted);
}

private:
static void destroy(Node* node) noexcept {
    if (node != nullptr) { destroy(node->left); destroy(node->right); delete
        ↪ node; }
}
static Node* clone(const Node* node) {
    if (node == nullptr) return nullptr;
    Node* copy = new Node(node->value);
    try { copy->left = clone(node->left); copy->right = clone(node->right); }
    catch (...) { destroy(copy); throw; }
    return copy;
}
static const Node* parent_of_impl(const Node* node, const Node* wanted) noexcept
    ↪ {
    if (node == nullptr || wanted == nullptr) return nullptr;
    if (node->left == wanted || node->right == wanted) return node;
    if (const Node* left = parent_of_impl(node->left, wanted)) return left;
    return parent_of_impl(node->right, wanted);
}

```

```

}
template <typename Visitor> static void preorder_impl(const Node* node, Visitor&
↪ visit) {
    if (node != nullptr) { visit(node->value); preorder_impl(node->left, visit);
        ↪ preorder_impl(node->right, visit); }
}
template <typename Visitor> static void inorder_impl(const Node* node, Visitor&
↪ visit) {
    if (node != nullptr) { inorder_impl(node->left, visit); visit(node->value);
        ↪ inorder_impl(node->right, visit); }
}
template <typename Visitor> static void postorder_impl(const Node* node,
↪ Visitor& visit) {
    if (node != nullptr) { postorder_impl(node->left, visit);
        ↪ postorder_impl(node->right, visit); visit(node->value); }
}
Node* release_root() noexcept { Node* result = root_; root_ = nullptr; return
↪ result; }
Node* root_{nullptr};
};

```

5.4 二叉搜索树

二叉搜索树要求左子树的键都小于根、右子树都大于根。中序周游因此正好是排序。插入重复键、删除不存在的键都是可预期状态，返回 false，不抛异常。删除有左右孩子的结点时，用左子树里最右的前驱替换它：先把前驱从原位置摘下，再让它继承被删结点的两棵子树，最后只 delete 被删结点一次。漏掉「先脱离原父」会形成环或二次释放。

```

template <typename T, typename Compare = std::less<T>>
class BinarySearchTree {
    struct Node {
        T value;
        Node* left{nullptr};
        Node* right{nullptr};
        template <typename U> explicit Node(U&& item) : value(std::forward<U>(item))
        ↪ {}
    };

public:
    BinarySearchTree() = default;
    BinarySearchTree(const BinarySearchTree& other) : root_(clone(other.root_)),
    ↪ compare_(other.compare_) {}
    BinarySearchTree& operator=(const BinarySearchTree& other) {

```

```

        if (this != &other) { BinarySearchTree copy(other); swap(copy); }
        return *this;
    }
    BinarySearchTree(BinarySearchTree&& other) : root_(other.root_),
    ↪ compare_(std::move(other.compare_)) { other.root_ = nullptr; }
    BinarySearchTree& operator=(BinarySearchTree&& other) {
        if (this != &other) { clear(); root_ = other.root_; other.root_ = nullptr;
        ↪ compare_ = std::move(other.compare_); }
        return *this;
    }
    ~BinarySearchTree() { clear(); }

    void swap(BinarySearchTree& other) {
        using std::swap; swap(root_, other.root_); swap(compare_, other.compare_);
    }
    [[nodiscard]] bool empty() const noexcept { return root_ == nullptr; }

    /// 算法 5.9: 插入唯一键。重复键是可预期状态, 返回 false。
    bool insert(const T& value) { return insert_impl(value); }
    bool insert(T&& value) { return insert_impl(std::move(value)); }

    [[nodiscard]] bool contains(const T& value) const {
        const Node* current = root_;
        while (current != nullptr) {
            if (equivalent(value, current->value)) return true;
            current = compare_(value, current->value) ? current->left :
    ↪ current->right;
        }
        return false;
    }

    /// 算法 5.10: 删除不存在的键是幂等的可预期状态, 返回 false (D-001 §3c)。
    bool remove(const T& value) { return remove_impl(root_, value); }
    void clear() noexcept { destroy(root_); root_ = nullptr; }

    template <typename Visitor>
    void inorder(Visitor&& visit) const { inorder_impl(root_, visit); }

private:
    [[nodiscard]] bool equivalent(const T& left, const T& right) const {
        return !compare_(left, right) && !compare_(right, left);
    }
    template <typename U> bool insert_impl(U&& value) {
        Node** link = &root_;
        while (*link != nullptr) {
            if (equivalent(value, (*link)->value)) return false;
            link = compare_(value, (*link)->value) ? &(*link)->left :
    ↪ &(*link)->right;
        }

```

```

    }
    *link = new Node(std::forward<U>(value));
    return true;
}

bool remove_impl(Node*& link, const T& value) {
    if (link == nullptr) return false;
    if (compare_(value, link->value)) return remove_impl(link->left, value);
    if (compare_(link->value, value)) return remove_impl(link->right, value);
    Node* removed = link;
    if (removed->left == nullptr) { link = removed->right; delete removed;
        ↪ return true; }
    Node** predecessor_link = &removed->left;
    while ((*predecessor_link)->right != nullptr) predecessor_link =
        ↪ &(*predecessor_link)->right;
    Node* replacement = *predecessor_link;
    *predecessor_link = replacement->left;
    replacement->left = removed->left;
    replacement->right = removed->right;
    link = replacement;
    delete removed;
    return true;
}

static void destroy(Node* node) noexcept { if (node != nullptr) {
    ↪ destroy(node->left); destroy(node->right); delete node; } }
static Node* clone(const Node* node) {
    if (node == nullptr) return nullptr;
    Node* copy = new Node(node->value);
    try { copy->left = clone(node->left); copy->right = clone(node->right); }
    catch (...) { destroy(copy); throw; }
    return copy;
}

template <typename Visitor> static void inorder_impl(const Node* node, Visitor&
    ↪ visit) {
    if (node != nullptr) { inorder_impl(node->left, visit); visit(node->value);
        ↪ inorder_impl(node->right, visit); }
}

Node* root_{nullptr};
Compare compare_{};
};

```

5.5 堆与优先队列

最小堆是一棵完全二叉树，父结点不大于孩子；用数组存时，下标 i 的孩子是 $2i+1$ 和 $2i+2$ 。sift_down 必须比较左右两个孩子。空堆上 remove_min() 返回 nullptr。

```

#include "modern.hpp"

#include <iostream>

int main() {
    dsa::MinHeap<int> heap;
    for (int value : {5, 1, 4, 2}) {
        heap.insert(value);
    }
    std::cout << " 依次取出最小元:";
    while (auto value = heap.remove_min()) {
        std::cout << ' ' << *value;
    }
    std::cout << '\n';

    const int weights[] = {2, 3, 4, 7};
    const dsa::HuffmanTree tree(weights, 4);
    std::cout << " 权 2,3,4,7 的 Huffman 树根权 = " << tree.total_weight() << '\n';
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch05/heap_huffman \
    code/ch05/heap_huffman/demo.cpp -o /tmp/heap-demo
/tmp/heap-demo

```

依次取出最小元: 1 2 4 5
 权 2,3,4,7 的 Huffman 树根权 = 16

```

template <typename T>
class MinHeap {
public:
    static_assert(std::is_nothrow_move_constructible<T>::value &&
        ↪ std::is_nothrow_move_assignable<T>::value,
        "MinHeap growth relies on non-throwing moves; use a
        ↪ noexcept-movable element type.");

    MinHeap() = default;
    MinHeap(const MinHeap& other) : data_(other.capacity_ ? new T[other.capacity_] :
    ↪ nullptr), size_(other.size_), capacity_(other.capacity_) {
        try { for (std::size_t i = 0; i < size_; ++i) data_[i] = other.data_[i]; }
        catch (...) { delete[] data_; throw; }
    }
    MinHeap& operator=(const MinHeap& other) { if (this != &other) { MinHeap
    ↪ copy(other); swap(copy); } return *this; }
    MinHeap(MinHeap&& other) noexcept { swap(other); }

```

```

MinHeap& operator=(MinHeap&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        capacity_ = other.capacity_;
        other.data_ = nullptr;
        other.size_ = other.capacity_ = 0;
    }
    return *this;
}

~MinHeap() { delete[] data_; }

void swap(MinHeap& other) noexcept { using std::swap; swap(data_, other.data_);
    ↪ swap(size_, other.size_); swap(capacity_, other.capacity_); }

[[nodiscard]] bool empty() const noexcept { return size_ == 0; }

[[nodiscard]] std::size_t size() const noexcept { return size_; }

void insert(const T& value) { ensure_capacity(); data_[size_] = value;
    ↪ sift_up(size_++); }

void insert(T&& value) { ensure_capacity(); data_[size_] = std::move(value);
    ↪ sift_up(size_++); }

[[nodiscard]] std::optional<T> remove_min() {
    if (empty()) return std::nullopt;
    T value = std::move(data_[0]);
    --size_;
    if (size_ == 0) return value;
    data_[0] = std::move(data_[size_]);
    sift_down(0);
    return value;
}

private:
void ensure_capacity() {
    if (size_ < capacity_) return;
    const std::size_t next = capacity_ == 0 ? 4 : capacity_ * 2;
    T* fresh = new T[next];
    // The class contract requires non-throwing move assignment, so the
    // migration loop cannot fail. Allocation failure is thrown before fresh
    ↪ exists.
    for (std::size_t i = 0; i < size_; ++i) fresh[i] = std::move(data_[i]);
    delete[] data_;
    data_ = fresh;
    capacity_ = next;
}

void sift_up(std::size_t index) {
    while (index != 0 && data_[index] < data_[(index - 1) / 2]) {
        using std::swap;
        swap(data_[index], data_[(index - 1) / 2]);
        index = (index - 1) / 2;
    }
}

```

```

    }
}
void sift_down(std::size_t index) {
    for (;;) {
        const std::size_t left = index * 2 + 1;
        const std::size_t right = left + 1;
        std::size_t smallest = index;
        if (left < size_ && data_[left] < data_[smallest]) smallest = left;
        if (right < size_ && data_[right] < data_[smallest]) smallest = right;
        if (smallest == index) return;
        using std::swap;
        swap(data_[index], data_[smallest]);
        index = smallest;
    }
}
T* data_{nullptr};
std::size_t size_{0};
std::size_t capacity_{0};
};

```

5.6 Huffman 树及其应用

Huffman 树反复取出两个最小权，合成它们的和，直到只剩一棵——这就是前缀编码的那棵树。根权等于全部叶子权之和。合并时若 new 父结点失败，会拆开并销毁已经取出的两棵子树。

```

class HuffmanTree {
    struct Node { int weight; Node* left{nullptr}; Node* right{nullptr}; explicit
        ↪ Node(int w):weight(w){} };
    struct ByWeight { Node* node{nullptr}; bool operator<(const ByWeight& other)
        ↪ const noexcept { return node->weight < other.node->weight; } };
public:
    HuffmanTree()=default;
    explicit HuffmanTree(const int* weights, std::size_t count) {
        if (count == 0) return;
        if (weights == nullptr) throw std::invalid_argument("non-empty Huffman input
            ↪ requires weights");
        MinHeap<ByWeight> heap;
        try {
            for (std::size_t i = 0; i < count; ++i) {
                if (weights[i] < 0) throw std::invalid_argument("Huffman weights
                    ↪ must be non-negative");
                Node* leaf = new Node(weights[i]);
                try { heap.insert(ByWeight{leaf}); }
            }
        }
    }
};

```

```

        catch (...) { delete leaf; throw; }
    }
    while (heap.size() > 1) {
        Node* left = heap.remove_min()->node;
        Node* right = heap.remove_min()->node;
        Node* parent = nullptr;
        try {
            if (left->weight > std::numeric_limits<int>::max() -
                right->weight) {
                throw std::overflow_error("Huffman weight sum overflows int");
            }
            parent = new Node(left->weight + right->weight);
            parent->left = left;
            parent->right = right;
            heap.insert(ByWeight{parent});
        } catch (...) {
            if (parent != nullptr) { parent->left = parent->right = nullptr;
                delete parent; }
            destroy(left);
            destroy(right);
            throw;
        }
    }
    root_ = heap.remove_min()->node;
} catch (...) {
    while (auto item = heap.remove_min()) destroy(item->node);
    throw;
}

}

HuffmanTree(const HuffmanTree&)=delete; HuffmanTree& operator=(const
HuffmanTree&)=delete;
HuffmanTree(HuffmanTree&& other)noexcept:root_(other.root_)
{other.root_=nullptr;} HuffmanTree& operator=(HuffmanTree&&
other)noexcept{if(this!=&other)
{destroy(root_);root_=other.root_;other.root_=nullptr;}return *this;}
~HuffmanTree(){destroy(root_);} [[nodiscard]] int total_weight()const
noexcept{return root_?root_->weight:0;}
private: static void destroy(Node*n)noexcept{if(n)
{destroy(n->left);destroy(n->right);delete n;}} Node* root_{nullptr};
};

```


第 6 章 树

一般树允许一个结点有任意多个孩子。本章用「左孩子、右兄弟」表示它：child 指向第一个孩子，sibling 指向下一个兄弟。并查集则回答另一类问题：若干元素分属哪些互不相交的集合。

源码：一般树和并查集、可运行示例、测试。

6.1 树的定义和基本术语

树允许一个结点有任意多个孩子。森林是若干互不相交的树。任意树可以与二叉树互相转换：把兄弟连起来、只保留长子指针，再顺时针旋转，就得到左孩子/右兄弟那棵二叉树。

6.1.1 树和森林

例如 A 的孩子是 B、C、D，B 的孩子是 E、F。画成二叉树形状之后：



任意度树只需两个指针域。代价是不能 $O(1)$ 取得「第 k 个孩子」，必须沿兄弟链走过去。

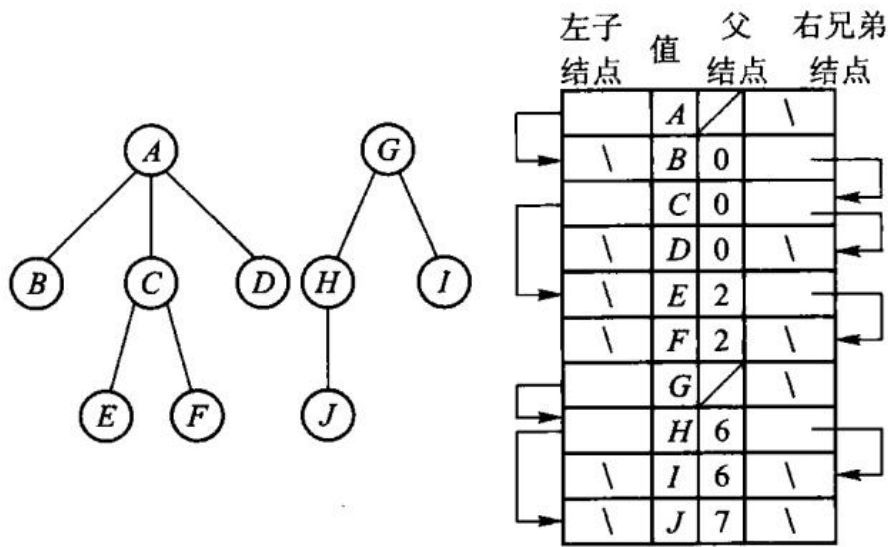


图 6.7 「左子/右兄」表示两棵树

三种周游的访问顺序不同：

周游	顺序	本例
先根	结点，再孩子子树	A B E F C D
后根	孩子子树，再结点	E F B C D A
层次	按离根的距离	A B C D E F

并查集只回答两类问题：元素属于哪个集合，以及两个元素是否同集合。`find(x)` 沿父指针走到根；路径压缩让沿途结点以后直接指向根。`unite(a, b)` 把两个根合并，并优先把较矮的树挂在较高的树下。

递归周游与销毁在极深树上有栈溢出风险。

度为 2 的有序树还不是二叉树：删掉第一个孩子后，第二个会顶上来。二叉树必须严格区分左右空位。

6.1.2 森林与二叉树的等价转换

把每棵树的兄弟从左到右用右指针串起来，只保留长子作为左指针，再顺时针旋转，森林就变成一棵二叉树。反过来：二叉树的左孩子是长子，右孩子是下一个兄弟，拆开就还原成森林。这两种转换互逆，原书没错。

6.1.3 树的抽象数据类型

树的运算是：建根、插入第一个孩子、插入下一个兄弟、问父结点、删子树、三种周游。本书把它们直接写在 `GeneralTree` 上。

6.1.4 树的周游

先根先访问结点再进孩子子树；后根相反；层次按离根距离。这三种都没错，实现见 6.2。

6.2 树的链式存储结构

6.2.1 「子结点表」表示方法

每个结点保存一块孩子指针数组。取第 k 个孩子是 $O(1)$ ，但度不固定时要扩容，插入兄弟也要搬数组。本章不把它当主实现。

6.2.2 静态「左子/右兄」表示法

用数组下标代替指针，适合结点数事先已知的场合。语义与动态版相同。

6.2.3 动态表示法

每个结点单独 new，用指针相连。长度变化大时比静态数组合适。

6.2.4 动态「左子/右兄」表示

这是本章的主实现：任意度树只需两个指针域。代价是不能 O(1) 取得「第 k 个孩子」。

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::GeneralTree<char> tree;
    tree.create_root('A');
    auto* b = tree.insert_first(tree.root(), 'B');
    auto* c = tree.insert_next(b, 'C');
    tree.insert_next(c, 'D');
    tree.insert_first(b, 'E');
    tree.insert_next(tree.root()->child->child, 'F');

    std::cout << " 先根: ";
    tree.preorder([](char value) { std::cout << value; });
    std::cout << "\n后根: ";
    tree.postorder([](char value) { std::cout << value; });
    std::cout << "\n层次: ";
    tree.breadth_first([](char value) { std::cout << value; });
    std::cout << '\n';

    dsa::DisjointSet sets(5);
    sets.unite(0, 1);
    sets.unite(1, 2);
    sets.unite(3, 4);
    std::cout << "0 与 2 同集合: " << (sets.same(0, 2) ? " 是" : " 否") << '\n';
    std::cout << "0 与 3 同集合: " << (sets.same(0, 3) ? " 是" : " 否") << '\n';
}
```

在仓库根目录运行：

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch06/general_tree \
    code/ch06/general_tree/demo.cpp -o /tmp/tree-demo
/tmp/tree-demo
```

输出是：

先根: ABEFCD
后根: EFBCDA
层次: ABCDEF
0 与 2 同集合: 是
0 与 3 同集合: 否

`insert_first(parent, value)` 把新结点插到孩子链的最前面; `insert_next(node, value)` 插到该结点的下一个兄弟。先插入 B, 再 `insert_next(B, C)`、`insert_next(C, D)`, 孩子从左到右就是 B、C、D。

`create_root` 清空旧树并新建根。`insert_first` 先让新结点的 `sibling` 指向原来的第一个孩子, 再把它接到 `parent->child` 上——所以后插入的孩子会出现在兄弟链前端。`delete_subtree` 沿着父的孩子链或森林的根兄弟链找到指向它的指针, 改写成下一个兄弟, 再递归销毁。

孩子-兄弟树:

```
template <typename T>
class GeneralTree {
public:
    struct Node {
        T value;
        Node* child{nullptr};
        Node* sibling{nullptr};
        Node* parent{nullptr};

        explicit Node(const T& value) : value(value) {}
    };

    GeneralTree() = default;

    GeneralTree(const GeneralTree& other) : root_(clone(other.root_, nullptr)) {}

    GeneralTree& operator=(const GeneralTree& other) {
        if (this != &other) {
            GeneralTree copy(other);
            swap(copy);
        }
        return *this;
    }

    GeneralTree(GeneralTree&& other) noexcept : root_(other.release()) {}

    GeneralTree& operator=(GeneralTree&& other) noexcept {
        if (this != &other) {
            clear();

```

```

        root_ = other.release();
    }
    return *this;
}

~GeneralTree() { clear(); }

void swap(GeneralTree& other) noexcept {
    using std::swap;
    swap(root_, other.root_);
}

[[nodiscard]] Node* root() noexcept { return root_; }
[[nodiscard]] const Node* root() const noexcept { return root_; }

void create_root(const T& value) {
    clear();
    root_ = new Node(value);
}

Node* insert_first(Node* parent, const T& value) {
    if (parent == nullptr) {
        throw std::invalid_argument("parent must not be null");
    }
    Node* node = new Node(value);
    node->sibling = parent->child;
    node->parent = parent;
    parent->child = node;
    return node;
}

Node* insert_next(Node* node, const T& value) {
    if (node == nullptr) {
        throw std::invalid_argument("node must not be null");
    }
    Node* next = new Node(value);
    next->sibling = node->sibling;
    next->parent = node->parent;
    node->sibling = next;
    return next;
}

[[nodiscard]] Node* parent_of(Node* node) const noexcept {
    return node == nullptr ? nullptr : node->parent;
}

void delete_subtree(Node* node) {

```

```

    if (node == nullptr) {
        return;
    }

    Node** link = node->parent == nullptr ? &root_ : &node->parent->child;
    while (*link != nullptr && *link != node) {
        link = &(*link)->sibling;
    }
    if (*link != node) {
        throw std::invalid_argument("node is not part of this tree");
    }

    *link = node->sibling;
    node->sibling = nullptr;
    destroy(node);
}

void clear() noexcept {
    destroy(root_);
    root_ = nullptr;
}

template <class Visitor>
void preorder(Visitor&& visitor) const {
    pre(root_, visitor);
}

template <class Visitor>
void postorder(Visitor&& visitor) const {
    post(root_, visitor);
}

template <class Visitor>
void breadth_first(Visitor&& visitor) const {
    std::vector<Node*> queue;
    for (Node* node = root_; node != nullptr; node = node->sibling) {
        queue.push_back(node);
    }
    for (std::size_t index = 0; index < queue.size(); ++index) {
        visitor(queue[index]->value);
        for (Node* child = queue[index]->child; child != nullptr;
             child = child->sibling) {
            queue.push_back(child);
        }
    }
}

```

```

private:
    // Recursive destruction and traversals preserve the textbook presentation.
    // They have a Stack Overflow Risk for a pathologically deep tree.
    static void destroy(Node* node) noexcept {
        if (node == nullptr) {
            return;
        }
        destroy(node->child);
        destroy(node->sibling);
        delete node;
    }

    static Node* clone(const Node* node, Node* parent) {
        if (node == nullptr) {
            return nullptr;
        }
        Node* copy = new Node(node->value);
        copy->parent = parent;
        try {
            copy->child = clone(node->child, copy);
            copy->sibling = clone(node->sibling, parent);
        } catch (...) {
            destroy(copy);
            throw;
        }
        return copy;
    }

    template <class Visitor>
    static void pre(Node* node, Visitor& visitor) {
        for (; node != nullptr; node = node->sibling) {
            visitor(node->value);
            pre(node->child, visitor);
        }
    }

    template <class Visitor>
    static void post(Node* node, Visitor& visitor) {
        for (; node != nullptr; node = node->sibling) {
            post(node->child, visitor);
            visitor(node->value);
        }
    }

    Node* release() noexcept {
        Node* result = root_;
        root_ = nullptr;
    }

```

```

        return result;
    }

    Node* root_{nullptr};
};

```

6.2.5 父指针表示法和并查集

`find` 在返回根之前把沿途结点的父指针改成根，这就是路径压缩。`unite` 比较两棵树的秩，把较矮的挂到较高的下面；两棵一样高时，新根的秩加 1。

```

class DisjointSet {
public:
    explicit DisjointSet(std::size_t count) : parent_(count), rank_(count, 0) {
        for (std::size_t index = 0; index < count; ++index) {
            parent_[index] = index;
        }
    }

    std::size_t find(std::size_t index) {
        if (index >= parent_.size()) {
            throw std::out_of_range("disjoint-set index");
        }
        if (parent_[index] != index) {
            parent_[index] = find(parent_[index]);
        }
        return parent_[index];
    }

    bool unite(std::size_t left, std::size_t right) {
        left = find(left);
        right = find(right);
        if (left == right) {
            return false;
        }
        if (rank_[left] < rank_[right]) {
            std::swap(left, right);
        }
        parent_[right] = left;
        if (rank_[left] == rank_[right]) {
            ++rank_[left];
        }
        return true;
    }
}

```

```

[[nodiscard]] bool same(std::size_t left, std::size_t right) {
    return find(left) == find(right);
}

private:
    std::vector<std::size_t> parent_;
    std::vector<std::size_t> rank_;
};

```

6.3 树的顺序存储结构

当结点按某种周游次序排进数组时，还要额外记下「下一个兄弟在哪」或「有几个孩子」，才能还原树形。原书给了四种，思路没错，本章不另写一套未验证实现。

6.3.1 带右链的先根次序表示

按先根次序存放结点，每个结点另存「下一个兄弟」的下标。

6.3.2 带双标记的先根次序表示

两个布尔标记分别表示「有没有孩子」「有没有下一个兄弟」，比显式右链更省空间。

6.3.3 带度数的后根次序表示

按后根次序存放，每个结点记下度数。扫描时用栈按度数弹出孩子、合成子树。

6.3.4 带双标记的层次次序表示

按层存放，标记含义与 6.3.2 类似，只是次序改成层次周游。

6.4 K 叉树

每个结点至多 K 个孩子。满 K 叉树与完全 K 叉树的定义与二叉树平行：用编号 $Ki+1, \dots, Ki+K$ 找孩子。K=2 就是二叉树。本章主实现仍是任意度的左子/右兄，不单独做一份 K 叉数组。

第 7 章 图

图由顶点和边组成。有向边表示单向关系，无向边表示双向关系，带权边表示代价。先弄清题目属于哪一种，再选算法。

源码：图与七个算法、可运行示例、交叉验证测试。

7.1 图的定义和基本术语

题目	前提	结果
「从一个点能走到哪些点」	任意图	DFS 或 BFS 的访问序列
「课程/任务的可行顺序」	有向无环图	拓扑序；有环则无解
「一个源点到各点最短路」	边权非负	Dijkstra 距离数组
「任意两点最短路」	顶点数较小	Floyd 距离矩阵
「连通所有点且总权最小」	无向连通图	Prim 或 Kruskal 的边集

最短路的「最短」是边权总和最小，不一定是经过边数最少。最小生成树的目标也不是任意两点最短，而是在保证所有顶点连通的前提下让选中的全部边总权最小。

本单元用邻接矩阵。没有边记为 `infinity` (`int` 最大值的四分之一，给加法留余量)。环、非连通图等正常失败状态用 `optional` 表达。DFS 仍是递归，深图有栈溢出风险。

7.2 图的抽象数据类型

图的运算是加边、问顶点数、以及下面各节的周游与最优化。原书【代码 7.1】【代码 7.2】的 ADT 声明残缺且标识符被空格切断；本书直接定义在 `Graph` 上。

7.3 图的存储结构

7.3.1 相邻矩阵

邻接矩阵的第 `from` 行、第 `to` 列是边权。构造时对角线为 0，其余为 `infinity`。`add_edge(..., directed=false)` 同时写入对称位置。零权边可以表示——原书把 0 同时当成「无边」是错的。

7.4 图的周游

7.4.1 先跑一遍

下面这张有向图：0→1(2)、0→2(7)、1→2(1)、1→3(5)、2→3(1)、3→4(3)。从0到4的最短路是0→1→2→3→4，总权7，不是那条权为7的直达边0→2再往后走。

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::Graph graph(5);
    graph.add_edge(0, 1, 2);
    graph.add_edge(0, 2, 7);
    graph.add_edge(1, 2, 1);
    graph.add_edge(1, 3, 5);
    graph.add_edge(2, 3, 1);
    graph.add_edge(3, 4, 3);

    std::cout << "DFS(0):";
    for (std::size_t vertex : graph.dfs(0)) {
        std::cout << ' ' << vertex;
    }
    std::cout << "\nBFS(0):";
    for (std::size_t vertex : graph.bfs(0)) {
        std::cout << ' ' << vertex;
    }

    const auto topo = graph.topological_sort();
    std::cout << "\n拓扑序:";
    if (topo) {
        for (std::size_t vertex : *topo) {
            std::cout << ' ' << vertex;
        }
    } else {
        std::cout << " 无 (有环) ";
    }

    const auto distance = graph.dijkstra(0);
    std::cout << "\nDijkstra(0→4) = " << distance[4] << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch07/graph \
code/ch07/graph/demo.cpp -o /tmp/graph-demo
```

```
/tmp/graph-demo
```

```
DFS(0): 0 1 2 3 4  
BFS(0): 0 1 2 3 4  
拓扑序: 0 1 2 3 4  
Dijkstra(0→4) = 7
```

若再加上 4→0 形成环，`topological_sort()` 返回 `nullopt`。

7.4.2 深度优先与广度优先

DFS 进入一个顶点就立刻标记已访问，再沿编号从小到大的出边递归。BFS 用队列按层扩展，先到的顶点先出队。

```
[[nodiscard]] std::vector<std::size_t> dfs(std::size_t source) const {  
    check_vertex(source);  
    std::vector<bool> seen(vertices());  
    std::vector<std::size_t> result;  
    visit_depth_first(source, seen, result);  
    return result;  
}
```

```
[[nodiscard]] std::vector<std::size_t> bfs(std::size_t source) const {  
    check_vertex(source);  
    std::vector<bool> seen(vertices());  
    std::queue<std::size_t> queue;  
    std::vector<std::size_t> result;  
    queue.push(source);  
    seen[source] = true;  
    while (!queue.empty()) {  
        const std::size_t from = queue.front();  
        queue.pop();  
        result.push_back(from);  
        for (std::size_t to = 0; to < vertices(); ++to) {  
            if (adjacency_[from][to] < infinity && !seen[to]) {  
                seen[to] = true;  
                queue.push(to);  
            }  
        }  
    }  
    return result;  
}
```

7.4.3 拓扑排序

统计每个顶点的入度，把入度为 0 的点入队；每取出一个点，就把它指出的入度减 1。若最终取出的点数少于顶点数，图里有环。

```
[[nodiscard]] std::optional<std::vector<std::size_t>> topological_sort() const {
    std::vector<std::size_t> indegree(vertices());
    for (std::size_t from = 0; from < vertices(); ++from) {
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (from != to && adjacency_[from][to] < infinity) {
                ++indegree[to];
            }
        }
    }
    std::queue<std::size_t> queue;
    for (std::size_t vertex = 0; vertex < vertices(); ++vertex) {
        if (indegree[vertex] == 0) {
            queue.push(vertex);
        }
    }
    std::vector<std::size_t> result;
    while (!queue.empty()) {
        const std::size_t from = queue.front();
        queue.pop();
        result.push_back(from);
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (from != to && adjacency_[from][to] < infinity && --indegree[to] ==
                ↪ 0) {
                queue.push(to);
            }
        }
    }
    return result.size() == vertices() ?
        ↪ std::optional<std::vector<std::size_t>>(result)
        : std::nullopt;
}
```

7.5 最短路径

7.5.1 单源最短路径

Dijkstra 反复选出尚未确定的、当前距离最小的顶点，用它的出边松弛邻居。边权必须非负。

```

[[nodiscard]] std::vector<int> dijkstra(std::size_t source) const {
    check_vertex(source);
    std::vector<int> distance(vertices(), infinity);
    std::vector<bool> used(vertices());
    distance[source] = 0;
    for (std::size_t count = 0; count < vertices(); ++count) {
        const std::size_t from = nearest_unvisited(distance, used);
        if (from == vertices() || distance[from] == infinity) {
            break;
        }
        used[from] = true;
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity &&
                distance[to] > distance[from] + adjacency_[from][to]) {
                distance[to] = distance[from] + adjacency_[from][to];
            }
        }
    }
    return distance;
}

```

7.5.2 每对顶点之间的最短路径

Floyd 枚举中转点 via, 比较 from→to 与 from→via→to。测试里五个源点的 Dijkstra 与 Floyd 逐项对拍。

```

[[nodiscard]] std::vector<std::vector<int>> floyd() const {
    auto distance = adjacency_;
    for (std::size_t via = 0; via < vertices(); ++via) {
        for (std::size_t from = 0; from < vertices(); ++from) {
            for (std::size_t to = 0; to < vertices(); ++to) {
                if (distance[from][via] < infinity && distance[via][to] < infinity) {
                    distance[from][to] = std::min(
                        distance[from][to], distance[from][via] + distance[via][to]);
                }
            }
        }
    }
    return distance;
}

```

7.6 最小生成树

目标不是任意两点最短，而是连通全部顶点且选中边的总权最小。

7.6.1 Prim 算法

从指定源点生长，每次把离当前树最近的顶点加进来。非连通则返回 `nullopt`。

```
[[nodiscard]] std::optional<std::vector<Edge>> prim(std::size_t source) const {
    check_vertex(source);
    std::vector<int> distance(vertices(), infinity);
    std::vector<std::size_t> predecessor(vertices());
    std::vector<bool> used(vertices());
    std::vector<Edge> result;
    distance[source] = 0;
    for (std::size_t count = 0; count < vertices(); ++count) {
        const std::size_t from = nearest_unvisited(distance, used);
        if (from == vertices() || distance[from] == infinity) {
            return std::nullopt;
        }
        used[from] = true;
        if (from != source) {
            result.push_back({predecessor[from], from, distance[from]});
        }
        for (std::size_t to = 0; to < vertices(); ++to) {
            if (!used[to] && adjacency_[from][to] < distance[to]) {
                distance[to] = adjacency_[from][to];
                predecessor[to] = from;
            }
        }
    }
    return result;
}
```

7.6.2 Kruskal 算法

把边按权排序，用并查集跳过会形成环的边。连通图上与 Prim 得到相同总权、 $n-1$ 条边。

```
[[nodiscard]] std::optional<std::vector<Edge>> kruskal() const {
    std::vector<Edge> edges;
    for (std::size_t from = 0; from < vertices(); ++from) {
        for (std::size_t to = from + 1; to < vertices(); ++to) {
            if (adjacency_[from][to] < infinity) {
                edges.push_back({from, to, adjacency_[from][to]});
            }
        }
    }
    return edges;
}
```

```

    }
}

std::sort(edges.begin(), edges.end());
std::vector<std::size_t> parent(vertices());
for (std::size_t vertex = 0; vertex < vertices(); ++vertex) {
    parent[vertex] = vertex;
}
std::vector<Edge> result;
for (const Edge& edge : edges) {
    const std::size_t from_root = find_root(parent, edge.from);
    const std::size_t to_root = find_root(parent, edge.to);
    if (from_root != to_root) {
        parent[from_root] = to_root;
        result.push_back(edge);
    }
}
return result.size() + 1 == vertices() ?
    ⇨ std::optional<std::vector<Edge>>(result)
      : std::nullopt;
}

```


第 8 章 内部排序

排序把一个序列重排为非递减顺序。阅读时同时问四个问题：比较还是分配？是否稳定？额外空间多少？最好、平均、最坏时间是多少？

源码：全部手写排序、可运行示例、对拍测试。

8.1 排序问题的基本概念

方法	平均时间	稳定性	主要特点
直接插入	$O(n^2)$	稳定	小规模、近乎有序时简单有效
冒泡	$O(n^2)$	稳定	教学直观，通常不作为通用选择
选择	$O(n^2)$	不稳定	交换次数少
堆排序	$O(n \log n)$	不稳定	最坏情况也有保证，原地排序
快速排序	$O(n \log n)$ 平均	不稳定	实务常用，坏划分时会退化
归并排序	$O(n \log n)$	稳定	需要 $O(n)$ 辅助空间
计数/基数	与值域或位数有关	可稳定	适合整数键，不是比较排序

「稳定」指相等键在排序前后的相对顺序不变。例如记录按成绩排序时，两名同学仍按原来的姓名顺序出现。本章所有实现接受有符号整数，测试含重复和负数。没有用 `std::sort` / `std::make_heap` 替代手写算法。

8.2 插入排序

8.2.1 先跑一遍

```
#include "modern.hpp"

#include <iostream>
#include <vector>

namespace {
void print(const char* name, const std::vector<int>& values) {
    std::cout << name;
    for (int value : values) {
        std::cout << ' ' << value;
    }
    std::cout << '\n';
}
```

```

}
} // namespace

int main() {
    const std::vector<int> raw{3, -2, 7, 3, 0, -2, 9, 1};

    auto insertion = raw;
    dsa::sorting::insertion_sort(insertion);
    print(" 插入:", insertion);

    auto heap = raw;
    dsa::sorting::heap_sort(heap);
    print(" 堆排:", heap);

    auto quick = raw;
    dsa::sorting::quick_sort(quick);
    print(" 快排:", quick);

    auto radix = raw;
    dsa::sorting::radix_sort(radix);
    print(" 基数:", radix);
}

```

```

c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch08/sorting \
    code/ch08/sorting/demo.cpp -o /tmp/sort-demo
/tmp/sort-demo

```

```

插入: -2 -2 0 1 3 3 7 9
堆排: -2 -2 0 1 3 3 7 9
快排: -2 -2 0 1 3 3 7 9
基数: -2 -2 0 1 3 3 7 9

```

四种算法交出同一条非递减序列。插入排序保持两个 -2、两个 3 的相对次序；堆排和快排不保证这一点。

直接插入把 `values[index]` 抽出来，向前挪动所有比它大的元素，把空位留给它。相等元素不越过彼此，所以稳定。

```

// 算法 8.1: 直接插入排序。相等元素不越过彼此，故稳定。
inline void insertion_sort(std::vector<int>& values) {
    for (std::size_t index = 1; index < values.size(); ++index) {
        const int value = values[index];
        std::size_t hole = index;
        while (hole != 0 && value < values[hole - 1]) {
            values[hole] = values[hole - 1];

```

```

        --hole;
    }
    values[hole] = value;
}
}

```

8.2.2 Shell 排序

增量每次减半的插入排序。不稳定，平均优于直接插入。实现见同一源文件中的 `shell_sort`。

8.3 选择排序

8.3.2 堆排序

先把数组建成最大堆，再反复把堆顶与堆尾交换并缩小堆。`sift_down` 必须比较左右两个孩子。

```

// 算法 8.4: 手写最大堆筛选与堆排序, 不委托 std::make_heap/sort_heap.
inline void sift_down(std::vector<int>& values, std::size_t root, std::size_t
↳ count) {
    while (root * 2 + 1 < count) {
        std::size_t child = root * 2 + 1;
        if (child + 1 < count && values[child] < values[child + 1]) ++child;
        if (values[root] >= values[child]) return;
        using std::swap;
        swap(values[root], values[child]);
        root = child;
    }
}

inline void heap_sort(std::vector<int>& values) {
    for (std::size_t root = values.size() / 2; root != 0; --root) {
        sift_down(values, root - 1, values.size());
    }
    for (std::size_t end = values.size(); end > 1; --end) {
        using std::swap;
        swap(values[0], values[end - 1]);
        sift_down(values, 0, end - 1);
    }
}

```

8.4 交换排序

8.4.2 快速排序

选区间末元素为枢轴，把更小的元素换到左侧，再递归两边。全相等的输入必须也能结束。

```
inline std::size_t partition(std::vector<int>& values, std::size_t first,
    ↪ std::size_t last) {
    const int pivot = values[last - 1];
    std::size_t boundary = first;
    for (std::size_t index = first; index + 1 < last; ++index) {
        if (values[index] < pivot) {
            using std::swap;
            swap(values[boundary], values[index]);
            ++boundary;
        }
    }
    using std::swap;
    swap(values[boundary], values[last - 1]);
    return boundary;
}

inline void quick_sort_range(std::vector<int>& values, std::size_t first,
    ↪ std::size_t last) {
    if (last - first < 2) return;
    const std::size_t middle = partition(values, first, last);
    quick_sort_range(values, first, middle);
    quick_sort_range(values, middle + 1, last);
}

// 算法 8.6: 手写快排。
inline void quick_sort(std::vector<int>& values) { quick_sort_range(values, 0,
    ↪ values.size()); }
```

8.5 归并排序

稳定，需要 $O(n)$ 辅助空间。实现见 merge_sort。

8.6 分配排序和索引排序

8.6.2 基数排序

把有符号 `int` 的符号位翻转后，按字节做 4 趟计数收集。否则负数会按无符号序排到最大。

```
// 算法 8.11: LSD 基数排序。翻转符号位使二补码有符号 int 按无符号序排序。
inline void radix_sort(std::vector<int>& values) {
    std::vector<int> buffer(values.size());
    for (unsigned shift = 0; shift < 32; shift += 8) {
        std::size_t counts[256]{};
        for (int value : values) {
            const auto key = static_cast<std::uint32_t>(value) ^ 0x80000000U;
            ++counts[(key >> shift) & 0xffU];
        }
        std::size_t offset = 0;
        for (std::size_t& count : counts) { const std::size_t old = count; count =
            ↪ offset; offset += old; }
        for (int value : values) {
            const auto key = static_cast<std::uint32_t>(value) ^ 0x80000000U;
            buffer[counts[(key >> shift) & 0xffU]++] = value;
        }
        values.swap(buffer);
    }
}
```

8.7 排序算法的时间代价

比较排序在最坏情况下至少 $\Omega(n \log n)$ 次比较。插入、选择、冒泡是 $O(n^2)$ ；堆、归并最坏也是 $O(n \log n)$ ；快排平均 $O(n \log n)$ 、最坏 $O(n^2)$ 。计数和基数不是比较排序，代价取决于值域或位数。

第 9 章 文件管理与外部排序

当数据大到内存装不下时，排序的瓶颈变成磁盘读写。外部排序先生成有序顺串，再进行多路归并。置换选择用最小堆尽可能延长当前顺串；竞赛树维护多路输入中当前最小的选手。

源码：置换选择与竞赛树、可运行示例、测试。

9.1 主存储器和外存储器

内存按字节随机访问，外存按页块读写，一次定位往往比一次比较贵几个数量级。所以外排序的目标首先是减少读写次数，而不是减少内存里的比较。

9.2 文件的组织和管理

文件是外存上的记录集合。本章不实现页缓存和文件句柄，只抽出「如何生成更长的初始顺串」和「如何在 k 路中选最小」。

9.3 外排序

9.3.1 置换选择排序

顺串是磁盘上已经有序的一段记录。内存一次只能装 M 条时，朴素做法是每批排成一条长为 M 的顺串。置换选择可以做得更好：输出堆顶之后，若下一条记录不小于刚输出的值，它还可以进入当前顺串；否则冻结到下一趟。平均情况下第一趟长度约为 2M。

原书图 9.2 的输入是 50 49 35 45 30 25 15 60 16 27 1，工作区 M = 7。前 7 个建成最小堆后，堆顶是 15。接着读到 60——它比 15 大，可以进当前堆；再读到 16、27、1，它们都比当时的输出值小，被冻结。第一顺串因此是

15 25 30 35 45 49 50 60

长度 8，已经超过 M。剩下的 1 16 27 构成第二顺串。

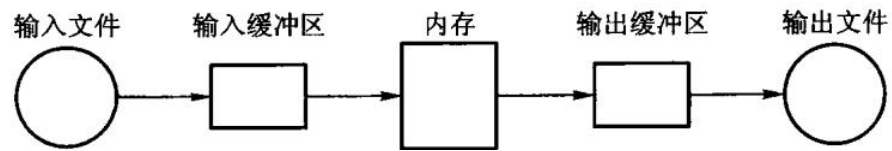


图 9.1 置换选择算法流程

若把整份输入推进一个堆再依次弹出，得到的是一条完全有序序列——那是堆排序，不是置换选择。旧实现曾经这样做，测试只断言 $\{3,1,2\} \rightarrow \{1,2,3\}$ ，堆排序也能过。现在的接口返回若干顺串，并用原书这组数据守门。

多路归并时，每次要在 k 路的队首里选出最小者。赢者树的内部结点保存胜者下标；败者树的内部结点保存败者，另用一个冠军槽记录全局最小。替换一名选手后，两者都只需沿叶到根重赛。

先跑一遍：

```
#include "modern.hpp"

#include <iostream>

int main() {
    const std::vector<int> input{50, 49, 35, 45, 30, 25, 15, 60, 16, 27, 1};
    const auto runs = dsa::external_sort::replacement_selection(input, 7);

    std::cout << " 工作区 M=7, 得到 " << runs.size() << " 个顺串\n";
    for (std::size_t index = 0; index < runs.size(); ++index) {
        std::cout << " 顺串 " << index + 1 << " (长度 " << runs[index].size() << ") :";
        for (int value : runs[index]) {
            std::cout << ' ' << value;
        }
        std::cout << '\n';
    }

    dsa::external_sort::LoserTree tree({20, 6, 8, 9, 11});
    std::cout << " 败者树当前冠军: " << *tree.winner() << '\n';
    tree.replace(1, 15);
    std::cout << " 替换后冠军: " << *tree.winner() << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch09/external_sort \
    code/ch09/external_sort/demo.cpp -o /tmp/extsort-demo
/tmp/extsort-demo
```

```
工作区 M=7, 得到 2 个顺串
顺串 1 (长度 8): 15 25 30 35 45 49 50 60
顺串 2 (长度 3): 1 16 27
败者树当前冠军: 6
替换后冠军: 8
```

把 M 改成 11（装得下全部输入），会只得到一个顺串，内容是整表有序——这时置换选择退化为堆排序，正是「内存够用」的边界。

replacement_selection(input, memory) 先读入至多 memory 条建成最小堆。循环中弹出堆顶写入当前顺串；若还有输入，按「incoming >= emitted 则入堆，否则冻结」分流。当前堆空了，就把冻结区建成新堆，开始下一趟。

```
// 算法 9.1: 置换选择。memory 是内存工作区能容纳的记录数 M。
// 返回若干顺串：每个顺串内部有序，第一趟的平均长度约为 2M，而不是 M。
// 不属于当前顺串的新记录被冻结，等当前堆耗尽后再建下一趟。
inline std::vector<std::vector<int>> replacement_selection(const std::vector<int>&
↳ input,
                                                    std::size_t memory) {
    if (memory == 0) {
        throw std::invalid_argument("replacement selection memory must be positive");
    }
    if (input.empty()) {
        return {};
    }

    std::size_t next = 0;
    std::vector<int> heap;
    heap.reserve(memory);
    while (next < input.size() && heap.size() < memory) {
        heap.push_back(input[next++]);
    }
    detail::heap_from(heap);

    std::vector<std::vector<int>> runs;
    std::vector<int> current_run;
    std::vector<int> frozen;

    while (!heap.empty() || next < input.size() || !frozen.empty()) {
        if (heap.empty()) {
            if (!current_run.empty()) {
                runs.push_back(std::move(current_run));
                current_run = {};
            }
            heap = std::move(frozen);
            frozen = {};
            while (next < input.size() && heap.size() < memory) {
                heap.push_back(input[next++]);
            }
            detail::heap_from(heap);
            if (heap.empty()) {
                break;
            }
        }

        const int emitted = detail::heap_pop(heap);
```

```

        current_run.push_back(emitted);

        if (next == input.size()) {
            continue;
        }
        const int incoming = input[next++];
        if (incoming >= emitted) {
            detail::heap_push(heap, incoming);
        } else {
            frozen.push_back(incoming);
        }
    }
    if (!current_run.empty()) {
        runs.push_back(std::move(current_run));
    }
    return runs;
}

```

9.3.2 二路外排序

对 m 个顺串两两归并，趟数是 $\lceil \log_2 m \rceil$ 。置换选择把初始顺串变长，就是为了减小 m 。

9.3.3 多路归并——选择树

赢者树用完全二叉数组：叶存放选手下标，内部结点写 better(左, 右)。败者树在内部结点写败者，另用 `champion_` 记全局胜者。替换后只沿叶到根重赛。

```

// 代码 9.2: 赢者树。内部结点保存两名选手比较后的胜者下标，根是全局最小。
class WinnerTree {
public:
    explicit WinnerTree(std::vector<int> players) : players_(std::move(players)) {
        if (players_.empty()) {
            return;
        }
        leaf_base_ = detail::TournamentOps::next_power_of_two(players_.size());
        tree_.assign(leaf_base_ * 2, detail::TournamentOps::no_player);
        for (std::size_t index = 0; index < players_.size(); ++index) {
            tree_[leaf_base_ + index] = index;
        }
        for (std::size_t node = leaf_base_ - 1; node > 0; --node) {
            tree_[node] = detail::TournamentOps::better(players_, tree_[node * 2],
                                                         tree_[node * 2 + 1]);
        }
    }
}

```

```

[[nodiscard]] std::optional<std::size_t> winner_index() const {
    if (players_.empty() || tree_[1] == detail::TournamentOps::no_player) {
        return std::nullopt;
    }
    return tree_[1];
}

[[nodiscard]] std::optional<int> winner() const {
    const auto index = winner_index();
    return index ? std::optional<int>(players_[*index]) : std::nullopt;
}

void replace(std::size_t player, int value) {
    if (player >= players_.size()) {
        throw std::out_of_range("tournament player");
    }
    players_[player] = value;
    std::size_t node = leaf_base_ + player;
    tree_[node] = player;
    while (node > 1) {
        node /= 2;
        tree_[node] = detail::TournamentOps::better(players_, tree_[node * 2],
                                                    tree_[node * 2 + 1]);
    }
}

private:
    std::vector<int> players_;
    std::vector<std::size_t> tree_;
    std::size_t leaf_base_{0};
};

```

// 代码 9.3: 败者树。内部结点保存败者下标, 另用 *champion_* 记录全局胜者。

// 替换一名选手时只需沿叶到根重赛, 不必访问兄弟子树的内部结构。

```

class LoserTree {
public:
    explicit LoserTree(std::vector<int> players) : players_(std::move(players)) {
        if (players_.empty()) {
            return;
        }
        leaf_base_ = detail::TournamentOps::next_power_of_two(players_.size());
        loser_.assign(leaf_base_, detail::TournamentOps::no_player);
        subtree_winner_.assign(leaf_base_ * 2, detail::TournamentOps::no_player);
        for (std::size_t index = 0; index < players_.size(); ++index) {
            subtree_winner_[leaf_base_ + index] = index;
        }
    }
};

```

```

    }
    for (std::size_t node = leaf_base_ - 1; node > 0; --node) {
        replay_node(node);
    }
    champion_ = subtree_winner_[1];
}

[[nodiscard]] std::optional<std::size_t> winner_index() const {
    if (players_.empty() || champion_ == detail::TournamentOps::no_player) {
        return std::nullopt;
    }
    return champion_;
}

[[nodiscard]] std::optional<int> winner() const {
    const auto index = winner_index();
    return index ? std::optional<int>(players_[*index]) : std::nullopt;
}

[[nodiscard]] std::optional<std::size_t> loser_at(std::size_t node) const {
    if (node == 0 || node >= loser_.size() ||
        loser_[node] == detail::TournamentOps::no_player) {
        return std::nullopt;
    }
    return loser_[node];
}

void replace(std::size_t player, int value) {
    if (player >= players_.size()) {
        throw std::out_of_range("tournament player");
    }
    players_[player] = value;
    subtree_winner_[leaf_base_ + player] = player;
    for (std::size_t node = (leaf_base_ + player) / 2; node > 0; node /= 2) {
        replay_node(node);
    }
    champion_ = subtree_winner_[1];
}

private:
void replay_node(std::size_t node) {
    const std::size_t left = subtree_winner_[node * 2];
    const std::size_t right = subtree_winner_[node * 2 + 1];
    loser_[node] = detail::TournamentOps::worse(players_, left, right);
    subtree_winner_[node] = detail::TournamentOps::better(players_, left,
        ↪ right);
}

```

```
std::vector<int> players_;  
std::vector<std::size_t> loser_;  
std::vector<std::size_t> subtree_winner_;  
std::size_t leaf_base_{0};  
std::size_t champion_{detail::TournamentOps::no_player};  
};
```


第 10 章 检索

检索的目标是在一组记录中找到目标键。顺序检索不要求有序；二分检索要求有序，靠每次排除一半区间加速；散列表用散列函数直接定位槽位，冲突时继续探测。

源码：检索、集合和散列表、可运行示例、墓碑探测测试。

检索是在一组记录里定位关键码等于给定值的那一条，或属性满足条件的那些条。成功就是至少找到一条，失败就是没有。精确匹配查单个值，范围查询查一个区间。

平均检索长度 $ASL = \sum P_i C_i$ ，其中 P_i 是查到第 i 个元素的概率， C_i 是比较次数。衡量算法还要看额外空间和实现复杂度。

可以把检索分成四类：基于线性表、按关键码直接访问（含散列）、树形索引、基于属性（倒排）。本章做前两类；树形索引见第 5、11、12 章，倒排见第 11 章。

10.1 基于线性表的检索

数据放在数组或链表里，按给定值 K 比较，直到命中或确定不在表中。

10.1.1 顺序检索

从表头逐个比到表尾。元素可以无序。最好 1 次，最坏 n 次，等概率平均 $(n+1)/2$ 。

10.1.2 二分检索

表必须有序。每次取中点，相等则停，否则丢掉一半。半开区间 $[first, last)$ 避免无符号下标上 $mid - 1$ 下溢。比较次数是 $O(\log n)$ ，不适合链表。

10.1.3 分块检索

把表分成若干块，块间有序、块内可以无序。先在块的索引上定位，再在块内顺序查。是顺序与二分之间的折中。本章不另写未验证的分块实现。

```
// 算法 10.2: 无需修改输入容器的顺序检索。
inline std::optional<std::size_t> sequential_search(const std::vector<int>& values,
    ↪ int key) {
    for (std::size_t index = 0; index < values.size(); ++index) {
        if (values[index] == key) return index;
    }
    return std::nullopt;
}
```

```
// 算法 10.3: 半开区间避免 `mid - 1` 在无符号下标下溢。
inline std::optional<std::size_t> binary_search(const std::vector<int>&
↪ sorted_values, int key) {
    std::size_t first = 0;
    std::size_t last = sorted_values.size();
    while (first < last) {
        const std::size_t middle = first + (last - first) / 2;
        if (sorted_values[middle] == key) return middle;
        if (sorted_values[middle] < key) first = middle + 1;
        else last = middle;
    }
    return std::nullopt;
}
```

10.2 集合的检索

集合只关心「在不在」，不保留重复，也不保证顺序。交、并、差都可以建立在检索之上。

10.2.1 集合的数学特性

元素互异；属于关系是命题，不是位置。因此插入已存在的键、删除不存在的键都是可预期的失败，返回 `false`。

10.2.2 计算机中的集合

可以用线性表、二叉搜索树或散列表实现。本章的 `IntSet` 用线性表加顺序检索，把接口钉死；大规模时应换成后面的散列表。

```
// 代码 10.4、算法 10.5-10.7: 不重复整数集合。
class IntSet {
public:
    [[nodiscard]] bool insert(int value) {
        if (contains(value)) return false;
        values_.push_back(value);
        return true;
    }
    [[nodiscard]] bool erase(int value) {
        const auto found = sequential_search(values_, value);
        if (!found) return false;
        values_.erase(values_.begin() + static_cast<std::ptrdiff_t>(*found));
        return true;
    }
}
```

```

[[nodiscard]] bool contains(int value) const { return sequential_search(values_,
↪ value).has_value(); }
[[nodiscard]] IntSet intersection(const IntSet& other) const {
    IntSet result;
    for (int value : values_) if (other.contains(value))
        ↪ (void)result.insert(value);
    return result;
}
[[nodiscard]] bool includes(const IntSet& other) const {
    for (int value : other.values_) if (!contains(value)) return false;
    return true;
}
[[nodiscard]] std::size_t size() const noexcept { return values_.size(); }
private:
    std::vector<int> values_;
};

```

10.3 散列方法

散列用函数把关键码映射成槽位下标，期望平均 $O(1)$ 。它不保持有序，不适合范围查询。装载因子过高或散列不均匀时会退化。

10.3.1 散列函数

好的散列函数计算要简单，地址要均匀。本章实现了 ELFHash：逐字节移位异或，并用无符号字符，避免 `char` 的符号性进入计算。

```

// 算法 10.8: ELFhash, 逐字节处理, 不把 char 的符号性带入散列。
inline std::size_t elf_hash(const std::string& text) noexcept {
    std::size_t hash = 0;
    for (unsigned char character : text) {
        hash = (hash << 4U) + character;
        const std::size_t high_bits = hash & 0xF0000000U;
        if (high_bits != 0) hash ^= high_bits >> 24U;
        hash &= ~high_bits;
    }
    return hash;
}

```

10.3.2 开散列方法（拉链法）

每个槽挂一条链表，冲突的关键码串在同一条链上。删除简单，不会截断别的链。装载因子可以大于 1。本章主实现是闭散列，开散列不另写一份。

10.3.3 闭散列方法（开地址法）

冲突时在表内继续探测。线性探测就是「回家地址 + i」。删除后不能立刻把槽位改成空，否则会截断后方元素的探测链。

假设表长为 5，键 1、6、11 的散列地址都为 1，于是它们依次放在槽 1、2、3：

槽：	0	1	2	3	4
		空	1	6	11
				空	

删除键 1 后，若把槽 1 直接标为空，查找键 6 从槽 1 开始会误以为「从未插入过 6」并提前停止。墓碑表示「这里曾有元素，但探测链还要继续」。插入新键时可以复用第一个墓碑，但查重必须继续走到真正的空槽或找到同键为止。

散列地址	关键码
0	
1	and
2	
3	end
4	else
5	
6	if
7	begin
8	do
9	
10	go
11	for
12	array

散列地址	关键码
13	while
14	with
15	until
16	then
17	
18	repeat
19	
20	
21	
22	
23	
24	
25	

图 10.5 例 10.2 中关键码对应的散列表

散列表追求平均 O(1) 查找，前提是装载因子不过高且散列分布均匀；它不保持键的有序关系，不适合按范围遍历。

10.3.4 闭散列表的算法

先跑一遍：

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::search::HashTable table(5);
    if (!table.insert(1) || !table.insert(6) || !table.insert(11)) {
        std::cout << " 插入 1、6、11 失败\n";
        return 1;
    }

    std::cout << " 插入 1、6、11 后:\n";
    for (std::size_t index = 0; index < table.capacity(); ++index) {
        const auto slot = table.slot_at(index);
        std::cout << " 槽 " << index << ": ";
        if (slot.state == dsa::search::HashTable::SlotState::used) {
            std::cout << slot.key << '\n';
        } else if (slot.state == dsa::search::HashTable::SlotState::tombstone) {
            std::cout << " 墓碑\n";
        } else {
            std::cout << " 空\n";
        }
    }

    if (!table.erase(1)) {
        std::cout << " 删除 1 失败\n";
        return 1;
    }
    std::cout << " 删除 1 后仍能找到 6? " << (table.contains(6) ? " 是" : " 否") <<
    '\n';
    if (!table.insert(16)) {
        std::cout << " 插入 16 失败\n";
        return 1;
    }
    std::cout << " 插入 16 后槽 1 是 "
        << table.slot_at(1).key
        << " (复用了墓碑) \n";
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch10/search_hash \
    code/ch10/search_hash/demo.cpp -o /tmp/hash-demo
/tmp/hash-demo
```

插入 1、6、11 后：

槽 0：空

槽 1：1

槽 2：6

槽 3：11

槽 4：空

删除 1 后仍能找到 6？是

插入 16 后槽 1 是 16（复用了墓碑）

若把删除写成「标空」而不是「标墓碑」，第二行会变成「否」——测试里有两条具名断言守住这件事。

HashTable::home 先取绝对值再取模，所以负键也能进表。find_slot 遇到 empty 就停，遇到 tombstone 继续；insertion_slot 记下沿途第一个墓碑，确认后方没有同键后复用它。按键删除返回 bool。

// 算法 10.9–10.13：线性探测闭散列表，显式区分空、占用和墓碑。

```
class HashTable {
public:
    enum class SlotState { empty, used, tombstone };
    struct SlotView { int key; SlotState state; };

    explicit HashTable(std::size_t capacity) : slots_(capacity) {
        if (capacity == 0) throw std::invalid_argument("hash table capacity must be
            ↪ positive");
    }

    [[nodiscard]] bool insert(int key) {
        const auto target = insertion_slot(key);
        if (!target) return false;
        Slot& slot = slots_[*target];
        if (slot.state == SlotState::used) return false;
        slot = Slot{key, SlotState::used};
        ++size_;
        return true;
    }

    [[nodiscard]] bool contains(int key) const { return find_slot(key).has_value();
        ↪ }

    [[nodiscard]] bool erase(int key) {
        const auto found = find_slot(key);
        if (!found) return false;
        slots_[*found].state = SlotState::tombstone;
        --size_;
        return true;
    }

    [[nodiscard]] std::size_t size() const noexcept { return size_; }
```

```

[[nodiscard]] std::size_t capacity() const noexcept { return slots_.size(); }
[[nodiscard]] SlotView slot_at(std::size_t index) const {
    if (index >= slots_.size()) throw std::out_of_range("hash table slot");
    return SlotView{slots_[index].key, slots_[index].state};
}

private:
struct Slot { int key{0}; SlotState state{SlotState::empty}; };
[[nodiscard]] std::size_t home(int key) const noexcept {
    const auto magnitude = key >= 0 ? static_cast<long long>(key) :
        ↪ -static_cast<long long>(key);
    return static_cast<std::size_t>(magnitude) % slots_.size();
}
[[nodiscard]] std::optional<std::size_t> find_slot(int key) const {
    for (std::size_t step = 0; step < slots_.size(); ++step) {
        const std::size_t index = (home(key) + step) % slots_.size();
        const Slot& slot = slots_[index];
        if (slot.state == SlotState::empty) return std::nullopt;
        if (slot.state == SlotState::used && slot.key == key) return index;
    }
    return std::nullopt;
}
[[nodiscard]] std::optional<std::size_t> insertion_slot(int key) const {
    std::optional<std::size_t> first_tombstone;
    for (std::size_t step = 0; step < slots_.size(); ++step) {
        const std::size_t index = (home(key) + step) % slots_.size();
        const Slot& slot = slots_[index];
        if (slot.state == SlotState::used && slot.key == key) return index;
        if (slot.state == SlotState::tombstone && !first_tombstone)
            ↪ first_tombstone = index;
        if (slot.state == SlotState::empty) return first_tombstone ?
            ↪ first_tombstone : index;
    }
    return first_tombstone;
}
std::vector<Slot> slots_;
std::size_t size_{0};
};

```

10.3.5 散列方法的效率分析

成功检索的期望探测次数随装载因子 α 上升。线性探测大约是 $(1+1/(1-\alpha))/2$ ；拉链法是 $1 + \alpha/2$ 。 α 接近 1 时闭散列急剧变差，应扩表或改开散列。

10.3.6 散列方法的应用

编译器符号表、缓存、去重都常用散列。需要范围查询或有序遍历时，应改用树或有序表。

第 11 章 索引技术

索引不是保存完整记录的主文件，而是一张「关键码到记录位置」的查找表。它的目标是在海量外存记录中避免逐条扫描：先查较小的索引，再按位置读取目标记录。

原书本章没有独立的【算法】 / 【代码】清单，因此这里不提供未经验证的示意实现，也不伪造台账条目。下面是概念教程；实现练习留给需要持久化存储的上层项目。

11.1 线性索引

线性索引由按 key 排序的 (key, location) 项组成，可在内存中二分查找。稠密索引为每条记录建项，适合主文件无序或需要直接定位每条记录的情形；稀疏索引只为每个有序数据块建项，通常保存该块的最小 key，查到块后再在块内查找。

当第一层索引也过大时，可为它再建一层索引。一次查询先在内存中的顶层找到目标块，再读一个索引块，最后读数据块。多级索引的关键指标不是比较次数，而是磁盘页访问次数。

11.2 静态多分树

磁盘按页读写，因此一个索引结点通常设计为恰好装入一个页。与二叉树相比，多分树每个结点拥有更多孩子，树高更低，查询需要的页面访问次数更少。静态索引在批量装入后结构不变；频繁插入和删除会带来溢出区或重组成本。

11.3 倒排索引

倒排索引将属性值或词项映射到记录集合，而不是从记录正向读取属性。例如：

计算机系 -> [0310, 0330, 0341]
英语专长 -> [0310, 0421]

查询「计算机系且擅长英语」就是两个列表的交集，结果为 [0310]。全文检索也使用同一结构：词项映射到包含该词的文档编号和位置列表。倒排表提升查询速度，但每次插入、删除、更新记录都必须同时维护相关列表。主文件仍然必要，因为倒排表通常只保存标识和位置，不保存完整记录。

11.4 B 树与 B+ 树

B 树和 B+ 树都是保持平衡的多路搜索树。查找从根开始，在一个页内确定下一条分支；插入使结点溢出时分裂并向父结点上推分界 key；删除使结点上溢时先向兄弟借 key，不能借再合并。

B+ 树将记录（或记录位置）集中在叶结点，内部结点只作导航；叶结点按 key 链接。因此它既支持点查询，也适合 key between low and high 这类范围扫描。关系型数据库常使用它来组织磁盘索引。

11.5 位图、签名和红黑树

位图索引适合取值很少的属性。每个属性值维护一个位串，第 i 位表示第 i 条记录是否具有该值；AND、OR、NOT 查询直接变成机器字的按位运算。若属性取值很多，位图会很大，需要压缩。

签名文件把文档特征散列成位串，用较小的签名快速排除不可能匹配的文档，再回到原文确认；它可能有假阳性。红黑树则用于内存有序索引，通过颜色规则保持近似平衡，使查询、插入和删除均为 $O(\log n)$ 。它适合内存映射，不替代磁盘页导向的 B+ 树。

11.6 练习路径

1. 为变长记录实现二级稀疏索引，统计一次查询读取了几个页。
2. 为若干文档建立倒排表，完成 AND、OR、短语查询。
3. 用固定大小数组模拟 B+ 树页，测试分裂、借位、合并和范围扫描。
4. 为低基数列实现压缩位图，对比扫描主表和位图过滤的成本。

第 12 章 高级数据结构

本章包含两个主题。可利用空间表复用固定数量的槽位：申请取得一个空闲槽，释放把它归还。最优二叉搜索树则把访问频率写成权重，用动态规划比较每个区间可能的根，得到总查找代价最小的树。

源码：空闲槽池与最优 BST、可运行示例、测试。

12.1 多维数组

多维数组是「数组的数组」。元素个数相对固定，一旦生成只改值、不改相对位置，因此用顺序存储，按下标随机访问。

12.1.1 多维数组的存储

C++ 按行优先：先排最右下标。 $d_0 \times d_1 \times \cdots \times d_{n-1}$ 数组中元素 $A[j_0, \dots, j_{n-1}]$ 相对首地址的偏移是

$$d \cdot \left(\sum_{i=0}^{n-2} j_i \prod_{k=i+1}^{n-1} d_k + j_{n-1} \right)$$

每个元素定位时间相同，所以是随机存储结构。FORTRAN 按列优先，公式左右对调。

12.1.2 特殊矩阵

上/下三角矩阵和对角线对称矩阵都可以压成一维： n 阶下三角只需 $(n^2 + n)/2$ 个单元， $a_{i,j}$ ($i \geq j$) 落在 $\text{list}[(i^2 + i)/2 + j]$ 。对称矩阵只存一半，另一半用 $a_{i,j} = a_{j,i}$ 映射。

12.1.3 稀疏矩阵

非零元很少且分布不规则时， $\delta = t/(mn)$ 小于约 0.05 就按稀疏处理，改存三元组或十字链表，而不是整块二维数组。本章不另写未验证的十字链表实现。

12.2 广义表和存储管理

广义表的元素可以是原子，也可以是另一个表。纯表对应树，再入表对应有向无环图，循环表对应有环图。存储上常用头尾指针的结点；表共享时必须处理别名和回收。

12.2.1 广义表的定义和存储结构

表头是第一个元素，表尾是去掉表头后剩下的表。空表没有头尾。头尾表示法让递归算法自然对应「先处理头、再处理尾」。

12.2.2 可利用空间表

原书用重载 operator new/delete 和一条全局 avail 链实现结点复用。所有对象共享隐式全局状态，生命周期结束时还要 ::delete 整条链。现代实现改成显式的索引句柄池：acquire 从空闲栈弹出一个下标，release 把它推回去；耗尽返回 nullopt，重复/越界归还返回 false。释放后的下标失效，get 得到空指针。

普通二叉搜索树只要求中序遍历有序，树形可能很多。若键的查找频率不同，应把常查的键放得更靠近根。最优 BST 的输入包括成功查找权 $p[1..n]$ 与失败查找权 $q[0..n]$ ；动态规划对每个区间尝试每一个键作根，选择「左子树代价 + 右子树代价 + 本区间总权」最小的方案。

教材样例 $p = \{1, 5, 4, 3\}$ 、 $q = \{5, 4, 3, 2, 1\}$ 的总成本是 57，根是第 2 个键。cost[i][j] 是区间代价，root[i][j] 记录取得最小值的根，因而还能按根表重建树形。朴素实现为 $O(n^3)$ 。

先跑一遍：

```
#include "modern.hpp"

#include <iostream>

int main() {
    dsa::advanced::ReusableNodePool<int> pool(2);
    const auto first = pool.acquire(11);
    const auto second = pool.acquire(22);
    std::cout << " 申请到槽 " << *first << " 和 " << *second
                << ", 剩余 " << pool.available() << '\n';
    pool.release(*first);
    const auto reused = pool.acquire(44);
    std::cout << " 归还后再申请得到槽 " << *reused
                << ", 值为 " << *pool.get(*reused) << '\n';

    const auto tree = dsa::advanced::optimal_bst({1, 5, 4, 3}, {5, 4, 3, 2, 1});
    std::cout << " 最优 BST 总成本 " << tree.cost[0][4]
                << ", 根为键 " << tree.root[0][4] << '\n';
}
```

```
c++ -std=c++17 -Wall -Wextra -Werror -Icode/ch12/optimal_bst \
code/ch12/optimal_bst/demo.cpp -o /tmp/bst-demo
```

```
/tmp/bst-demo
```

```
申请到槽 0 和 1, 剩余 0  
归还后再申请得到槽 0, 值为 44  
最优 BST 总成本 57, 根为键 2
```

把 `optimal_bst({1,2}, {3,4})` 这种长度对不上的输入送进去, 会抛 `std::invalid_argument`。空树对应 `p = {}`、`q = {某个失败权}`, 代价为 0。

池用 `vector<optional<T>>` 表示槽位占用, `vector<size_t>` 当空闲栈。构造时下标从大到小入栈, 所以第一次 `acquire` 拿到 0。release 先确认该槽确实被占用, 再 `reset` 并归还。

```
template <typename T>
class ReusableNodePool {
public:
    explicit ReusableNodePool(std::size_t capacity) : slots_(capacity) {
        for (std::size_t index = 0; index < capacity; ++index) {
            free_.push_back(capacity - index - 1);
        }
    }

    [[nodiscard]] std::optional<std::size_t> acquire(const T& value) {
        if (free_.empty()) {
            return std::nullopt;
        }
        const std::size_t index = free_.back();
        free_.pop_back();
        slots_[index] = value;
        return index;
    }

    bool release(std::size_t index) {
        if (index >= slots_.size() || !slots_[index]) {
            return false;
        }
        slots_[index].reset();
        free_.push_back(index);
        return true;
    }

    [[nodiscard]] const T* get(std::size_t index) const noexcept {
        if (index >= slots_.size() || !slots_[index]) {
            return nullptr;
        }
    }
}
```

```

        return &*slots_[index];
    }

    [[nodiscard]] std::size_t available() const noexcept { return free_.size(); }

private:
    std::vector<std::optional<T>> slots_;
    std::vector<std::size_t> free_;
};

```

12.2.3 存储的动态分配和回收

向系统要一块、用完还回去，中间可能产生外部碎片（空闲块太碎）和内部碎片（分出去的块没用完）。首次适应、最佳适应、最坏适应是三种空闲块选择策略；相邻空闲块应合并。

12.2.4 失败处理策略和无用单元回收

分配失败可以：拒绝、压缩、或做垃圾回收。引用计数遇循环引用会漏；标记-清除能处理环，但要能从根集走到所有活对象。本章的句柄池把「失败」显式做成 `nullopt`，不劫持全局 `new`。

12.3 Trie 结构和 Patricia 树

Trie 按关键码的字符（或二进制位）逐层分支，公共前缀共享路径。适合字典和最长前缀匹配。Patricia 树把只有一个孩子的内部结点压缩掉，路径更短。二者都是原书没错的结构；本仓库没有单独的可运行实现，不在此印示意代码。

12.4 改进的二叉搜索树

12.4.1 最佳二叉搜索树

最优 BST 先校验 `q.size() == p.size() + 1`。空区间的 `cost[i][i] = 0`，`weight[i][i] = q[i]`。长度从 1 扩到 `n`，对每个区间 `[first, last]` 枚举根 `r`，候选代价是 `cost[first][r-1] + cost[r][last] + weight[first][last]`。

```

struct OptimalBstResult {
    std::vector<std::vector<long long>> cost;
    std::vector<std::vector<std::size_t>> root;
};

```

```

inline OptimalBstResult optimal_bst(const std::vector<int>& successful,
                                   const std::vector<int>& unsuccessful) {
    if (unsuccessful.size() != successful.size() + 1) {
        throw std::invalid_argument("weight count");
    }
    const std::size_t count = successful.size();
    OptimalBstResult result{
        std::vector<std::vector<long long>>(count + 1,
                                           std::vector<long long>(count + 1, 0)),
        std::vector<std::vector<std::size_t>>(count + 1,
                                           std::vector<std::size_t>(count + 1, 0)));
    std::vector<std::vector<long long>> weight(
        count + 1, std::vector<long long>(count + 1, 0));

    for (std::size_t index = 0; index <= count; ++index) {
        // The book's c table measures internal-key comparison cost; an empty
        // interval has zero c cost while its unsuccessful weight remains in w.
        result.cost[index][index] = 0;
        weight[index][index] = unsuccessful[index];
    }
    for (std::size_t length = 1; length <= count; ++length) {
        for (std::size_t first = 0; first + length <= count; ++first) {
            const std::size_t last = first + length;
            weight[first][last] = weight[first][last - 1] + successful[last - 1] +
                                unsuccessful[last];
            result.cost[first][last] = std::numeric_limits<long long>::max() / 4;
            for (std::size_t root = first + 1; root <= last; ++root) {
                const long long candidate = result.cost[first][root - 1] +
                                           result.cost[root][last] + weight[first]
↪ [last];
                if (candidate < result.cost[first][last]) {
                    result.cost[first][last] = candidate;
                    result.root[first][last] = root;
                }
            }
        }
    }
    return result;
}

```

12.4.2 平衡的二叉搜索树

AVL 树用平衡因子把左右高度差限制在 1 以内，插入删除后通过单旋转或双旋转恢复。查找、插入、删除都是 $O(\log n)$ 。红黑树是另一种近似平衡，见第 11.6 节。本章不另写未验证的 AVL 实现。

12.4.3 伸展树

访问一个结点后，用旋转把它搬到根附近：之字形走两步升一层，一字形走两步升两层。均摊 $O(\log n)$ ，不必在每个结点上存平衡信息。半伸展比全伸展少转一轮，适合连续访问同一结点。本章不另写未验证实现。

原书勘误

《数据结构与算法》（张铭、王腾蛟、赵海燕，高等教育出版社 2008）印刷清单里，按原样进编译器会失败，或对拍后算法结果确定是错的条目。每条都指向某个单元的 legacy.md，那里有命令和真实输出。

不收录：OCR 把 } 认成 1、把 ++ 拆成 + +、全角分号——那是扫描损伤，不是原书设计。也不收录「用了 cout」「没用 optional」这类现代化取舍。

底稿 dsa_raw.md 只读，本表是给对照纸书阅读的人用的。

怎么读

「印刷即错」指：把 OCR 噪声修回之后，逻辑或语法仍然不能通过现代 C++ 编译器，或与书中自己的约定对拍失败。现代实现写在 code/，书稿 book/ 印的是那份能跑的代码。

编译不过

清单	错在哪	证据
代码 2.1 / 代码 2.2 / 算法 2.5	成员函数名叫 delete，这是关键字	error: expected unqualified-id before 'delete' array_list
代码 2.1	class List 通篇没有 public:，ADT 的每个运算都是私有的	error: 'void List<T>::clear()' is private · 同上
算法 2.3	循环上界 n 从未声明	error: 'n' was not declared in this scope · 同上
代码 2.6	const Link* 赋给 Link* next，丢弃 const	error: assigning to 'Link<int> *' from 'const Link<int> *' · linked_list
代码 3.2	int top 与 bool top(T&) 重名，整类编译不过	error: 'bool arrStack<T>::top(T&)' conflicts with a previous declaration · array_stack

清单	错在哪	证据
算法 3.3	循环变量 i 从未声明	error: 'i' was not declared in this scope · 同上
代码 3.4	又是 Link<T>* top 与 bool top(T&) 重名	与代码 3.2 同一处错误，两个存储结构都犯 · linked_stack
算法 3.5 / 算法 3.9	s.pop(&tmp) 传指针，本书自己的栈 ADT 要的是引用	error: cannot convert 'double*' to 'double&' expression_eval
算法 3.11	enum rdType {0,1,2} (枚举值不能是字面量); public class knapNode (Java 语法，出现两次)	error: expected identifier before numeric constant / error: expected unqualified-id before 'public' knapsack knapsack
算法 3.12	同一个 stack.top 既当数据成员又当成员函数——依赖代码 3.2/3.4 的重名缺陷才可能存在	
算法 4.4	assert(str != '\0') 是指针与字符比较	error: ISO C++ forbids comparison between pointer and integer · string_class
算法 4.4	String(char* s) 使书中自己的例子 String s1 = "Hello" 在 C++11 起非法	字符串字面量是 const char* · 同上
代码 10.1	成员初始化写 key，数据成员却是 Key	error: member initializer 'key' does not name a non-static data member · search_hash

能编译、跑起来是错的

清单	错在哪	证据
算法 3.6 / 3.8 / 3.9	阶乘不查溢出： factorial(21) 返回负数， factorial(66) 返回 0	UBSan: signed integer overflow: 21 * 2432902008176640000 · recursion_and_stack
算法 3.6	负数静默返回 1	定义域错误，不是「空阶 乘」· 同上
算法 4.5	越界 return NULL，随后 strlen(nullptr)	运行期 SEGV · string_class
算法 4.6 / 算法 4.8	0 起始下标下返回 j - pLen + 1，一律差 1	四面数据对拍 std::string::find，含 图 4.12 自己的例子 · pattern_matching
算法 10.9	散列表析构写 delete HT， HT 是数组	应为 delete[] · search_hash

书内自相矛盾

位置	矛盾	说明
算法 3.5 正文 vs 代码	正文自己说 operand1 == 0.0 判断除零是错的，印 出来的代码照旧	见第 3 章表达式一节
代码 3.1 vs 算法 3.5 / 3.9	栈 ADT 是 pop(T&)，算法 写成 pop(&x)	两处清单从未放进同一个 翻译单元
第 4.3 节 vs 算法 4.6 / 4.8	正文明写下标从 0 起，返 回值却按 1 起始算	图 4.12 只画过程、没给返 回值，所以印出来没人发 现

反复出现、但单独一条通常还能「跑两下」

这些不是某一条清单独有，而是同一类所有权错误在多章重复。编译能过，拷贝一次就二次释放。

- 有析构、无拷贝构造 / 拷贝赋值：顺序表、链表、顺序栈、链式栈、字符串、最小堆、图的裸数组。
- 容器里 cout 报错、用 bool + 出参 表达空状态：第 2、3、4 章通病。
- int 当下标与容量：有符号溢出是未定义行为。

现代实现按 D-001 处理：显式五法则、`std::optional` 表达空、真错误抛标准异常。

曾经误判、已撤回

第 1 章股市传言的印刷矩阵里，B1 / B5 的最大最短路一度被读成 8，并据此声称「原书应返回 B1 而非正文的 B3」。那些 8 是 OCR 把 ∞ 认错；选起点用的图 1.3 是图片，从未被 OCR。按正文还原，原书这个样例是对的。详见 [adt/legacy.md](#)。

还没收进本表的

- 算法 2.11、代码 3.1、算法 7.6、算法 7.9 的结束标记被 OCR 吃掉，切片边界仍是手定 (T-008)。
- 第 6–12 章不少「标识符被空格切断」「全角分号」更像 OCR 而不是作者写错，故不列入上表。